# ESCUELA POLITÉCNICA NACIONAL

## FACULTAD DE INGENIERÍA DE SISTEMAS

## DISEÑO Y DESARROLLO DE UNA APLICACIÓN SOFTWARE, EN EL CONTEXTO DEL ENFOQUE ÁGIL, COMO SOPORTE A LA REDUCCIÓN DEL NÚMERO DE ANIMALES DE COMPAÑÍA SIN HOGAR EN EL DISTRITO METROPOLITANO DE QUITO

## DESARROLLO DEL BACK-END DE LA APLICACIÓN

**TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO EN SOFTWARE**

**GANDHY GABRIEL GARCÍA CASTAÑEDA**

**gandhy.garcia@epn.edu.ec**

**DIRECTOR: CARLOS EFRAÍN IÑIGUEZ JARRÍN**

**carlos.iniguez@epn.edu.ec**

**DMQ, febrero 2024**

# CERTIFICACIONES

Yo, GANDHY GABRIEL GARCÍA CASTAÑEDA declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

_____

GANDHY GABRIEL GARCÍA CASTAÑEDA

Certifico que el presente trabajo de integración curricular fue desarrollado por GANDHY GABRIEL GARCÍA CASTAÑEDA, bajo mi supervisión.

_____

Carlos Efraín Iñiguez Jarrín

DIRECTOR

# DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

GANDHY GABRIEL GARCÍA CASTAÑEDA

Carlos Efraín Iñiguez Jarrín

# DEDICATORIA

Dedico este trabajo a mi familia: Enrique, Estrellita, Alejandro, Dayana, por su apoyo incondicional en todo mi proceso de desarrollo profesional. También dedico el trabajo a Blanca por estar conmigo en los momentos buenos y no tan bueno. Todas estas personas me han brindado su amor, el cual me brindó fuerza para alcanzar este hito en mi vida.

# AGRADECIMIENTO

Agradezco a todos mis amigos y compañeros de la universidad, en especial a Erick y Fernando por su acompañamiento y apoyo a lo largo de toda la carrera y en esta etapa final de la misma. He aprendido mucho de ellos y espero que ellos también de mí. También agradezco a mi tutor, Carlos Íñiguez, por su paciencia y su sabiduría en todo el proceso de desarrollo de este proyecto. Gracias a él he podido aprender y desafiarme en cuanto a mis conocimientos técnicos. Finalmente agradecer a los docentes que me han impartido sus conocimientos en una o varias ocasiones, gracias a ellos he llegado a este punto y me han enseñado que además de formarme en el aspecto técnico, debo formarme en el aspecto humano. Gracias de corazón.

# ÍNDICE DE CONTENIDO

CERTIFICACIONES ..... I
DECLARACIÓN DE AUTORÍA ..... II
DEDICATORIA ..... III
AGRADECIMIENTO ..... IV
ÍNDICE DE CONTENIDO ..... V
ÍNDICE DE FIGURAS ..... VIII
ÍNDICE DE TABLAS ..... X
RESUMEN ..... XI
ABSTRACT ..... XII
1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO ..... 1
1.1 Objetivo general ..... 2
1.2 Objetivos específicos ..... 2
1.3 Alcance ..... 3
1.4 Marco teórico ..... 3
1.4.1 Back-end en el desarrollo de aplicaciones ..... 3
1.4.2 Componentes del Backend ..... 4
1.4.3 Arquitectura del back-end ..... 8
1.4.4 Framework de desarrollo y despliegue ..... 11
1.4.5 SCRUM ..... 13
2 METODOLOGÍA ..... 18
2.1 Sprint 0 ..... 18
2.2 SPRINT 1 ..... 21
2.2.1 Introducción ..... 21
2.2.2 Planificación ..... 22
2.2.3 Implementación ..... 23
2.2.4 Revisión ..... 29

2.2.5 Retrospectiva.....30
2.3 SPRINT 2 ..... 30
2.3.1 Introducción.....30
2.3.2 Planificación .....30
2.3.3 Implementación .....32
2.3.4 Revisión.....40
2.3.5 Retrospectiva.....41
2.4 SPRINT 3 ..... 43
2.4.1 Introducción.....43
2.4.2 Planificación .....43
2.4.3 Implementación .....44
2.4.4 Revisión.....47
2.4.5 Retrospectiva.....47
2.5 SPRINT 4 ..... 48
2.5.1 Introducción.....48
2.5.2 Planificación .....48
2.5.3 Implementación .....49
2.5.4 Revisión.....52
2.5.5 Retrospectiva.....52
2.6 SPRINT 5 ..... 53
2.6.1 Introducción.....53
2.6.2 Planificación .....53
2.6.3 Implementación .....55
2.6.4 Revisión.....61
2.6.5 Retrospectiva.....62
2.7 SPRINT 6 ..... 62
2.7.1 Introducción.....62
2.7.2 Planificación .....62

2.7.3 Implementación .....63
2.7.4 Revisión.....64
2.7.5 Retrospectiva.....64
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES..... 65
3.1 Resultados ..... 65
3.2 Conclusiones..... 69
3.3 Recomendaciones..... 70
4 REFERENCIAS BIBLIOGRÁFICAS ..... 71
5 ANEXOS..... 78
ANEXO I. Tabla comparativa de herramientas para el desarrollo del Backend 78
ANEXO II. Tabla comparativa de Bases de Datos..... 82
ANEXO III. Tabla comparativa de herramientas de Control de Versiones..... 82
ANEXO IV. Tabla comparativa de herramientas de Gestión de Proyectos..... 84
ANEXO V. Tabla comparativa de IDE's o Editores de código..... 85
ANEXO VI. Tabla comparativa de servicios de Hosting..... 86
ANEXO VII. Enlace API ..... 88
ANEXO VIII. Enlace Proyecto Notion..... 88

# ÍNDICE DE FIGURAS

**Figura 1.** División por capas de la arquitectura cebolla [27]..... 9
**Figura 2.** Ejemplo de arquitectura hexagonal [29] .....10
**Figura 3.** Código python para una API básica con FastAPI [31] .....11
**Figura 4.** Diagrama de clases del componente del Backend .....24
**Figura 5.** Diagrama de arquitectura Backend .....25
**Figura 6**. Diagrama de secuencia para "Crear publicación de adopción" .....27
**Figura 7.** Diagrama de secuencia para "Subir Foto".....28
**Figura 8.** Diagrama de secuencia para "Visualizar publicaciones de adopciones" .....29
**Figura 9.** Burndown chart Sprint 1.....29
**Figura 10.** Diagrama de secuencia para "Visualizar información de parroquias" .....33
**Figura 11.** Diagrama de secuencia para "Crear publicación de experiencia".....34
**Figura 12.** Diagrama de secuencia para "Visualizar publicaciones de experiencias".....35
**Figura 13.** Diagrama de secuencia para "Visualizar publicaciones favoritas" .....36
**Figura 14.** Diagrama de secuencia para "Agregar publicación en favoritos".....37
**Figura 15.** Diagrama de secuencia para "Eliminar publicación de favoritos" .....37
**Figura 16.** Diagrama de secuencia para "Obtener información del juego 'Quiz' " .....38
**Figura 17.** Diagrama de secuencia para "Guardar información de partida del juego 'Quiz' " .....39
**Figura 18.** Diagrama de secuencia para "Obtener información de la tabla de posiciones del juego 'Quiz' ".....40
**Figura 19.** Burndown chart Sprint 2.....41
**Figura 20.** Tablero Sprint Retrospective (Sprint 2) .....42
**Figura 21.** Diagrama de secuencia para "Visualizar información de juegos" .....45
**Figura 22.** Diagrama de secuencia para "Visualizar información de un usuario" .....45
**Figura 23.** Diagrama de secuencia para "Visualizar mis publicaciones de adopción".....46
**Figura 24.** Diagrama de secuencia para "Actualizar perfil de usuario" .....47
**Figura 25.** Burndown chart Sprint 3.....47
**Figura 26.** Diagrama de secuencia para "Dar Like a una publicación" .....50
**Figura 27.** Diagrama de secuencia para "Eliminar Like de una publicación" .....51
**Figura 28.** Diagrama de secuencia para "Agregar comentario en una publicación".....51
**Figura 29.** Diagrama de secuencia para "Visualizar los comentarios de una publicación" .....52
**Figura 30.** Burndown chart Sprint 4.....52

**Figura 31.** Diagrama de secuencia para "Obtener información del juego 'Adivina la palabra' " .....56
**Figura 32.** Diagrama de secuencia para "Guardar información de partida del juego 'Adivina la palabra' " .....57
**Figura 33.** Diagrama de secuencia para "Obtener información del juego 'Sopa de letras' " .....57
**Figura 34.** Diagrama de secuencia para "Regisrar usuario" .....58
**Figura 35.** Diagrama de secuencia para "Iniciar sesión" .....59
**Figura 36.** Diagrama de secuencia para "Obtener información del usuario actual" .....60
**Figura 37.** Diagrama de secuencia para "Visualizar información de organizaciones" .....61
**Figura 38.** Burndown chart Sprint 5 .....62
**Figura 39.** Burdnown chart Sprint 6 .....64
**Figura 40.** Interfaz gráfica de la API (Swagger UI) .....67
**Figura 41.** Base de datos del proyecto implementada en MongoDB (Vista desde MongoDB Compass) .....68

# ÍNDICE DE TABLAS

**Tabla 1.** División de tareas del Sprint 1 .....22
**Tabla 2.** División de tareas del Sprint 2 .....31
**Tabla 3.** División de tareas del Sprint 3 .....43
**Tabla 4.** División de tareas del Sprint 4 .....48
**Tabla 5**. División de tareas del Sprint 5 .....53
**Tabla 6.** División de tareas del Sprint 6 .....62
**Tabla 7.** Endpoints de la API expuesta por el Backend* .....65

# RESUMEN

El presente documento describe el desarrollo del componente Back-End de una aplicación móvil que busca apoyar el proceso de adopción de animales de compañía (perros y gatos) en el Distrito Metropolitano de Quito, con el objetivo de disminuir el número de animales en las calles. Este componente implementa todas las funcionalidades de la aplicación a través de una API REST y sus respectivos endpoints, incluyendo la interacción con una base de datos no relacional que almacena los datos de la aplicación. Para el diseño del componente se implementan principios de dos arquitecturas limpias: "hexagonal" y "por capas", además del uso de tecnologías y herramientas como Python, FastAPI, MongoDB y Firebase. El desarrollo del componente sigue las pautas del marco de trabajo SCRUM, dividiendo el trabajo en seis sprints. El documento detalla el proceso de cada sprint, desde sus objetivos, su planificación, su implementación, la revisión y la retrospectiva. El resultado es una API que expone todas las funcionalidades de la aplicación mediante endpoints que pueden ser accedidos desde un cliente (por ejemplo, Front-End).

**PALABRAS CLAVE:** Adopción de animales, Arquitecturas limpias, SCRUM, Python, FastAPI

# ABSTRACT

This document describes the development of the Back-End component of a mobile application that seeks to support the adoption process of companion animals (dogs and cats) in the Metropolitan District of Quito, with the aim of reducing the number of these animals on the streets. This component implements all the functionalities of the application through a REST API and its respective endpoints, including interaction with a non-relational database that stores the application's data. The component design implements principles of two clean architectures: "hexagonal" and "layered", in addition to the use of technologies and tools such as Python, FastAPI, MongoDB, and Firebase. For its development, the guidelines of the SCRUM framework are followed, dividing the work into six sprints. The document details the process of each sprint, from its objectives, planning, implementation, review, and retrospective. The result is an API that exposes all the functionalities of the application through endpoints that can be accessed from a client (for example, Front-End).

**KEYWORDS:** Animal adoption, Clean Architectures, SCRUM, Python, FastAPI

# 1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO

El crecimiento descontrolado de animales callejeros (animales que se encuentran en la vía pública, sin dueño y sin hogar), especialmente de animales de compañía (perros y gatos) es considerado un problema de salud pública en varias partes del mundo, con un impacto socioeconómico, político y de bienestar animal en la sociedad [1] [2] [3].

En el Distrito Metropolitano de Quito, Ecuador, un censo realizado en el 2022 por la Unidad de Vigilancia e Investigación de la "Unidad de Bienestar Animal" revela que existen 775005 perros y 554785 gatos abandonados, es decir, más de un millón de estos animales abandonados en las calles [4]. Frente a esta problemática, el Municipio de Quito destaca dos directrices de solución [5]:

1. la importancia de la participación ciudadana en programas de adopción responsable.
2. la educación y concientización sobre la tenencia responsable de animales de compañía.

Por otro lado, es importante destacar el impacto que las redes sociales tienen sobre la adopción de mascotas. Gracias a las redes sociales, las fundaciones de rescate atraen la atención de las personas que desean adoptar y realizar donaciones de dinero o alimento para los animales rescatados. Si bien las adopciones mediante espacios digitales pueden tener ventajas como la difusión y la facilidad de comunicación entre sus usuarios, también tienen desventajas como la entrega de animales sin esterilización o sin su vacunación respectiva, por lo que el uso de las redes sociales para este propósito se lo debe hacer de forma responsable y consciente [6].

En este sentido y considerando el comportamiento ciudadano, para abordar el problema de los animales de compañía sin hogar en el Distrito Metropolitano de Quito, el presente proyecto propone el desarrollo de una aplicación móvil como mecanismo para facilitar:

A) la comunicación entre personas que buscan un hogar para los animales de compañía y aquellas que desean adoptarlos
B) la difusión de información relacionada a la tenencia responsable de mascotas con el fin de informar, concienciar y educar a la ciudadanía

Por tanto, la aplicación móvil contempla el desarrollo de dos módulos funcionales:

1. Módulo de red social, donde los usuarios podrán:
    a. publicar y visualizar información sobre animales en adopción

    b. discutir y compartir ideas sobre casos de animales en adopción
    c. comunicarse con otros usuarios para poder iniciar un proceso de adopción
2. Módulo de concientización, que busca, mediante juegos educativos (gamificación), educar a la ciudadanía sobre la tenencia responsable de mascotas [7]. Esto permitirá a los usuarios aprender sobre la tenencia responsable de una manera entretenida y participativa. En este módulo los usuarios podrán:
    a. aprender sobre las temáticas mediante juegos simples (con elementos como puntaje, tiempo limitado y retos)
    b. evaluar sus conocimientos y compararlos con los de otros usuarios (mediante una tabla de posiciones, o “leaderboard”)

Con estas funcionalidades, se busca implementar un espacio centralizado de comunicación que sirva como soporte para los procesos de adopción de animales sin hogar y para la concientización de la sociedad quiteña sobre el cuidado de los animales de compañía.

Para el funcionamiento de la aplicación móvil, es necesario implementar la capa del Back-End, es decir, la lógica de servidor que gestione el acceso a los datos y exponga las funcionalidades para ser consumidas por la aplicación móvil (Front-End). Para ello, el Back-End debe incluir los siguientes elementos:

A) base de datos de la aplicación
B) controladores para los diferentes casos de uso de la aplicación
C) una API que sirva como proveedor de funcionalidades para el Front-End

En consecuencia, el presente componente propone el diseño y desarrollo de la capa Back-End de la aplicación móvil, teniendo en cuenta la lógica de negocio de los módulos de red social y de gamificación mencionados, aplicando las tecnologías pertinentes para la naturaleza de este tipo de aplicación.

## 1.1 Objetivo general

Desarrollar el Back-End de la aplicación incluyendo la implementación de la lógica de negocio, arquitectura y la exposición de una API para consumo de los dispositivos clientes (Front-end).

## 1.2 Objetivos específicos

1. Analizar los requisitos funcionales de la aplicación.
2. Diseñar la arquitectura de software del backend de la aplicación.

3. Implementar la Base de Datos que almacene los datos de la aplicación.
4. Exponer la lógica de negocio y las funcionalidades a través de una API.

## 1.3 Alcance

El alcance de este componente es el diseño y desarrollo de la capa del Back-End de la aplicación. Para esto, se seguirá SCRUM como marco de trabajo ágil para gestionar las tareas del proyecto, incluyendo los roles de Product Owner, Scrum Master y “Team”.

Para la implementación de este marco de trabajo, se seguirán las principales pautas definidas en la guía SCRUM adaptándolas a la realidad del equipo de trabajo de este proyecto, en resumen, las actividades a realizar serán [7] [8]:

- Definir un Product Backlog para identificar la lista de características y funcionalidades requeridas para la aplicación.
- Implementar las características del Product Backlog en iteraciones (6 Sprints).
- Coordinar actividades mediante reuniones de planificación (Sprint Planning) y definir un subconjunto del Product Backlog para cada sprint (Sprint Backlog).
- Comunicar avances del equipo de desarrollo mediante reuniones diarias (Daily Scrum)
- Revisar el avance general de cada sprint mediante una reunión (Sprint Review) para evaluar los resultados obtenidos
- Analizar potenciales mejoras en el equipo de trabajo para futuras iteraciones mediante una reunión (Sprint Retrospective).

Los artefactos finales serán la API expuesta para su uso por la aplicación del Front-End y la documentación del desarrollo del proyecto (escrito final, diagramas de clases, de arquitectura, de secuencia, de dominio).

## 1.4 Marco teórico

### 1.4.1 Back-end en el desarrollo de aplicaciones

El “front-end” y “back-end” son dos aspectos importantes al desarrollar una aplicación software. Mientras el front-end, denominado también “cliente”, se encarga de presentar al usuario la información de forma visual e interactiva, utilizando elementos propios de la interfaz de usuario (ej., botones, listas, casillas de verificación), el back-end, denominado también “servidor”, se encarga de gestionar los datos y proporcionar las funcionalidades que el front-end muestra [9]. En este apartado se describe las particularidades en el desarrollo del back-end.

Desarrollar el back-end involucra principalmente tomar decisiones importantes sobre la arquitectura que soportará la aplicación de lado del servidor. Estas decisiones serán claves para posteriormente, enfocarse en la implementación de bases de datos, operaciones de la aplicación (funcionalidades) y APIs [9] [10] [11].

En el ámbito del proyecto actual, “diseño y desarrollo de una aplicación software como soporte a la reducción del número de animales de compañía sin hogar en el Distrito Metropolitano de Quito”, el componente materia de este documento se enfoca en la implementación de funcionalidades y gestión de base de datos de la aplicación, además de exponer dichas funcionalidades a través de una API para consumo por parte del front-end.

A continuación, se describe en detalle los conceptos de los componentes involucrados en el desarrollo del back-end.

### 1.4.2 Componentes del Backend

#### *1.4.2.1 API*

De sus siglas “Application Programming Interface”, una API es “un conjunto de reglas definidas que permiten la comunicación entre diferentes aplicaciones. Actúa como una capa intermedia que procesa la transferencia de datos entre sistemas” [12].

En la actualidad, la mayoría de APIs son “Web APIs”, que se encargan de exponer los datos y las funcionalidades de una aplicación mediante el internet. Para hacer uso de estas APIs, se han desarrollado diferentes protocolos (reglas o “especificaciones de la API”) para la manipulación de los datos o el uso de funcionalidades de las aplicaciones. Entre estos protocolos, resaltan SOAP (Simple Object Access Protocol), XML-RPC (XML-Remote Procedure Call), JSON-RPC (JSON-Remote Procedure Call) y REST (Representational State Transfer). Este último es un conjunto de principios de arquitectura para APIs web [12].

#### *1.4.2.2 REST API*

Las APIs modernas, en su mayoría, se adhieren a los principios REST y al formato de datos JSON. Por tanto, dentro de los diferentes tipos de APIs (según el protocolo que usan), las API REST (o RESTful) resaltan por su popularidad y por permitir la comunicación entre dos sistemas de manera segura, confiable y eficiente [12] [13].

“REST” es un tipo de arquitectura de software que indica las condiciones que una API debe seguir para considerarse API REST, entre ellas: la API debe implementar una arquitectura cliente-servidor que gestione las solicitudes a través de HTTP, la comunicación cliente-

servidor debe ser sin estado (solicitudes independientes una de otra), los datos deben poder almacenarse en caché para optimizar las solicitudes y mantener una interfaz uniforme entre los elementos para que todos los datos enviados y recibidos tengan una estructura estandarizada [14] [13].

En [13] se listan los beneficios de implementar REST APIs:

- Los sistemas que usan este tipo de APIs pueden escalar eficientemente debido a que REST optimiza la comunicación entre cliente y servidor.
- El uso de servicios RESTful permite mantener la independencia entre cliente y servidor e incluso desacoplan componentes del lado del servidor. Esto evita que un cambio en la tecnología del servidor afecte a la aplicación o componente del cliente.
- Las REST APIs no dependen de la tecnología usada tanto en el cliente como el servidor. La comunicación entre ambos no se verá afectada.

Debido a estos beneficios, en específico la independencia de la tecnología, el desarrollo de la API para el back-end del presente componente seguirá la lógica y principios de las REST APIs.

#### *1.4.2.3 Endpoints*

En el contexto de las APIs, los endpoints son “divisiones” o “secciones” de la API a donde se puede apuntar (interactuar) para realizar solicitudes desde una aplicación. Al interactuar con los endpoints es posible consumir los datos manejados por la API. En general, cada endpoint tiene su propia URL [15].

En el contexto de las REST API, la comunicación entre front-end y back-end se realiza a través de solicitudes HTTP a los endpoints de una API. Estas solicitudes incluyen diferentes métodos HTTP [16]:

- GET: Usado para obtener datos
- HEAD: Similar a GET, pero sin recibir el cuerpo de la respuesta, es decir, solo su cabecera
- POST: Permite la creación de nuevos recursos
- PUT: Permite realizar cambios en los recursos
- DELETE: Usado para eliminar recursos
- CONNECT: Establece un túnel hacia el servidor para comunicarse con él
- OPTIONS: Describe las opciones de comunicación hacia un recurso
- TRACE: Realiza una prueba de mensaje de retorno (loop-back) a lo largo de la ruta hacia un recurso

- PATCH: Permite aplicar cambios parciales a un recurso

En este sentido, los endpoints de la REST API serán los puntos de conexión para que el front-end pueda hacer uso de las funcionalidades expuestas por la API y los métodos HTTP usados en la comunicación dependerán de la finalidad de la solicitud (por ejemplo: creación de usuarios, creación de publicaciones, agregar comentarios, entre otras).

#### *1.4.2.4 Lenguaje de servidor: Python*

Del lado del servidor web (Back-end), existen diferentes lenguajes de programación para desarrollar la lógica de negocio, entre los cuales destaca Python. Python es un lenguaje de programación interpretado, orientado a objetos (aunque soporta diferentes paradigmas como la programación funcional), con tipado dinámico, ampliamente utilizado en aplicaciones web, eficiente y fácil de aprender [17] [18].

Para este componente, se ha seleccionado Python debido a que posee diferentes bibliotecas que apoyan a las funciones que debe realizar el back-end de una aplicación como, por ejemplo, la comunicación con la base de datos, la conexión con otros sitios web o la implementación de las funcionalidades de la aplicación [17].

#### *1.4.2.5 Base de datos: MongoDB*

Una base de datos es una colección diseñada, organizada y gestionada de datos. Se encarga de almacenar los datos de una aplicación a manera de respaldo o persistencia de estos y permite su acceso, creación, modificación y eliminación [19] [20].

Las bases de datos pueden ser clasificadas en relacionales y no relacionales. Las relacionales organizan los datos en grupos de datos llamados “tablas” o “relaciones”. Las tablas contienen filas y columnas, donde cada fila corresponde a los registros (objetos o instancias) y las columnas corresponden a los atributos de los objetos [19] [20]. Por otro lado, las bases de datos no relacionales (o también llamadas “bases de datos NoSQL”) organizan los datos de manera diferente a las bases de datos relaciones, de hecho, la estrategia usada para el almacenamiento de datos es específica a la estructura de la base de datos. En general, las bases de datos no relacionales no usan el lenguaje SQL para interactuar con los datos [20].

En el contexto de las bases de datos NoSQL existen diferentes enfoques, dentro de los cuales resaltan las bases de datos basadas en colecciones y documentos (grupos de pares “clave: valor” que almacenan la información de algún registro) y las basadas en grafos (la información se almacena en “nodos”, los cuales se enlazan o relacionan mediante “enlaces”) [20].

MongoDB es una de las bases de datos NoSQL más populares. MongoDB es una base de datos basada en documentos y de código abierto. Su elemento más básico es el documento, el cual es una estructura de datos compuesto por pares “clave: valor”, similar a la estructura de los objetos JSON. Los valores inmersos en los documentos pueden incluir datos, otros documentos, arreglos o arreglos de documentos [21]. La página oficial de MongoDB resalta algunas de las ventajas de trabajar con el documento como elemento básico [21]:

- Los documentos se relacionan con tipos de datos nativos en varios lenguajes de programación
- Los documentos embebidos (documento dentro de otro) y arreglos de datos permiten omitir el uso de operaciones de “JOIN” complejas
- El esquema dinámico que soportan permite el polimorfismo

Para el componente que se desarrollará en este proyecto, se ha escogido a MongoDB como motor de base de datos debido a su flexibilidad y las ventajas expuestas anteriormente. Además, la estructura de los documentos similar a JSON se ajusta con la estructura de los objetos que se manejan en los cuerpos de las solicitudes en la API.

#### *1.4.2.6 Servicio de almacenamiento: Firebase Cloud Storage*

Firebase es una plataforma que proporciona herramientas para el desarrollo de aplicaciones, desde bases de datos, hosting, compilación e incluso servicio de almacenamiento. Esta última opción es usada en la aplicación para el almacenamiento de las imágenes en la aplicación, tanto de usuarios como de publicaciones u organizaciones (los datos de la aplicación se guardan en la base de datos Mongo, pero las imágenes no). Firebase Cloud Storage está basado en la nube de Google (Google Cloud) y se utiliza para guardar y recuperar contenido multimedia [22].

#### *1.4.2.7 Autenticación: JWT – JOSE*

JWT (JSON Web Token) es un estándar que define un método para transmitir de forma segura información entre partes codificadas con un objeto de tipo JSON. En el presente componente es usado para el proceso de autenticación de los usuarios y el manejo de las sesiones. Un JWT contiene “reclamaciones” (“claims”), las cuales son fragmentos de información relacionadas con un contexto, generalmente representadas como pares “clave-valor” [23].

En procesos de autenticación de sistemas web, se utiliza JWT para que una vez iniciado sesión, el sistema retorne un token al usuario, este token, al estar relacionado con sus credenciales (en forma de reclamaciones) permite autenticarlo y autorizarlo para las

funcionalidades de un sistema. En posteriores llamadas al sistema web, el token será incluido en la cabecera de las solicitudes (cabecera “Authorization”) y de esta manera el usuario podrá acceder a dichas funcionalidades. La ventaja de usar JWT es su simplicidad de complejidad y tamaño (en comparación con “Simple Web Token – SWT” o “Security Assertion Markup Language – SAML”) lo que lo convierte en una buena opción para entornos HTML y HTTP [23].

En este componente se utilizó la librería de python “JOSE” (JavaScript Object Signing and Encryption) que permite implementar la tecnología de JWT, desde la configuración de los tokens (establecer reclamaciones y cifrado) hasta la generación de estos. Su selección se debe a que permite realizar todo el proceso de cifrado y descifrado de los JWT de forma sencilla y con total configuración de los parámetros del proceso [24].

### 1.4.3 Arquitectura del back-end

Una buena práctica en el desarrollo de aplicaciones es mantener una estructura para los componentes a desarrollar en el back-end. En la práctica, el back-end en sí es un componente de software.

Robert Martin define a la arquitectura de un sistema de software como “La forma dada a un sistema por aquellos que lo construyeron. Dicha forma se basa en la división del sistema en componentes, su disposición y cómo se comunican entre sí” [25].

Para el Backend, se ha extraído la filosofía de “dividir el código por capas”; filosofía que comparten dos arquitecturas: Hexagonal y por capas (o cebolla). Esto permitirá eliminar la dependencia de la tecnología en la que se implemente el sistema. La **Figura 1** muestra la arquitectura del componente en la cual se puede ver la división por capas (basada en la arquitectura cebolla).

Ambas arquitecturas se engloban en un grupo de arquitecturas llamadas “arquitecturas limpias” (en inglés, “Clean Architectures”), las cuales constituyen un modelo de arquitectura de software que se basa en la separación de responsabilidades y la separación del sistema en capas. Además, los sistemas resultantes son independientes de frameworks, interfaces de usuario, bases de datos y de cualquier dominio de negocio [25].

Si bien la implementación de una arquitectura “limpia” puede variar en las capas, generalmente se trabaja con las siguientes (desde la capa más “interna” hasta la más “externa”, en donde las capas externas son dependientes exclusivamente de sus capas internas) [26]:

- Capa de dominio: Contiene toda la lógica del dominio de la aplicación, incluyendo reglas de negocio, entidades, procedimientos, entre otros.
- Capa de aplicación: Abarca los casos de uso o tareas específicas de la aplicación y es la encargada de gestionar la comunicación entre la capa de dominio, la capa de infraestructura y la interfaz de usuario.
- Capa de infraestructura: Constituye la implementación técnica de las funcionalidades que permitan a la aplicación cumplir con sus objetivos. Incluye elementos de red, base de datos y cualquier otro aspecto técnico que requiera el sistema.

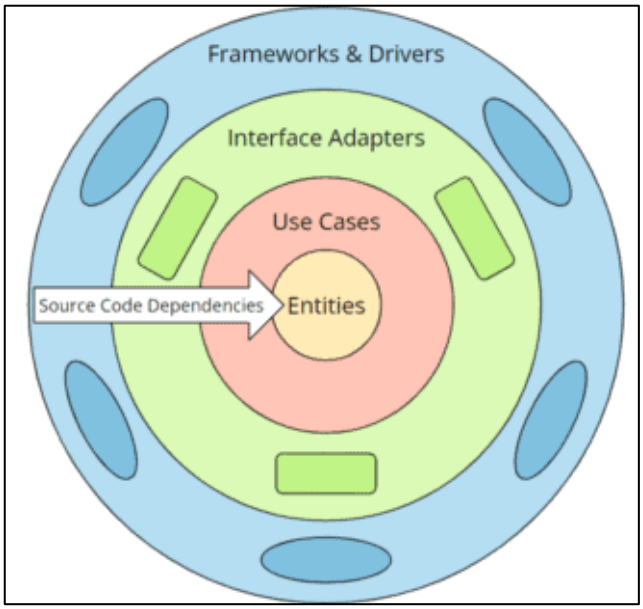


**Figura 1.** División por capas de la arquitectura cebolla [27]

El color amarillo hace referencia a la capa de dominio, el rojo a la capa de aplicación y el azul a la capa de infraestructura. El color verde es el enlace entre la lógica de la aplicación (funcionalidades) y las dependencias externas (infraestructura).

La arquitectura hexagonal, también llamada “Puertos y Adaptadores”, se caracteriza por dividir el modelo de dominio de la infraestructura que la rodea. En otras palabras, busca separar la lógica del negocio de la tecnología usada. Esta arquitectura se representa por un hexágono como se muestra en la **Figura 2**, cuyo núcleo contiene al sistema (aplicación). Fuera del hexágono, existen los “puertos”, que representan las interfaces con las que la aplicación interactúa con otros sistemas (UI, interfaces de red, bases de datos), mientras que los “adaptadores” sirven como una “conexión” entre el núcleo del hexágono y su exterior [26] [28].

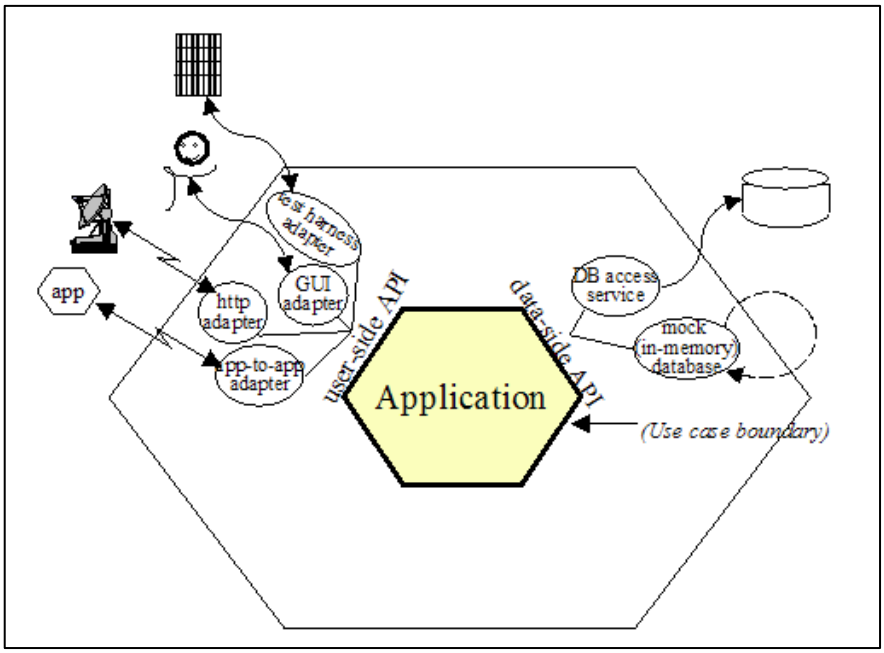


**Figura 2.** Ejemplo de arquitectura hexagonal [29]

La arquitectura cebolla está basada en 4 capas, las cuales son: las 3 capas presentadas en la definición de arquitectura “clean”, y adicionalmente la capa de presentación, que se encarga de gestionar la validación de datos, su presentación al usuario, y todo lo relacionado a la interfaz de usuario. Esta última capa se comunica con la capa de aplicación [26].

De la arquitectura hexagonal, se ha aplicado el enfoque de aislar la lógica de la aplicación de las dependencias externas, como conexiones a bases de datos o la propia tecnología a utilizar (Ejemplo: FastAPI), es decir, elementos de la capa de infraestructura (azul). Aunque no se implementaron “puertos” y “adaptadores” explícitamente, éstos pueden ser análogos a las capas de aplicación y dominio de la arquitectura “cebolla”.

Por otro lado, se implementaron las capas definidas por la arquitectura cebolla (a excepción de la capa de presentación, pues todo lo relacionado a interfaz de usuario será desarrollado en el componente del front-end). En consecuencia, la división del código se ha hecho en una capa de *dominio* (sección roja en la **Figura 1**), que contendrá todas las entidades básicas de la lógica del negocio (y las interfaces que definan el acceso a las entidades externas de manera abstracta); una capa de *aplicación*, que se constituirá por todos los “casos de uso” o funcionalidades principales relacionados con cada entidad; y, finalmente, una capa de *infraestructura* que tendrá todas las implementaciones (de las interfaces de la capa de dominio) que son dependientes de la tecnología (acceso a base de datos, endpoints de la API, autenticación). Es decir, con el uso de este tipo de arquitectura por

capas, bastará con modificar la capa de infraestructura si se desea cambiar de tecnología (por ejemplo, un framework diferente a FastAPI para la implementación de los endpoints).

Además, es importante indicar que, para construir un componente de software con este tipo de arquitecturas, es necesario aplicar patrones de diseño, inyección de dependencias, y, sobre todo, los principios de diseño SOLID [25].

Como conclusión de esta sección, en este componente se ha optado por usar los conceptos de las dos arquitecturas mencionadas, especialmente de la arquitectura cebolla, con el objetivo de desacoplar la tecnología de las funcionalidades principales y de las entidades (dominio) del negocio. De esta forma, si se necesita cambiar de tecnología, solamente se deberían aplicar cambios en una capa (infraestructura).

### 1.4.4 Framework de desarrollo y despliegue

#### *1.4.4.1 FastAPI*

Al trabajar con Python encontramos diferentes frameworks para diferentes propósitos. FastAPI surge como un framework web que permite construir APIs en Python (versiones superiores a 3.8) de manera rápida y sencilla, basándose en el tipado que ofrece este lenguaje de programación. Además, es compatible con estándares abiertos como “OpenAPI” o “JSON Schema” [30].

La página oficial de FastAPI muestra un ejemplo básico de cómo construir una API, el cual evidencia la facilidad y sencillez con la que se puede desarrollar con este framework:

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
async def root():
    return {"message": "Hello World"}
```

**Figura 3.** Código python para una API básica con FastAPI [31]

#### *1.4.4.2 Docker*

Docker es una plataforma abierta para desarrollar, desplegar y ejecutar aplicaciones, que permite simplificar su proceso de desarrollo y ejecución mediante “contenedores”, es decir, ambientes virtuales que contienen todos los archivos necesarios para que la aplicación se ejecute [32] [33].

Además, en el caso del desarrollo del back-end de una aplicación, Docker facilita el proceso de pruebas debido a que los contenedores pueden volver a crearse una vez que hayan sido configurados.

En el presente componente, se utilizó Docker para desplegar la base de datos en un servidor y de esta manera poder ejecutarlo en cualquier servicio en la nube o de forma local con solamente comandos simples de Docker.

#### *1.4.4.3 Railway*

Plataforma que permite desplegar y alojar aplicaciones web de forma rápida y sencilla, pues se le delega las tareas de despliegue y permite que un equipo de desarrollo se enfoque en la implementación de la aplicación y no en el despliegue de ésta. Tiene integración para CI / CD (Integración y despliegue continuo) con Github [34] [35].

La API de este componente (incluyendo la base de datos dockerizada) será alojada en un servidor de railway configurado para poder exponer los endpoints necesarios para la aplicación y conectado directamente al repositorio de Github del proyecto. Los aspectos fundamentales por los que se seleccionó esta herramienta es su facilidad de uso y sus métricas de costos transparentes que permiten visualizar en tiempo real el uso del servidor, y finalmente la configuración sencilla de la conexión entre API y una base de datos dockerizada.

#### *1.4.4.4 Visual Studio Code*

Para el desarrollo de este componente se ha utilizado principalmente el editor de código “Visual Studio Code”, debido a su compatibilidad con diferentes lenguajes de programación (entre ellos Python) y su integración con GitHub. Además, este entorno de desarrollo es multiplataforma, así como totalmente personalizable mediante extensiones [36].

Todas estas herramientas fueron usadas para el desarrollo del Back-end de la aplicación, FastAPI por su facilidad al momento de crear la API, su eficiencia e incluso porque al construir los endpoints, FastAPI los documenta automáticamente. Docker se usó para el despliegue de esta API junto con la base de datos MongoDB. Y, finalmente, Visual Studio Code sirvió como herramienta tanto para la creación de la API como para la configuración de los contenedores de Docker.

#### *1.4.4.5 Poetry*

Poetry es una herramienta usada para gestionar dependencias en proyectos construidos sobre versiones de Python 3.8 o superiores. Este gestor de dependencias permite facilitar el proceso de instalación o actualización de dependencias mediante comandos sencillos y

un archivo (pyproject.toml) que contiene información resumida de las dependencias y sus versiones. Además, poetry permite crear un ambiente virtual con todas las librerías instaladas para los desarrolladores. De esta manera, todos los miembros del equipo pueden trabajar con el mismo ambiente de trabajo para ejecutar el componente del Backend [37] [38].

#### *1.4.4.6 Black*

Black es una herramienta para formatear código escrito en Python. Es fácil de usar y permite formatear automáticamente todo el código de un proyecto con un simple comando. De esta manera, se puede garantizar una estructura consistente para que sea legible y fácil de mantener por todos los miembros del proyecto [39].

### 1.4.5 SCRUM

Marco de trabajo ágil enfocado en generar valor mediante soluciones para problemas complejos. Caracterizado por la estructura de un equipo con un “Product Owner” (encargado de definir lo que se necesita para resolver el problema), un “Scrum Team” (encargado de implementar la solución) y un “Scrum Master” (encargado de que el ambiente de trabajo se ajuste al marco de SCRUM) y por la división del trabajo en “Sprints” [40].

SCRUM establece una forma de trabajar basada en el empirismo, es decir, el fundamento del conocimiento es la experiencia y en el desarrollo de un proyecto de forma iterativa e incremental con el objetivo de “optimizar la previsibilidad y para controlar el riesgo” [40].

SCRUM trabaja bajo cuatro pilares que deben seguirse a lo largo de todo el proyecto [40]:

- **Transparencia:** Todo el trabajo realizado debe ser visible tanto para el equipo encargado de desarrollarlo, como para las personas que recibirán el producto o servicio una vez acabado el proyecto.
- **Inspección:** El avance del proyecto, sus artefactos e incluso el progreso hacia los objetivos planteados (y acordados) se debe monitorear frecuentemente, para evitar problemas antes de que ocurran.
- **Adaptación:** Si la inspección indica que existen desviaciones en relación con lo planificado, se deben tomar las medidas necesarias para evitar más desviaciones y poder ajustarse a los objetivos del proyecto. En un equipo de SCRUM, se espera que el equipo (Scrum Team) pueda adaptarse en cualquier momento.

El éxito de una implementación de SCRUM también depende de qué tan bien se hayan aplicado sus cinco valores durante el proyecto: *Compromiso*, por parte del equipo, de lograr los objetivos y apoyarse unos a otros. *Enfoque* en el trabajo del Sprint para avanzar hacia dichos objetivos. *Apertura*, por parte del equipo y los interesados (stakeholders), acerca del trabajo y los retos que se encuentren. *Respeto*, tanto dentro de los miembros del equipo, como con el cliente o las personas a las que se les entregará el resultado final. Y finalmente, *coraje*, para afrontar problemas y tomar las decisiones necesarias [40].

Durante el Sprint (con una duración de alrededor de un mes o menos), se planificarán y desarrollarán diferentes eventos (en donde se aplicarán los pilares de SCRUM), los cuales se listan a continuación [40]:

- Sprint Planning: En esta sesión se define todo el trabajo que se realizará en el Sprint y la planificación es el resultado de la participación de todos los involucrados (Scrum Master, Product Owner, Developers). Las tareas que se realizarán se plasman en el “Sprint Backlog” que es un subconjunto de las tareas más importantes o prioritarias del “Product Backlog”. Además, se fija un objetivo a cumplir durante el Sprint.
- Daily Scrum: Reunión pequeña (15 minutos) en donde se inspecciona el progreso hacia los objetivos y, si se da el caso, ajustar el Sprint Backlog.
- Sprint Review: En este evento, se busca inspeccionar el resultado del Sprint y determinar cambios, si es necesario. Es una reunión de máximo cuatro horas en donde se presenta el resultado, se discute el objetivo del Sprint, se analizan los cambios que se han presentado y se decide lo que se hará en el siguiente Sprint (inclusive se puede modificar el Product Backlog).
- Sprint Retrospective: En esta sesión, de máximo tres horas para un Sprint de un mes, busca incrementar la calidad y efectividad del trabajo del equipo. Se analiza cómo fue el desarrollo del Sprint, en relación con las personas, las interacciones, los procesos, herramientas y la “definición de Done”. Con esta información compartida entre el equipo, se busca identificar oportunidades de mejora que serán aplicadas al siguiente Sprint.

En SCRUM se trabaja con los siguientes artefactos [40]:

- *Product Backlog*, que es una recopilación de todo lo que se debe hacer para obtener el producto, servicio o resultado final del proyecto; de esta lista se seleccionan los ítems para el *Sprint Backlog*.

- El *Sprint Backlog* es un artefacto que indica lo que se deberá hacer en el Sprint. Tanto el Product Backlog como el Sprint Backlog se relacionan, respectivamente, con el objetivo del producto (define a lo que se desea llegar con el desarrollo del proyecto) y el objetivo del Sprint (define lo que se desea lograr al final de cada Sprint).
- El *Incremento* representa un avance concreto hacia el objetivo del producto, y el cual se va realizando, refinando y reforzando con cada Sprint. Este incremento se entrega al final de cada Sprint buscando cumplir los requisitos definidos en la "definición de hecho" (Definition of Done), la cual indica el estado de cada incremento en aspectos de la calidad esperada.

En el componente del back-end se ha aplicado SCRUM para gestionar el proyecto, sus hitos, roles, entregables y sesiones. Durante las actividades del proyecto se han seguido las pautas generales mencionadas en esta sección y en la guía oficial de SCRUM. La selección de este marco de trabajo para el componente en cuestión se debe a que, el proyecto que se aborda es un problema complejo caracterizado por cambios frecuentes en los requisitos y SCRUM ofrece la flexibilidad para adaptarse a los requisitos cambiantes.

#### *1.4.5.1 Elementos del Product Backlog*

En este proyecto, el Product Backlog se compone principalmente de historias de usuario: requerimientos del producto expresados desde la perspectiva del usuario. Generalmente se especifica un rol, un propósito y el valor que genera el requisito especificado, a continuación, se muestra la plantilla usada en el presente proyecto [41] [42]:

Como [rol], quiero [objetivo / propósito], para [beneficio / valor].

Estas historias de usuario se agruparon en otro tipo de PBI (Product Backlog Item) llamado "épica". Las épicas son, a grandes rasgos, historias de usuario más grandes y complejas de implementar. De ahí que se las defina en base a historias de usuario más pequeñas [41].

Cada historia de usuario tiene tareas específicas que deben realizarse para cumplir su propósito. Por tanto, una épica contiene varias historias de usuario, y, a su vez, cada historia de usuario abarca tareas específicas.

Finalmente, existen tareas correspondientes a corregir errores o realizar cambios pequeños en el código (por ejemplo, formateo, cambio de nombres de variables, etc.). Estas tareas pueden estar o no asociadas a una historia de usuario y están relacionadas a cambios solicitados por el Product Owner.

El Product Backlog, y el Sprint Backlog están constituidos por historias de usuario y tareas de corrección. Las épicas permiten organizar el trabajo y visualizar la trazabilidad con los objetivos del proyecto, y las tareas específicas ayudan a simplificar el trabajo de cada historia de usuario.

#### *1.4.5.2 Estimación de esfuerzo (Planning Poker)*

En cada Sprint Planning, el equipo de desarrollo debe estimar el "esfuerzo" que implica realizar un conjunto de elementos del Product Backlog en un sprint y si están dispuestos a cumplir con todo lo requerido. Esta estimación puede ejecutarse al asignar "puntos de esfuerzo" o "puntos de historia" a los diferentes PBI seleccionados. Estos valores de cada PBI son relativos pues adquieren un significado dentro del equipo de desarrollo y permiten tener una idea del esfuerzo necesario para cumplir las tareas o historias de usuario determinadas [43].

En este componente se optó por una técnica llamada "Planning Poker" estimar los puntos de esfuerzo de cada historia (y, por ende, de cada tarea) en las reuniones de Sprint Planning. Su proceso se explica a continuación [44]:

- El equipo define una escala que se usará para asignar valores al esfuerzo de una historia de usuario o tarea (en el caso del Backend se seleccionó la escala de Fibonacci: 0, 1, 2, 3, 5, 8, …). Un número más alto en la escala indica un esfuerzo mayor.
- El equipo también define un punto de referencia para el futuro. Por ejemplo, un valor de 5 puede indicar una funcionalidad básica en una aplicación como realizar una interfaz de usuario de Inicio de Sesión.
- Los integrantes del equipo tendrán a su disposición cartas que representan cada uno de los valores posibles en la escala.
- Se lee en voz alta el enunciado de un PBI y los miembros del equipo pueden discutir ideas o hacer preguntas en relación con su implementación.
- Todos los miembros del equipo de desarrollo eligen una de las cartas (indicando el esfuerzo que consideran para el PBI en cuestión) y las muestran al mismo tiempo. Si todos han elegido el mismo valor, este se asigna como los puntos de esfuerzo del PBI. Por otro lado, si no se ha llegado a un consenso, los miembros con mayor y menor puntuación elegido explican su selección. Una vez terminado esta discusión, se repite la votación hasta llegar a un consenso.

Este proceso puede tener variantes: con diferentes escalas, con otro tipo de cartas o con otro tipo de método para llegar al consenso, sin embargo, se ha explicado el proceso

estándar y que se ha usado para el desarrollo de este componente. También es importante mencionar que este “Planning Poker” se ha realizado de forma virtual con una aplicación web que permite realizar esta dinámica de forma remota (“Planning Poker Online”) [45].

#### *1.4.5.3 JIRA*

Como soporte para llevar a SCRUM a la práctica, existen diferentes herramientas para los equipos de desarrollo, como JIRA, ClickUP o Trello. En este componente se utiliza JIRA por ser una herramienta de gestión de proyectos ágiles y que permite gestionar los artefactos de Scrum (por ejemplo, Product y Sprint Backlog), además de que tiene integración con Github y una interfaz intuitiva para usuarios que hayan usado herramientas similares y una popularidad respaldada por más de 65000 empresas alrededor del mundo que la han usado [46].

#### *1.4.5.4 Github*

GitHub es una plataforma para el versionamiento de código, en donde se pueden tener diversos proyectos individuales o de equipo. Al estar basado en Git, tiene soporte para repositorios, ramas, commits y pull requests [47].

Durante el desarrollo de este proyecto, se usó GitHub y Git para el soporte al versionamiento del código fuente de la aplicación (tanto front-end como back-end) debido a la familiaridad que el equipo tiene con esta herramienta y su popularidad.

#### *1.4.5.5 Notion*

Herramienta que permite documentar ideas, apuntes, planes y cualquier información para proyectos colaborativos. Su estructura sencilla basada en páginas y bloques, y su gran variedad de plantillas, la hace eficiente, personalizable y fácil de usar [48].

Para este proyecto, se utilizó una plantilla basada en SCRUM, y se agregaron las secciones necesarias para tener un espacio en donde cualquier miembro del proyecto pueda visualizar información como los registros de reuniones como Sprint Planning o Daily Scrum, las convenciones de codificación, la definición de Done, los roles de los miembros del equipo y el Product Backlog, entre otros.

# 2 METODOLOGÍA

Esta sección describe el desarrollo del componente del Backend a través de 6 Sprints (además del Sprint 0), siguiendo la filosofía del marco de trabajo SCRUM (sección 1.4.5). Cada subsección presenta un Sprint en concreto, y lo describe en 5 partes: una introducción, donde se describe el objetivo del Sprint, y las 4 partes restantes se corresponden con los eventos de SCRUM (Planning, Daily, Review y Retrospective).

NOTA: Esta estructura no aplica para el Sprint 0 en donde se realizaron tareas de gestión del proyecto y se definió información relevante para el desarrollo de Sprints posteriores.

1. Introducción: Descripción y objetivo del sprint
2. Planificación: Correspondiente al evento de “Planificación del Sprint”, describe el conjunto de Items del Product Backlog (PBI, por sus siglas en inglés) que el equipo ha seleccionado para cumplir el objetivo definido en la sección anterior (Introducción). También se detalla la estimación de esfuerzo y tareas asociadas por cada PBI.
3. Implementación: Corresponde al evento del “Daily”, en donde el equipo comunica el trabajo realizado diariamente con el objetivo de sincronizarse. Esta sección describe las decisiones de ingeniería tomadas por el equipo en el proceso de implementación de cada PBI.
4. Revisión: Hace referencia al evento de “Sprint Review” en donde se expone el incremento del Sprint y se recibe retroalimentación de los stakeholders. Con este evento, se analiza el esfuerzo real versus el estimado al inicio del Sprint. Además, se verifica la satisfacción de los stakeholders con el incremento y, si se da el caso, se modifica el Backlog para consideración del siguiente Sprint.
5. Retrospectiva: Detalla las conclusiones de la reunión de “Sprint Retrospective”, las cuales no se relacionan con el producto sino con el equipo y el proceso, destacando oportunidades de mejora para el trabajo en equipo de Sprints posteriores.

## 2.1 Sprint 0

En este Sprint se realizaron algunas tareas de gestión del proyecto, con el objetivo de realizar todos los preparativos para facilitar el trabajo en Sprints posteriores. En este sentido, se ejecutaron las siguientes tareas:

- Se realizó un análisis de herramientas (los resultados del análisis se muestran en los anexos: ANEXO I, ANEXO II, ANEXO III, ANEXO IV, ANEXO V y ANEXO VI) dentro del equipo de desarrollo para definir el conjunto de herramientas y tecnologías con las que se trabajará en el proyecto, tanto para el componente de Frontend como de Backend. Para el caso específico del Backend, se tomaron las siguientes decisiones:

    1. Se seleccionó Python como lenguaje de programación junto con el framework web “FastAPI”.

    2. Trabajar con una base de datos no relacional, en concreto, MongoDB, por su flexibilidad en la estructura de los datos y su compatibilidad con Python. Se optó por una base de datos no relacional porque la estructura de los datos es dinámica y no se requiere un esquema fijo de estos. El objetivo de la base de datos es almacenar grandes conjuntos de datos (escalabilidad) y las relaciones entre ciertas entidades podrían ser complejas de implementar en una base de datos basada en tablas con filas y columnas [49].

    3. Se usará Docker para mantener el backend (API) y la base de datos dentro de contenedores individuales.

    4. Cabe aclarar que se usará Docker para mantener la base de datos dentro de un contenedor.

    5. El control de versiones del código será mediante Git (El repositorio estará alojado en Github).

    6. La gestión del proyecto se realizará mediante dos plataformas. JIRA, para gestionar las tareas, pues permite adaptar la división del trabajo siguiendo las pautas de SCRUM (sprints, historias de usuario, gráficos “burndown” y roles). Notion, para documentar toda la información del proyecto (mockups, registros de reuniones, definición de Done, etc.).

    7. Se seleccionó Visual Studio Code como IDE (Entorno de Desarrollo Integrado), especialmente por su interfaz gráfica sencilla y la posibilidad de agregar extensiones para aumentar funcionalidades.

    8. Inicialmente se seleccionó AWS EC2 como plataforma de despliegue de la API. Sin embargo, en el Sprint 1, se encontró una alternativa más sencilla y compatible con la estructura del componente del backend: Railway. Con

Railway se facilitaba el proceso de despliegue de la API e incluso de la base de datos alojada en un contenedor de Docker. Además de esto, Railway proporciona un bono inicial para uso gratuito de sus servidores. Esto será explicado más a detalle en sprints posteriores (pues el servidor de Railway también sirvió para pruebas de integración en etapas tempranas del proyecto) y en el sprint final (6) en donde se despliega la herramienta para el uso de la aplicación móvil.

- Se definió el enfoque de arquitectura que tendrá el componente (Backend) mediante discusiones dentro del equipo de desarrollo. La arquitectura seleccionada es una adaptación de dos arquitecturas “limpias”: hexagonal y por capas.
- Se definieron los requisitos del producto mediante historias de usuario, épicas, y tareas en un Product Backlog. Dentro de estos PBI, se usaron 5 épicas para organizar los requerimientos:
    1. Promoción de la adopción responsable: Épica que aborda todos los requerimientos relacionados con el fomento de la adopción como método para reducir la cantidad de animales de compañía en las calles.
    2. Aprendizaje mediante gamificación: Contiene todos los requerimientos del módulo de gamificación propuesto para el proyecto.
    3. Soporte al Proceso de Adopción: Requerimientos asociados al proceso de adopción de animales de compañía (para adoptar y para dar en adopción).
    4. Funcionalidades de usuario y perfil: Épica relacionada al ingreso del usuario a la aplicación (inicio de sesión y registro) y el control de sus datos en la aplicación (perfil).
    5. Diseño y desarrollo: Épica creada por el equipo de desarrollo para agrupar PBIs relacionadas con el diseño, la arquitectura o el ambiente de desarrollo.
- Se creó el repositorio documental (Notion) para mantener un registro de toda la información asociada al proyecto. Este repositorio puede ser accedido mediante el enlace del ANEXO VIII.
- Se inicializó un proyecto en JIRA y se lo configuró para que contenga a todos los elementos del Product Backlog definidos anteriormente y para que el equipo tenga acceso al mismo. Cada historia de usuario, épica, tarea u otro PBI tiene un código

estándar compuesto por el código del proyecto, seguido de un guión y el número de PBI (único). Por ejemplo:

PAWQ – 56

Cabe aclarar que la numeración de cada PBI tiene solamente fines de organización y no indican prioridad ni orden de implementación.

- Se creó el repositorio en Github con todas las dependencias iniciales (Poetry, Black) y se otorgó el acceso respectivo a los miembros del equipo. Las ramas del repositorio se estructuraron de la siguiente manera:

    1. Una rama “main” que contendrá todas las funcionalidades una vez hayan sido revisadas y aprobadas en cada sprint.
    2. Una rama “development” que tendrá el rol de rama principal en el proceso de desarrollo. Todo pull request (PR) deberá ser enviado hacia esta rama.
    3. Al integrar Github con JIRA, cada tarea específica podrá tener su propia rama en donde cada miembro del equipo podrá trabajar de manera aislada y con la última versión del código (development). Una vez finalizada la tarea, se la revisa y finalmente se realiza el respectivo PR hacia development.

- Se definieron horarios para las reuniones “Daily” que se realizarán en posteriores Sprints (lunes a viernes a las 19:00). Adicionalmente, estas reuniones diarias se grabarían y guardarían en el repositorio de Notion.

- Se definió un proceso para la estimación de puntos de esfuerzo de cada historia de usuario. Los puntos se definen por una escala de Fibonacci en donde un valor más alto implica más esfuerzo para la historia de usuario (o PBI) en cuestión. La estrategia para la estimación seleccionada fue “Poker Planning”

## 2.2 SPRINT 1

### 2.2.1 Introducción

El objetivo de este Sprint es “permitir al usuario realizar publicaciones de adopción y visualizarlas”. En este Sprint se pudieron evitar algunas tareas relacionadas con el levantamiento del entorno de desarrollo o tareas de gestión del proyecto, debido a los avances realizados en la etapa del Sprint 0, por lo que el equipo se enfocó en implementar la arquitectura del backend y de la base de datos, además de la creación del primer endpoint de la API.

### 2.2.2 Planificación

Para lograr el objetivo planteado, se seleccionaron 5 historias de usuario, cuya descripción, estimación, prioridad y tareas asociadas se muestran a continuación:

**Tabla 1.** División de tareas del Sprint 1

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-27. Como persona interesada en dar en adopción animales de compañía, quiero realizar publicaciones con información específica del animal para informar a los adoptantes sobre el perfil del animal que deseo dar en adopción | 55 | Muy alta | • Crear un endpoint para crear una publicación de adopción<br>• Crear un endpoint para guardar una foto de la adopción en la base de datos<br>• Implementación MongoDBPublicationRepository<br>• Implementación capa de aplicación para “Publication” |
| PAWQ-28. Como persona interesada en adoptar, quiero ver las publicaciones de animales en adopción para encontrar un animal de mi interés al cual pueda brindarle un hogar | 89 | Muy alta | • Definir elementos de la capa de dominio y aplicación para las entidades “publicación”, “foto” y “usuario”<br>• Crear endpoint para recuperar las adopciones de la base de datos (incluyendo paginación y una opción de filtro por fecha, ubicación y especie) |
| PAWQ-73. Como desarrollador, | 55 | Muy alta | • Diseñar la arquitectura del backend (hexagonal) mediante un |

| | | | |
|---|---|---|---|
| quiero diseñar la arquitectura del Backend para garantizar un código organizado y escalable | | | diagrama de clases y un diagrama de arquitectura. |
| PAWQ-75. Como desarrollador, quiero realizar el diseño de la base de datos para asegurar una estructura de datos eficiente y coherente. | 55 | Muy alta | • Definir entidades, atributos y relaciones de la base de datos MongoDB |
| PAWQ-76. Como desarrollador, quiero implementar Docker para almacenar la base de datos del proyecto, y facilitar el despliegue | 21 | Muy alta | • Definir estructura de un contenedor para almacenar la base de datos del proyecto. |

Cabe aclarar que las tareas listadas en la Tabla 1 están relacionadas con el componente del Backend. Las tareas del Frontend correspondientes a estas historias de usuario no comprenden al alcance de este documento.

### 2.2.3 Implementación

Previamente, en el Sprint 0 se definió la arquitectura hexagonal como base para la estructuración de todos los componentes del Backend. Para este sprint específico, se implementaron las entidades del dominio relacionadas con todas las funcionalidades que se deseaban implementar. Además, también se definieron sus componentes de la capa de aplicación e infraestructura (arquitectura por capas) y en ciertos casos, sus implementaciones con tecnologías específicas (Ej: MongoDB y FastAPI).

A continuación, se describe el desarrollo de cada historia de usuario:

### *2.2.3.1 PAWQ-73: Definir la arquitectura del backend*

Con base en lo ejecutado en el sprint 0 (adaptar la arquitectura hexagonal y por capas al componente) se realizó un diagrama de clases para identificar las entidades involucradas en el dominio de la aplicación (con sus respectivos atributos) y un diagrama de arquitectura para representar la organización de estos componentes. Con esta información inicial, se crearon los archivos y carpetas (dominio, aplicación, infraestructura) para todas las entidades que manejará la aplicación (Ej: Persona, Publicación, Comentario, etc.).

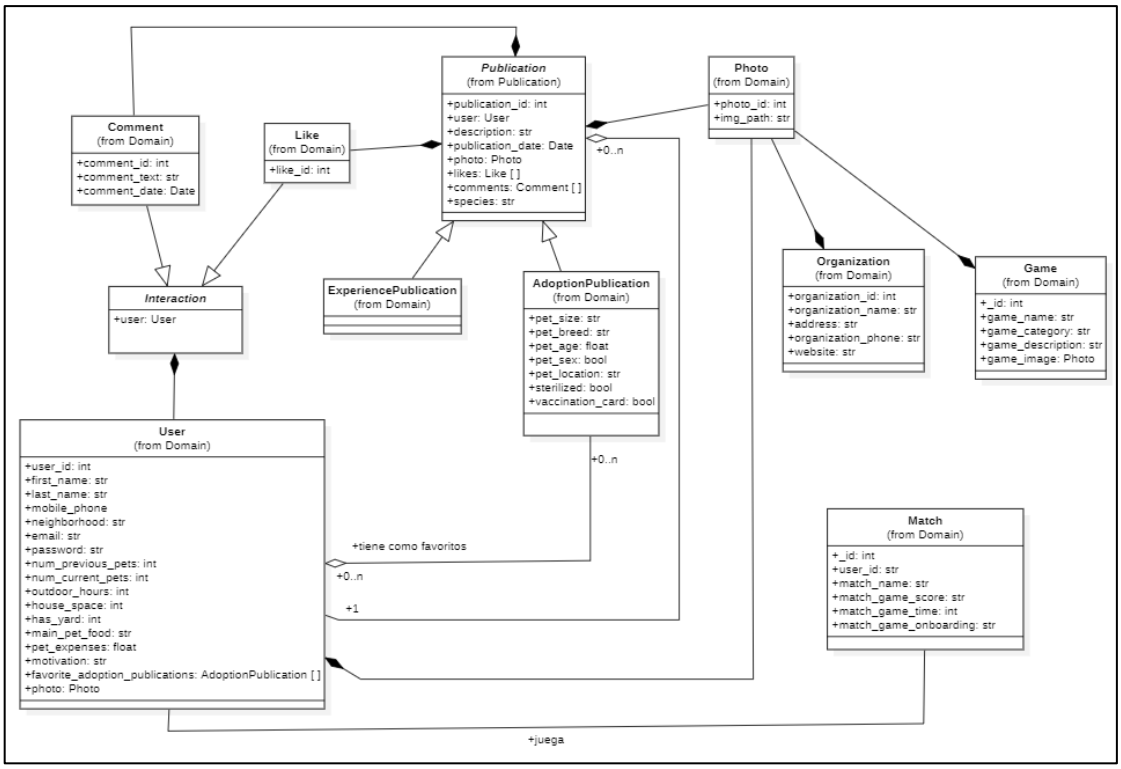


**Figura 4.** Diagrama de clases del componente del Backend

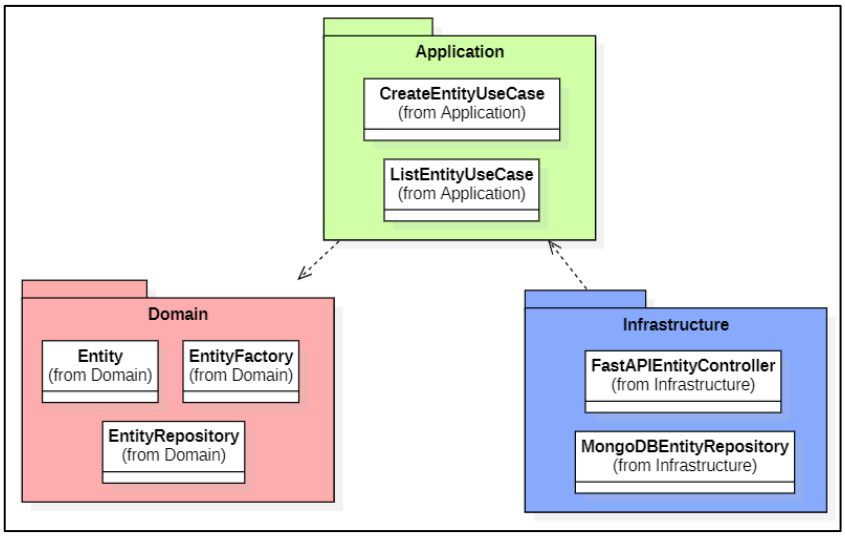


**Figura 5.** Diagrama de arquitectura Backend

El diagrama de la **Figura 5** establece que, por cada entidad de la **Figura 4**, existirán tres carpetas, una por cada capa de la arquitectura cebolla. En la carpeta de dominio se tendrá la entidad (clase en Python con sus atributos), una clase que abstraiga la lógica de creación de objetos de esta entidad (siguiendo el patrón “Factory Method”) y una interfaz relacionada con la entidad que establezca las operaciones que se podrán realizar sobre ésta última (en relación con la persistencia de los datos).

Por otro lado, en la capa de aplicación, se tendrán todos los “casos de uso” identificados y relacionados con la entidad específica (por ejemplo, en la capa de aplicación de la entidad “User” un caso de uso puede ser “Login”). Es importante mencionar que, por cada caso de uso, se tiene una clase con su nombre. Cabe aclarar que estos “casos de uso” no hacen referencia a las acciones del sistema desde el punto de vista del usuario, sino que simplemente hacen referencia a las funcionalidades relacionadas con cada entidad del dominio.

Finalmente, la capa de infraestructura estará compuesta por la implementación de los casos de uso. Esto abarca desde los endpoints correspondientes a la entidad, agrupados en un “Controlador” (implementado en FastAPI) que se encargará de usar los casos de uso para lograr las funcionalidades requeridas por la entidad. Además, en esta capa también se tienen las clases concretas que implementan una interfaz “Repository” de la capa de dominio, es decir, la implementación de persistencia de una entidad con una tecnología específica (en nuestro caso, MongoDB).

A nivel de código, los casos de uso (aplicación) hacen uso de los “Repository” de cada entidad, pero, teniendo en cuenta que estos “Repository” son interfaces, se usará la

“inyección de dependencias” para que, aunque los casos de uso interactúen con las interfaces, en tiempo de ejecución interactuarán directamente con las implementaciones de estos repositorios (al momento de inyectar dependencias es necesario indicar la implementación de cada interfaz). Esto se lo realizará con la librería “inject” de Python.

La nomenclatura para los nombres de los elementos (por ejemplo: “casos de uso”, “repository”, entre otros) y la aplicación de patrones como “Factory method” o “Singleton” fue inspirada en el proyecto “The sheriff” (el cual también sigue una arquitectura hexagonal) [50].

#### *2.2.3.2 PAWQ-75: Definir estructura de la base de datos*

Para esta historia de usuario, se utilizó el diagrama de clases de la **Figura 4** en donde se visualizan todas las entidades y atributos que se manejarán dentro de la API y la base de datos, es decir, las principales clases del dominio de la arquitectura hexagonal y sus colecciones equivalentes en MongoDB.

Con esta información, se logró tener una estructura inicial para la base de datos la cual se plasmó en un script de Python en donde se crean las colecciones iniciales con algunos datos (documentos) lo cual se describe en la siguiente tarea.

#### *2.2.3.3 PAWQ-76: Implementar contenedor para la base de datos*

Una vez definida la estructura de la base de datos, se implementó un contenedor en Docker para almacenar la base de datos. El resultado fue la actualización del repositorio con las variables de entorno para la conexión a la base de datos (url, username, password, nombre de la base de datos), la creación de los archivos “docker-compose.yml” y “Dockerfile” con las configuraciones para levantar el contenedor (con este archivo, cualquier miembro del equipo puede replicar la creación del contenedor de la base de datos) y finalmente un script de python para crear las colecciones y documentos iniciales (publicaciones, usuarios, etc) en la base de datos.

#### *2.2.3.4 PAWQ-27: Publicar información específica del animal*

Se puede notar que solamente las historias de usuario anteriormente explicadas pertenecen a la épica de “Diseño y Desarrollo”. Las historias de usuario “PAWQ-27” y “PAWQ-28” son las únicas tareas de este Sprint relacionadas con funcionalidades de la aplicación. En el caso de la primera, se busca permitir al usuario crear publicaciones de adopción.

Para lograr esto, primero se debieron implementar los componentes de dominio y aplicación para la entidad de “Publication” y en concreto, para su clase hija

“AdoptionPublication”. Es decir, se crearon las clases “Publication” y “AdoptionPublication”, junto con su “Factory” y su interfaz “Repository” (dominio). Posteriormente, se creó el caso de uso “CreateAdoptionPublicationUseCase”, el cual hará uso de las clases mencionadas. Finalmente, se implementaron los componentes de infraestructura: la clase “MongoDBPublicationRepository” para el acceso a los datos de las publicaciones y la clase “FastAPIAdoptionController” para la implementación de todos los endpoints relacionados con las publicaciones de adopción.

Aunque inicialmente se deseaba tener un solo “repositorio” para las publicaciones, los diferentes atributos de las publicaciones de adopción y “experiencia” hicieron que la lógica de la arquitectura hexagonal se modifique, para que los componentes del dominio, aplicación e infraestructura de cada uno de estos tipos de publicación sean independientes. Por tanto, la clase “MongoDBPublicationRepository” se cambió a “MongoDBAdoptionPublicationRepository” y se encargará del acceso a todos los datos relacionados con este tipo de entidad (AdoptionPublication). Toda esta lógica se representa en la **Figura 5**.

La comunicación entre estos componentes se puede representar por el siguiente diagrama de secuencia (en donde se asume que el Frontend realiza la solicitud hacia el endpoint del Backend y éste último ejecuta una funcionalidad o devuelve un resultado):

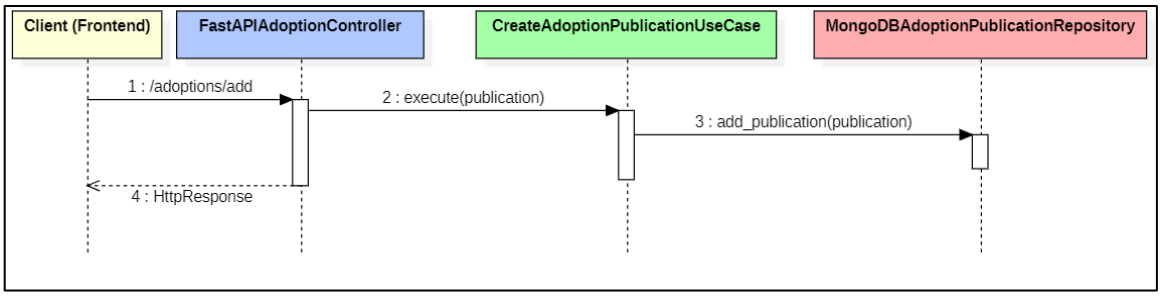


**Figura 6**. Diagrama de secuencia para "Crear publicación de adopción"

En la **Figura 6** se puede visualizar la interacción de las tres capas de la arquitectura (dominio – azul, aplicación – verde, infraestructura – rojo). Es importante mencionar que para que este proceso funcione correctamente, se implementaron las entidades “Photo” y “User” (incluyendo sus clases “Factory”), las cuales son usadas en la clase “MongoDBAdoptionPublicationRepository” (cada publicación tiene una foto y un usuario asociados). El endpoint “/adoptions/add” (POST) solicita un objeto con la estructura de “AdoptionPublication”, y con toda esta información recibida procede a ejecutar el caso de uso respectivo y éste, a su vez, utiliza el repositorio de las publicaciones de adopción para guardar estos datos en la base de datos MongoDB.

La publicación creada guarda una referencia (enlace) a su respectiva foto, la cual inicialmente se deseaba guardar en la base de datos, pero por facilidad del manejo de archivos, se optó por guardarla en un servicio de almacenamiento de archivos de Firebase (Storage) mediante otro endpoint que se debe llamar antes de crear la publicación. Su proceso se describe a continuación:

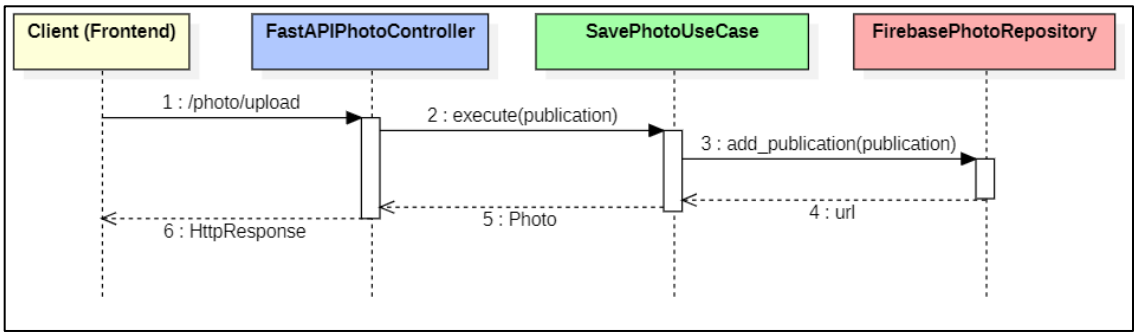


**Figura 7.** Diagrama de secuencia para "Subir Foto"

El endpoint “/photo/upload” de la **Figura 7** solicita un archivo (imagen) por parte del cliente y lo almacena en FirebaseStorage. El repositorio que implementa esta tecnología “FirebasePhotoRepository” devuelve la url de la imagen y el endpoint retorna un objeto de tipo “Photo” con esta información.

#### *2.2.3.5 PAWQ-28: Visualizar publicaciones de adopciones*

La historia de usuario PAWQ-28 busca mostrar publicaciones con lógica de paginación y opción de filtros específicos. Inicialmente se planificó la creación de dos endpoints: uno que recupere todas las publicaciones de adopciones de la base de datos, y otro que recupere publicaciones de adopciones con filtros. Finalmente se decidió unificar esto en un solo endpoint.

Con las clases creadas en las tareas de la historia de usuario PAWQ-27 (especialmente aquella relacionadas con la entidad “AdoptionPublication”) podemos enfocarnos solamente en la creación del caso de uso para visualizar publicaciones. Se usará la misma clase para crear este endpoint (FastAPIAdoptionController) y también el mismo repositorio (MongoDBAdoptionPublicationRepository) para agregar un método de obtener publicaciones de adopción.

El proceso “Visualizar publicaciones de adopciones” asume que el cliente (Frontend) realiza una solicitud hacia el endpoint del Backend y se devolverá la información de las publicaciones para su visualización. Esto sigue el flujo proporcionado por el siguiente diagrama de secuencia:

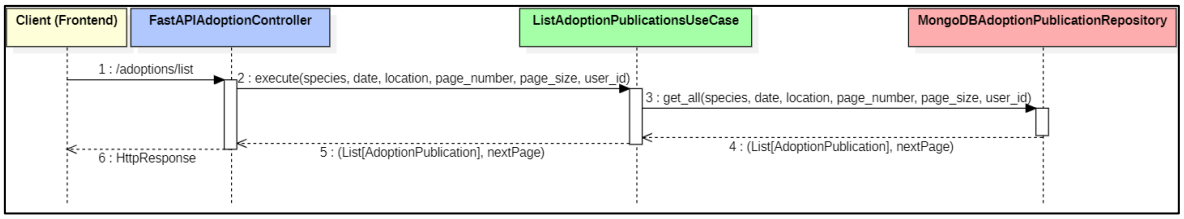


**Figura 8.** Diagrama de secuencia para "Visualizar publicaciones de adopciones"

El endpoint “/adoptions/list” (GET) necesita que el cliente indique los filtros de búsqueda (especie, fecha, ubicación) y datos de paginación (número de página actual, tamaño de página), y devuelve una tupla que contiene una lista de las publicaciones encontradas y un entero que representa el número de página siguiente.

### 2.2.4 Revisión

Una vez terminado el tiempo del Sprint (2 semanas), se realizó la reunión del Sprint Review, en donde se expusieron los avances del sprint y se hizo una demostración (demo) de la aplicación ejecutándose y permitiendo al usuario visualizar diferentes publicaciones de adopciones (esto último constituyendo el objetivo del sprint). Se debe tomar en cuenta que la aplicación utilizó los datos y los endpoints definidos para el backend en este sprint.

Aunque las tareas fueron finalizadas en el tiempo del sprint, debido a que no se cumplió con la definition of Done, no se considera la entrega del incremento, y, por tanto, el sprint se considera fallido. En específico, la historia de usuario PAWQ-27 no se considera terminada por completo (según sus criterios de aceptación) y se la reasigna al sprint 2 (como deuda técnica).

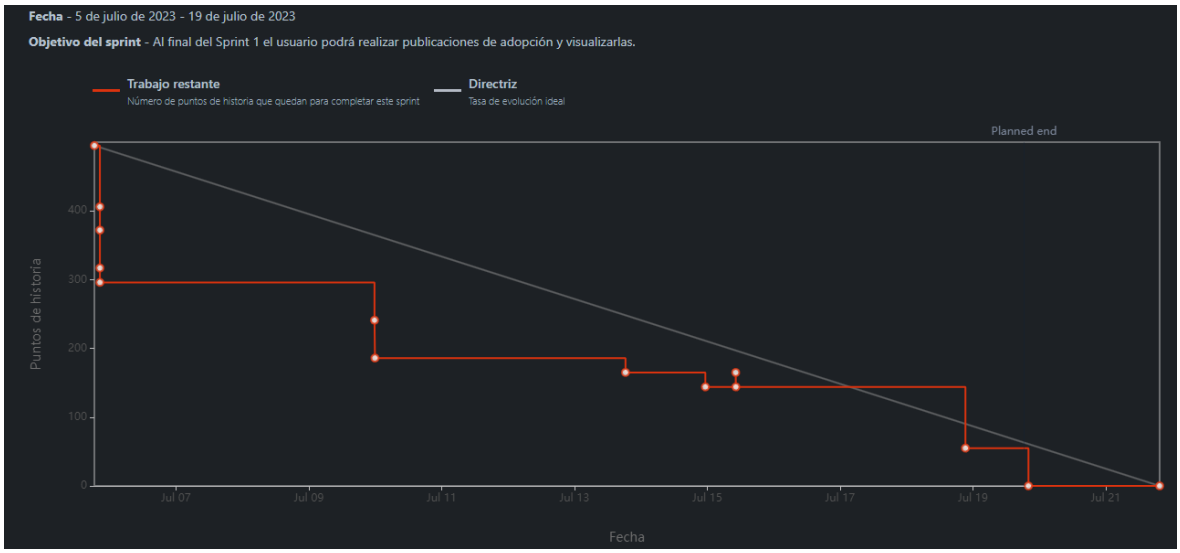


**Figura 9.** Burndown chart Sprint 1

Finalmente, el equipo analizó el gráfico burndown del sprint (**Figura 9**), en donde es evidente que el inicio del sprint avanzó rápidamente y pasada la primera semana, los PBI pendientes fueron finalizando acorde al cronograma. Este gráfico nos indica que las tareas fueron terminadas dentro de la estimación inicial, sin embargo, se debe recordar que el sprint no se consideró como completado.

### 2.2.5 Retrospectiva

La reunión de retrospectiva se centró en los retos que abordó el equipo como la gestión eficiente del tiempo y la estimación de las tareas. Las conclusiones de la reunión fueron las siguientes:

1. Mejorar el proceso de estimación de tareas con el objetivo de no alterar el cronograma y de considerar un tiempo para las pruebas del código (y evitar funcionalidades parciales como el caso de PAWQ-27).
2. Refinar la definición de Done antes de cada Sprint para que el equipo pueda tener en cuenta los entregables al final del sprint.
3. Fomentar la comunicación dentro del equipo, especialmente cuando se tengan problemas o bloqueos. Además, se reestructuró el formato de las reuniones diarias (daily) y su registro en Notion.
4. Continuar con el protocolo de revisión de código y mantener el flujo de trabajo.

## 2.3 SPRINT 2

### 2.3.1 Introducción

El segundo Sprint tiene por objetivo “permitir al usuario realizar publicaciones de experiencias, visualizar experiencias de otros usuarios, guardar publicaciones de adopción como ‘favoritos’ y se podrá evaluar al usuario en temas de tenencia responsable y aspectos legales de la tenencia de mascotas”.

En este Sprint, debido al aprendizaje del anterior sprint y al hecho de haber encontrado la velocidad del equipo, se asumió más esfuerzo que el Sprint 1 (sin contar la historia de usuario no completada en el anterior sprint: “PAWQ-27” y que ha sido agregada para el Sprint Backlog del Sprint 2).

### 2.3.2 Planificación

Para lograr el objetivo planteado, se seleccionaron 4 historias de usuario (además de PAWQ-27 que, al no ser completada en el anterior Sprint, se consideró en la planificación

de este sprint, pero no en su estimación), cada una de las cuales se describen a continuación.

**Tabla 2.** División de tareas del Sprint 2

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-27. Como persona interesada en dar en adopción animales de compañía, quiero realizar publicaciones con información específica del animal para informar a los adoptantes sobre el perfil del animal que deseo dar en adopción | Historia no completada en Sprint 1 | Muy alta | Además de refinar las tareas mencionadas en el Sprint 1, se agregó la siguiente:<br>• Crear endpoint para recuperar información de las parroquias de Quito |
| PAWQ-12. Como promotor de la adopción responsable, quiero crear publicaciones sobre mi experiencia al adoptar animales para motivar a otras personas a adoptar animales de compañía | 30 | Muy alta | • Crear endpoint para guardar la imagen de la publicación de Experiencia en la base de datos<br>• Crear endpoint para crear una publicación de Experiencia |
| PAWQ-15. Como persona que promueve la adopción responsable, quiero poder ver las publicaciones con experiencias de otros usuarios para sentirme inspirado por las historias exitosas de adopción y ver cómo los animales han encontrado un hogar | 21 | Muy alta | • Crear endpoint para recuperar las publicaciones de experiencias exitosas de la base de datos (incluyendo paginación y una opción de filtro por fecha y especie)<br>• Modificar guardado de fechas en el backend (formato datetime) |
| PAWQ-24. Como persona interesada en la adopción responsable de mascotas, | 89 | Muy alta | • Crear endpoint para recuperar información de la |

| | | | |
|---|---|---|---|
| quiero responder preguntas sobre temáticas de tenencia responsable, leyes y sanciones para evaluar mi conocimiento sobre las normativas que rigen la adopción responsable de mascotas | | | temática de la base de datos (Quiz)<br>• Crear endpoint para guardar la puntuación obtenida en el juego<br>• Crear endpoint para recuperar la posición del jugador actual y los 4 primeros jugadores del leaderboard (global) |
| PAWQ-29. Como usuario interesado en adoptar animales de compañía, quiero poder guardar publicaciones de animales en adopción como favoritos para acceder fácilmente a las opciones que más me interesan en el futuro | 34 | Media | • Crear endpoint para recuperar todas las publicaciones guardadas por un usuario.<br>• Crear endpoint para añadir una publicación en favoritos.<br>• Crear endpoint para eliminar una publicación de favoritos. |

### 2.3.3 Implementación

Después de haber definido la estructura básica para cualquier funcionalidad que involucre una entidad del dominio (por ejemplo: Usuario, Publicación, Comentario, etc.) en el anterior Sprint, se realizarán las mismas tareas para el resto de funcionalidades: crear las clases de la capa de dominio (entidad, factory y repositorio), de aplicación (casos de uso) y de infraestructura (implementación de las interfaces repositorio, implementación en tecnología específica, MongoDB, FastAPI, Firebase, etc.).

Con esta información en mente, se presenta la implementación de cada PBI del sprint:

#### *2.3.3.1 PAWQ-27: Publicar información específica del animal*

Esta historia de usuario fue incluida en este sprint por no haber sido completamente implementada en el Sprint anterior. Se finalizó la funcionalidad de crear publicaciones con la información enviada en la solicitud al endpoint “/adoptions/add” y además se agregó un nuevo endpoint para recuperar información de las parroquias de Quito (información que es

necesaria en el Frontend y se eligió no colocarla directamente en el código de este último componente). Esta información de parroquias se almacenará en la base de datos y podrá ser accedida mediante este nuevo endpoint. De esta forma, cuando el cliente (Frontend) solicite la creación de una nueva publicación, la localización o sector (parroquia) del animal tendrá una estructura uniforme (la que ha sido definida por el endpoint).

Por tanto, se realizaron las siguientes acciones:

- Creación de una nueva entidad llamada “Parish” (Parroquia en inglés) con su respectiva interfaz repositorio (“ParishRepository”). Es importante resaltar que, al ser una entidad con un propósito limitado (no tendrá otros usos), “Parish” no tiene su clase Factory correspondiente.
- Creación de único caso de uso: “ListParishesUseCase”
- En la capa de infraestructura, implementar:
    - El repositorio (“MongoDBParishRepository”)
    - La clase que gestione el endpoint para recuperar la información de las parroquias (“FastAPIParishController”).

Adicionalmente, se modificó el script de Python que inserta datos iniciales a la base de datos con el fin de hacer posible el ingreso de la información de las parroquias (cada parroquia tiene un documento que almacena una “clave” y un “valor”, necesarios para el componente de frontend).

Con todo esto realizado, el proceso de listar la información de parroquias se define en el siguiente diagrama de secuencia de la **Figura 10**: :

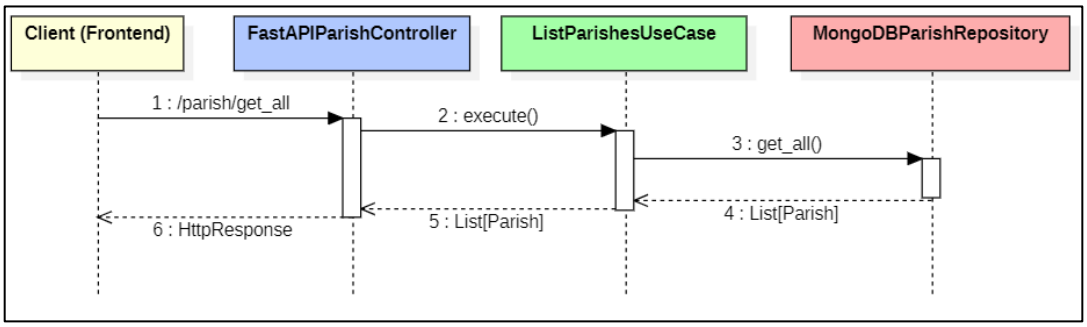


**Figura 10.** Diagrama de secuencia para "Visualizar información de parroquias"

El endpoint “/parish/get_all” (GET), el método de ejecución del caso de uso y el método del repositorio que recupera la información de parroquias, no tienen parámetros. El endpoint es sencillo y permite visualizar de manera más simple la arquitectura implementada. La

respuesta del endpoint es una lista de objetos de tipo “Parish”, es decir, todas las parroquias de Quito.

#### *2.3.3.2 PAWQ-12: Crear publicaciones de experiencias*

Esta historia de usuario es similar a la historia de usuario PAWQ-27 (relacionada con publicaciones de adopción). En este caso se busca que el usuario pueda crear publicaciones de experiencia, es decir, sus experiencias adoptando a sus mascotas pasadas y/o actuales.

Como consecuencia de la experiencia obtenida al implementar el PBI PAWQ-27, el esfuerzo necesario para esta nueva historia de usuario disminuyó. El principio es el mismo, lo único que cambia es la entidad en cuestión, de ahí que los puntos de esfuerzo de esta historia son menores a los de PAWQ-27.

Dada la similitud con la historia PAWQ-27, el proceso para implementar esta historia de usuario también es similar. Se creó la clase “ExperiencePublication” que herede de la clase previamente creada “Publication”. Esta nueva entidad tendrá su propia clase Factory y su propio repositorio (“ExperiencePublicationRepository”). En este punto, es importante mencionar que tanto la Factory de Adopciones como de Experiencias son hijas de una clase abstracta “PublicationFactory”.

El caso de uso para esta funcionalidad es “CreateExperiencePublicationUseCase” y los elementos de la infraestructura creados son la implementación del repositorio (“MongoDBExperiencePublicationRepository”) y la clase que contenga a los endpoints relacionados con la entidad “ExperiencePublication” (“FastAPIExperienceController”).

Con todas estas nuevas clases, el diagrama de secuencia para “Crear publicación de experiencia” es el que se muestra en la **Figura 11**:

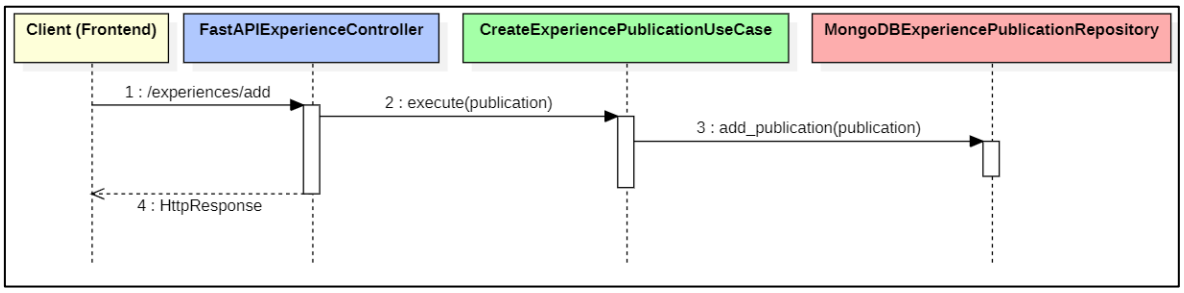


**Figura 11.** Diagrama de secuencia para "Crear publicación de experiencia"

En la **Figura 11**, el proceso inicia con la solicitud al endpoint “/experiences/add” (POST), la API asume que el cliente indica la información de la publicación de experiencia en el cuerpo de la solicitud y una vez llamado el endpoint, se ejecuta el caso de uso para crear la

publicación de experiencia. El caso de uso utiliza la implementación del repositorio con MongoDB para ingresar la publicación en la base de datos.

NOTA: Debido a que las publicaciones de experiencia también tienen una imagen asociada, se planteó la idea de crear un nuevo endpoint con este propósito (ver Tabla 2), pero se optó por utilizar el mismo endpoint creado en el sprint anterior: “photo/upload”, cuyo propósito es cargar una fotografía al servicio de almacenamiento de Firebase).

Una tarea adicional que se implementó al momento de crear estos elementos del backend es la aplicación del patrón de diseño “Singleton” para todas las clases que contengan los endpoints (Ej: FastAPIExperienceController). Esto con el objetivo de mejorar el rendimiento del backend al momento de realizar varias tareas relacionadas con una entidad. Por ejemplo, si se desea listar todas las publicaciones de adopciones y después crear una publicación de adopción, el controlador de los endpoints de Adopciones solo se instanciará una vez y se utilizará esta misma instancia en ambas operaciones.

#### *2.3.3.3 PAWQ-15: Visualizar publicaciones de experiencias*

Al igual que el caso anterior, esta historia de usuario es análoga a la implementada en el primer sprint (PAWQ-28), pero con publicaciones de experiencias. Por tanto, se omite la explicación detallada de los elementos creados. Solamente se debe tener en cuenta que se mantiene la estructura de los componentes en cada capa de la arquitectura (y se aprovecha lo realizado en la historia de usuario PAWQ-12: “repository “y “controlador”). Por tanto, el proceso de visualizar publicaciones de experiencias se representa en la **Figura 12**:

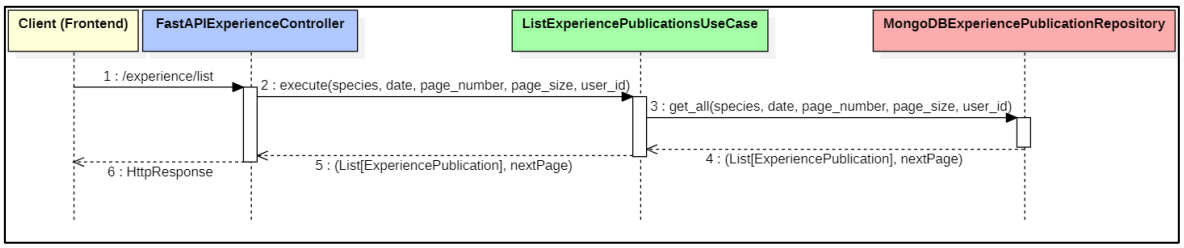


**Figura 12.** Diagrama de secuencia para "Visualizar publicaciones de experiencias"

El endpoint “/experience/list” (GET) espera del cliente la información de filtros de especie y fecha y, mediante el caso de uso y el repositorio de las publicaciones de experiencias devuelve una tupla que contiene una lista de Publicaciones de Experiencia y el número de la siguiente “página” (lógica de paginación).

#### *2.3.3.4 PAWQ-29: Guardar publicaciones como favoritos*

Esta historia de usuario incluye la funcionalidad de permitir al usuario guardar publicaciones de adopción de su agrado en una sección separada llamada “Favoritos” para su posterior visualización. Para esto, se implementarán tres endpoints: uno para visualizar las publicaciones favoritas de un usuario, y dos para agregar y eliminar una publicación de los favoritos del usuario.

Para esta funcionalidad, inicialmente se propuso tener a todas las publicaciones favoritas de un usuario incrustadas en los datos del usuario, pero con un posterior análisis se decidió crear una nueva colección en la base de datos que almacene las publicaciones favoritas por cada usuario. Es decir, en esta colección se guarda una lista de identificadores de publicaciones y cada lista tiene un identificador único igual al ID del usuario que haya guardado ese conjunto de publicaciones de adopción como favoritas. A pesar de haber creado esta nueva colección, no se optó por crear una nueva entidad, sino que se usarán los componentes asociados a la entidad usuario (User, UserRepository, MongoDBUserRepository, FastAPIUserController) para completar la lógica de publicaciones favoritas.

Por tanto, para visualizar las publicaciones favoritas de un usuario, solamente se ha agregado el endpoint correspondiente en la clase FastAPIUserController. También se creó el caso de uso “ListFavoritePublicationsUseCase” y se implementó el método para recuperar las publicaciones favoritas en el repositorio “MongoDBUserRepository”. A continuación, en la **Figura 13**, se muestra el diagrama de secuencia para “Visualizar publicaciones favoritas”:

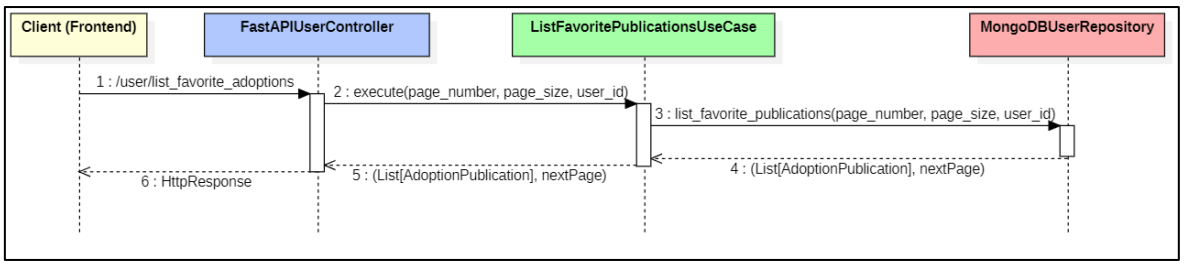


**Figura 13.** Diagrama de secuencia para "Visualizar publicaciones favoritas"

El proceso en la **Figura 13** inicia con la llamada al endpoint “/user/list_favorite_adoptions" (GET), el cual recibe información de paginación y del usuario actual. Posteriormente, se utiliza el caso de uso “ListFavoritePublicationsUseCase” y el repositorio “MongoDBUserRepository”. Finalmente se retorna una tupla con la lista de publicaciones favoritas del usuario y el número de la siguiente página (para implementar paginación).

Posteriormente, se usaron las mismas entidades (con otros endpoints, casos de uso y métodos en el repositorio) para cubrir las funcionalidades de guardar o eliminar publicaciones de adopción de la lista de favoritos del usuario. En la **Figura 14**, se muestra el diagrama de secuencia para “Agregar publicación en favoritos”:

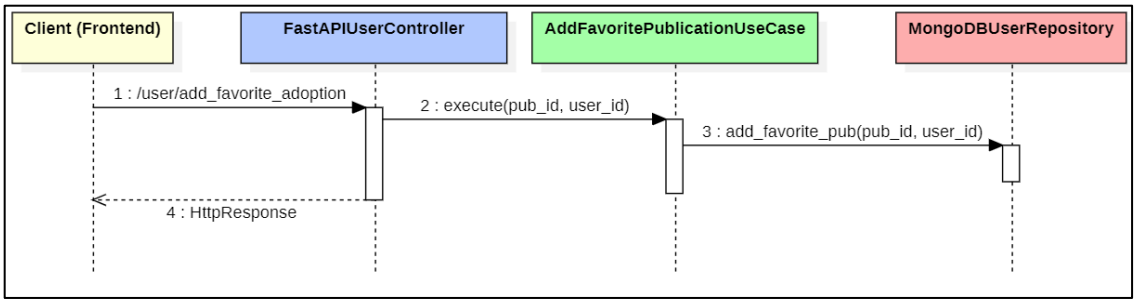


**Figura 14.** Diagrama de secuencia para "Agregar publicación en favoritos"

El endpoint para agregar una publicación como favorita es de tipo POST (“/user/add_favorite_adoption”) y el cliente debe indicar el usuario actual y el ID de la publicación que se quiere agregar a favoritos.

Finalmente, el último endpoint creado es similar al endpoint “/user/add_favorite_adoption”, pero con el propósito de eliminar la publicación de la lista de favoritos de un usuario específico. Usando las mismas clases (con un caso de uso diferente y un método del repositorio diferente), el diagrama de secuencia resultante se muestra en la **Figura 15**:

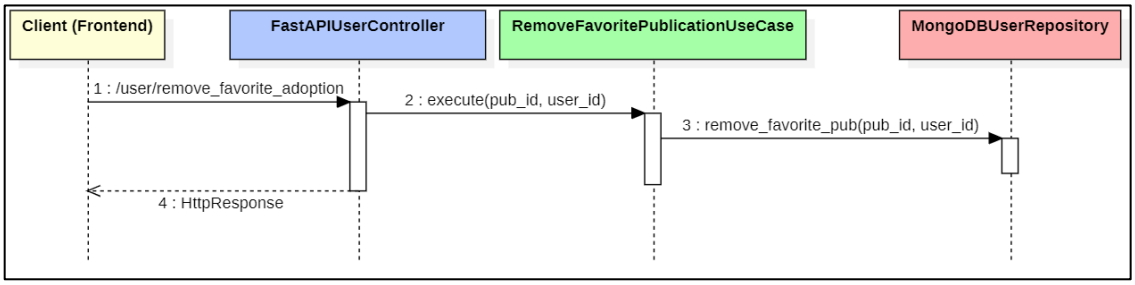


**Figura 15.** Diagrama de secuencia para "Eliminar publicación de favoritos"

La **Figura 15** muestra el endpoint “/user/remove_favorite_adoption” (DELETE) el cual envía un mensaje al caso de uso “RemoveFavoritePublicationUseCase” y éste último hace uso del repositorio del usuario (“MongoDBUserRepository”) para eliminar la publicación de la base de datos.

#### *2.3.3.5 PAWQ-24: Evaluar conocimiento de tenencia responsable*

Esta historia de usuario busca tener un juego (Quiz) que permita evaluar a los usuarios sobre su conocimiento en tenencia responsable y leyes sobre el cuidado animal. La lógica del juego está implementada en el componente del frontend. En cuanto al componente del

backend se ha decidido implementar tres endpoints: uno para recuperar la información del juego (preguntas y respuestas), uno para guardar la puntuación de una partida jugada por un usuario, y otro para recuperar la posición del jugador (usuario) actual y los cuatro primeros puntajes en la tabla de posiciones global.

Inicialmente se tenía una sola entidad “Game” para la lógica de los juegos de la aplicación. Sin embargo, se decidió separar la lógica de información básica del juego (nombre, instrucciones, etc.) de las “partidas” jugadas (con un usuario específico, un puntaje, etc.). De ahí que se tenga una entidad “Game” y una entidad “Match” como se observó en la **Figura 4**.

Con estas consideraciones, para todos los endpoints mencionados en esta sección, se creó una entidad “QuizGameMatch” la cual hereda de la clase “Match”. De igual manera se tiene una clase Factory “QuizGameMatchFactory” que hereda de la Factory “MatchFactory”. Esto se realizó de esta manera porque en futuros Sprints se implementarán más juegos y por tanto otro tipo de “partidas”. También se creó el repositorio del juego “QuizGameMatchRepository”, su implementación “MongoDBQuizGameMatchRepository” y la clase contenedora de los endpoints: “FastAPIQuizGameMatchController”. A partir de estos componentes, solamente es necesario crear los endpoints en esta última clase, crear los casos de uso en la capa de aplicación y codificar los métodos en la clase implementada del repositorio.

Por tanto, el diagrama de secuencia para obtener información de la temática del juego se describe a continuación en la **Figura 16**:

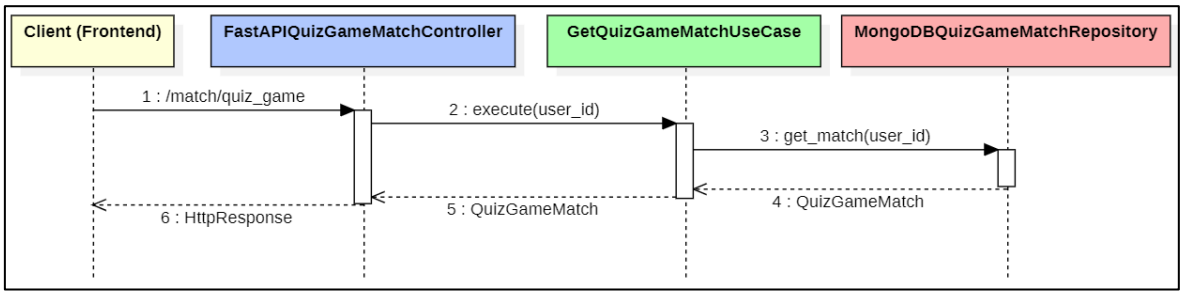


**Figura 16.** Diagrama de secuencia para "Obtener información del juego 'Quiz' "

Para recuperar información del juego (incluyendo las preguntas y respuestas de la partida actual) se implementó el endpoint “/match/quiz_game” (GET), el cual recibe el ID del usuario actual, utiliza el caso de uso “GetQuizGameMatchUseCase” y obtiene la información de la base de datos mediante el repositorio MongoDBQuizGameMatchRepository. El resultado del proceso de la **Figura 16** retorna una instancia de la clase “QuizGameMatch”, la cual tiene todos los campos de la clase “Match”

(ver **Figura 4**) y adicionalmente un atributo que guarda la información de las preguntas y respuestas (de hecho, se crearon clases “Question” y “Answer” para estructurar esta información y también para agregar atributos que indiquen cuál es la respuesta correcta de una pregunta). Toda esta información se almacena en la instancia “QuizGameMatch” retornada, que representa una partida del juego “Quiz”. Cabe aclarar que, debido a que existe una partida de “Quiz” por cada usuario, si un usuario nuevo hace una llamada a este endpoint, se crea una nueva instancia de partida para el juego, pero si el usuario ya ha jugado antes (es decir, tiene una partida registrada), la API devuelve esta partida y no crea una nueva.

Ahora, para el proceso de guardar la puntuación obtenida en el juego (y por ende crear o actualizar los datos de la partida del usuario), se sigue la lógica descrita en la **Figura 17**:

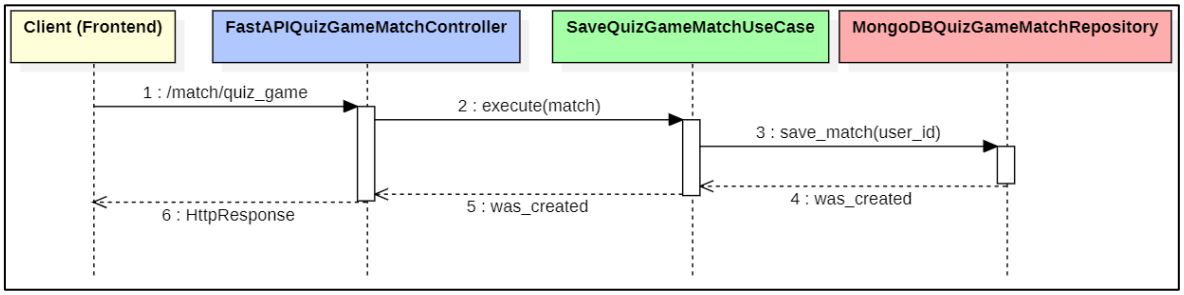


**Figura 17.** Diagrama de secuencia para "Guardar información de partida del juego 'Quiz' "

El endpoint “/match/quiz_game” en la **Figura 17** tiene un nombre idéntico al endpoint descrito anteriormente en la **Figura 16**, pero se ejecuta mediante el método HTTP “PUT”. El endpoint necesita toda la información de una partida, es decir, una instancia de “QuizGameMatch” para poder guardar esta información (incluyendo la puntuación) mediante el caso de uso “SaveQuizGameMatch” y el repositorio “MongoDBQuizGameMatchRepository”. El endpoint crea una nueva partida (instancia de QuizGameMatch) en la base de datos si no existe una partida asociada con el usuario actual. En caso de que la partida ya exista, simplemente actualiza sus valores.

Finalmente, tenemos el endpoint relacionado con la posición del usuario actual en la tabla de posiciones global (dependiendo de los puntos obtenidos por cada usuario). Además, el endpoint también retorna una lista de las primeras cuatro posiciones con sus datos (usuario y puntaje). Este proceso se refleja en el diagrama de secuencia mostrado a continuación:

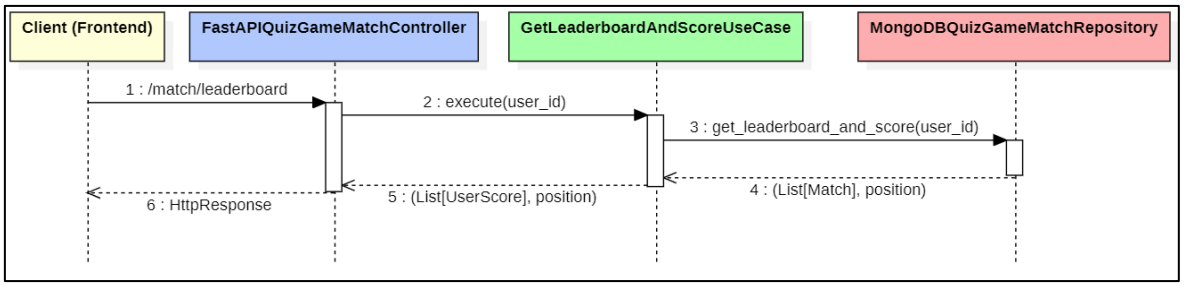


**Figura 18.** Diagrama de secuencia para "Obtener información de la tabla de posiciones del juego 'Quiz' "

El endpoint final es “/match/leaderboard” (GET) que solicita información del usuario actual y utiliza el caso de uso “GetLeaderboardAndScoreUseCase” y el repositorio “MongoDBQuizGameMatchRepository” para obtener una lista de las cuatro primeras posiciones del “leaderboard” global y la posición del usuario actual en dicho “leaderboard”.

Cabe aclarar que se creó la entidad “UserScore” para representar el puntaje en la tabla de posiciones, con el fin de estructurar la información para el frontend. “UserScore” contiene todos los datos esenciales de un usuario (nombres y foto) y el puntaje en el juego.

### 2.3.4 Revisión

En este Sprint se acordó la implementación de más funcionalidades que el Sprint anterior. Esto se debe a que el Sprint 1, si bien incluyó la implementación de la creación y visualización de publicaciones de adopción, la mayoría de las tareas se centraron en la gestión del proyecto y la inicialización de la arquitectura (ver Tabla 1).

Con la experiencia adquirida en el Sprint 1, para este Sprint 2 se fortalecieron las habilidades de estimación de las historias de usuario. Además, con una definición de Done más concreta y documentada, se realizaron las tareas buscando cumplir todos los criterios de aceptación de cada historia de usuario.

La demostración de la aplicación incluyó el uso de todos los endpoints por parte de la aplicación móvil. Debido a que se cumplió con la definición de Done y no se tuvieron tareas incompletas, se consideró al Sprint como completado (incremento entregado). Sin embargo, el Product Owner compartió algunas sugerencias y cambios en el producto. Por tanto, se agregaron PBIs para el siguiente sprint (solamente relacionados con la interfaz de usuario, se decir, el componente del Frontend).

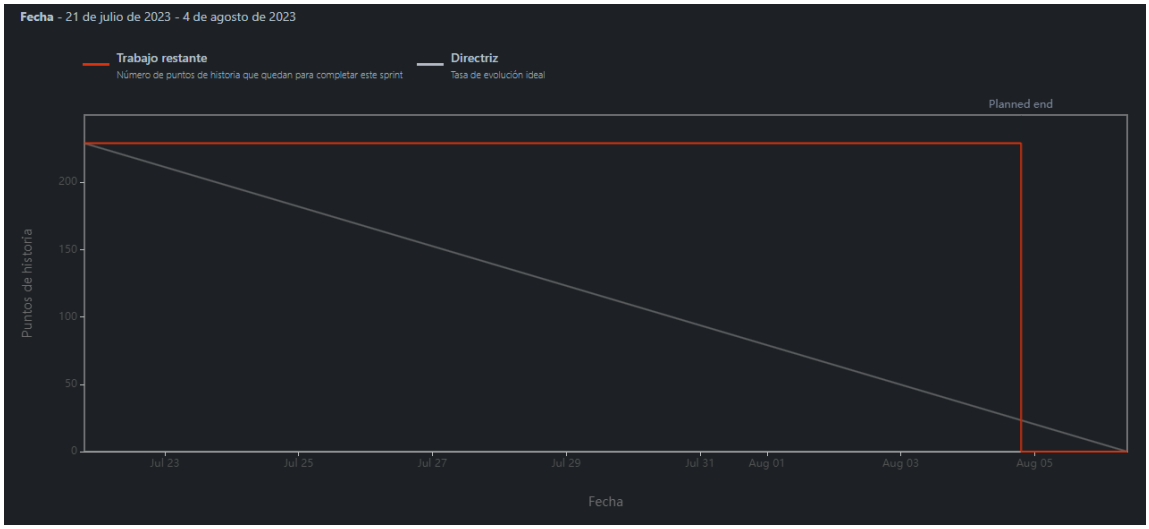


**Figura 19.** Burndown chart Sprint 2

El gráfico Burndown (**Figura 19**) nos indica que las tareas se marcaron como completadas cerca del final del sprint. Esto se debe a que las tareas no se marcaban como “completadas” en JIRA aún cuando ya habían sido terminadas. De hecho, todos los PBI seleccionados para este Sprint se finalizaron dentro del tiempo planificado (incluyendo la historia de usuario PAWQ-27 del Sprint 1). Esto permitió que se tome más en cuenta la gestión de tareas en JIRA para evitar la alteración de gráficos como el de la **Figura 19**.

### 2.3.5 Retrospectiva

La reunión de retrospectiva del segundo sprint se realizó mediante una técnica visual para identificar: a) lo que se debe continuar haciendo, b) lo que se debe dejar de hacer, c) lo que se puede empezar a hacer y d) lo que se va a realizar (ver **Figura 20**).

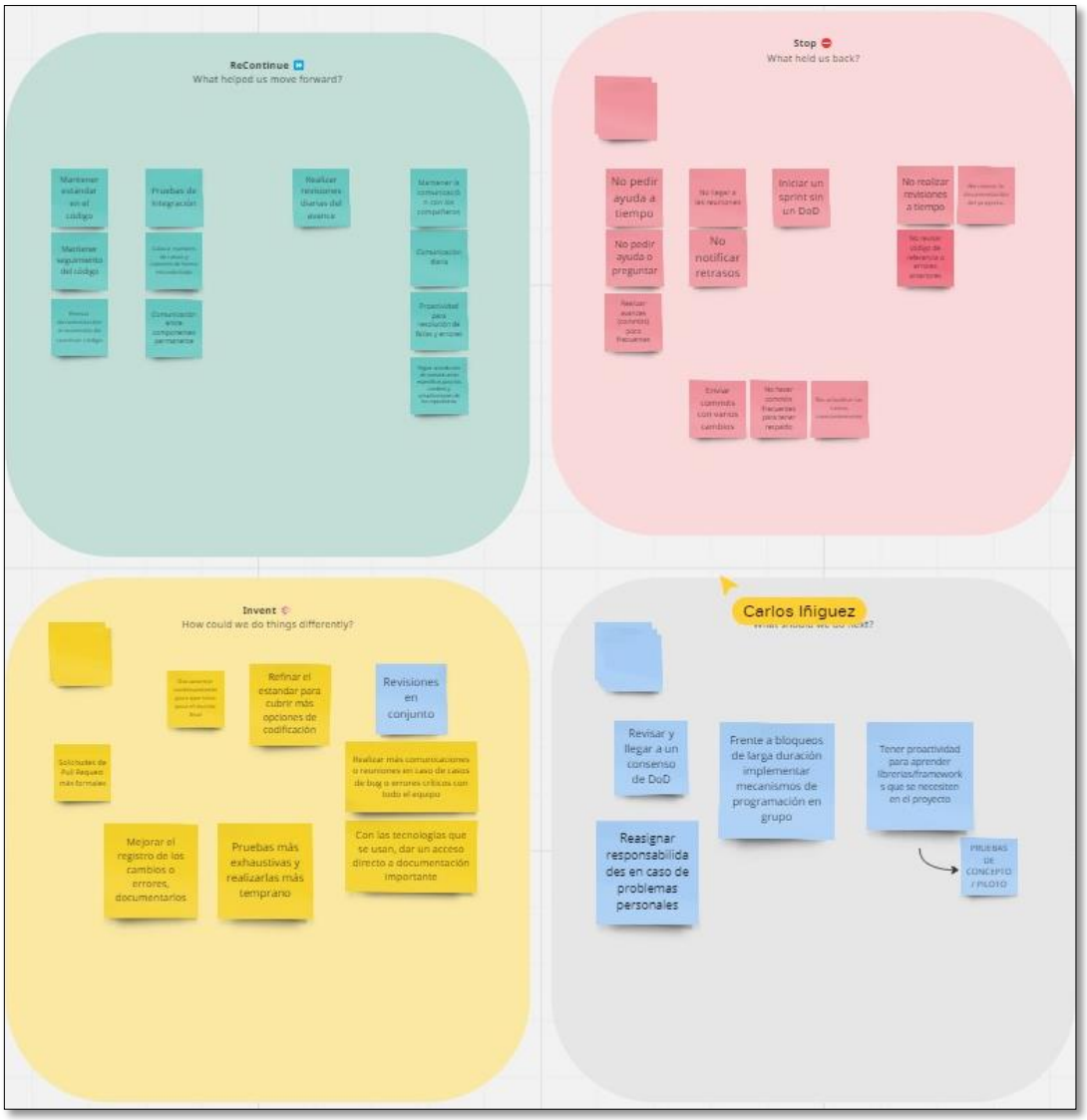


**Figura 20.** Tablero Sprint Retrospective (Sprint 2)

Las conclusiones generales de esta reunión fueron:

1. Mantener la estructura de las reuniones diarias, la comunicación del equipo y la revisión de código.
2. Comunicar rápidamente los bloqueos o problemas que surjan con el objetivo de solucionarlos lo antes posible.
3. Mejorar la gestión de los commits en el repositorio de Github, teniendo en cuenta que no deben abarcar demasiados cambios. En estos commits se debe describir brevemente el código modificado, y estructurar su mensaje.
4. Realizar pruebas con mayor tiempo de anticipación (no establecerlas al límite de finalizar el sprint sino antes)

5. Promover la proactividad para la revisión de documentación y si es necesario programar en equipo para comprender posibles problemas u oportunidades de mejora.
6. En casos particulares se podrán reasignar tareas, con previa comunicación a todo el equipo.

## 2.4 SPRINT 3

### 2.4.1 Introducción

El objetivo del tercer Sprint es “permitir al usuario registrar y visualizar las características de personas aptas para adopción”. Además, se considerará la funcionalidad de mostrar la información de los juegos (catálogo).

### 2.4.2 Planificación

Para lograr el objetivo de este Sprint, se seleccionaron 3 historias de usuario (ver Tabla 3).

**Tabla 3.** División de tareas del Sprint 3

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-21. Como usuario, quiero visualizar los juegos disponibles sobre temas de tenencia responsable, leyes y sanciones del bienestar animal para seleccionar un juego específico | 34 | Alta | • Crear endpoint para recuperar información de los juegos disponibles |
| PAWQ-25. Como persona interesada en dar en adopción animales de compañía, quiero identificar a las personas aptas para una adopción responsable de estos. | 89 | Muy alta | • Crear endpoint para visualizar información de un adoptante (usuario)<br>• Crear endpoint para recuperar todas las publicaciones de un usuario (“mis publicaciones”)<br>• Modificar entidad “Usuario”, “Comentario” y “Like” para ajustarse a |

| | | | |
|---|---|---|---|
| | | | una nueva lógica para estos datos. |
| PAWQ-26. Como persona interesada en adoptar animales de compañía, quiero registrar información sobre mi capacidad de adopción para que la persona que pone en adopción a una mascota sepa que la mascota tendrá un cuidado responsable bajo mi protección. | 25 | Muy alta | • Crear endpoint para enviar datos del formulario de aptitud |

### 2.4.3 Implementación

A continuación, se describe la implementación de cada historia de usuario listada en la Tabla 3:

#### *2.4.3.1 PAWQ-21: Catálogo de Juegos*

En el sprint anterior se implementaron endpoints para el juego de “Quiz”. No obstante, se considera que se implementarán más juegos en la aplicación, lo que genera la necesidad de un nuevo endpoint que retorne la información básica de todos los juegos. En este sentido, se trabajará con las clases relacionadas con la entidad “Game” como se muestra en la **Figura 21**: el endpoint “/game/get_games”, que atiende peticiones por GET, devuelve la información de todos los juegos desde la base de datos, para esto, utiliza el caso de uso “GetAllGamesUseCase” y el repositorio “MongoDBGameRepository”,

También, el script de inserción de datos iniciales en la base de datos fue modificado para hacer posible el almacenamiento de toda la información de los juegos (nombre, categoría, descripción y url de la imagen).

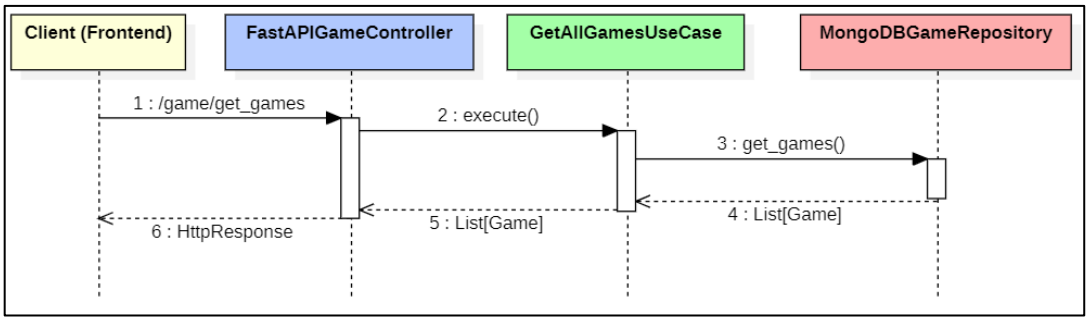


**Figura 21.** Diagrama de secuencia para "Visualizar información de juegos"

### *2.4.3.2 PAWQ-25: Identificar personas aptas para adopción*

Para lograr que los usuarios puedan identificar la aptitud de otros usuarios en relación con el cuidado animal, se plantea incluir, en los perfiles de los usuarios, la información como: cantidad de mascotas anteriores, tamaño de su domicilio, cantidad de tiempo disponible, entre otros. Para esto se implementarán dos endpoints:

1. Un endpoint que devuelva la información del perfil del usuario incluyendo la información mencionada.
2. Un endpoint que recupere las publicaciones de adopción realizadas por el usuario autenticado en la aplicación. Con esto, el usuario puede visualizar los comentarios de las publicaciones y, en consecuencia, los perfiles de personas interesadas en sus publicaciones.

Para el primer endpoint, se usarán las clases relacionadas con la entidad “User”, como se muestra en la **Figura 22**. Se utiliza el endpoint “/user/get_by_id” (GET), que devuelve la información de un usuario que coincide con el ID enviado como parámetro. El endpoint, internamente interactúa con el caso de uso “GetUserByIdUseCase” y el repositorio de usuario “MongoDBUserRepository”, que devuelve una instancia de User correspondiente al usuario que coincide con el parámetro ID.

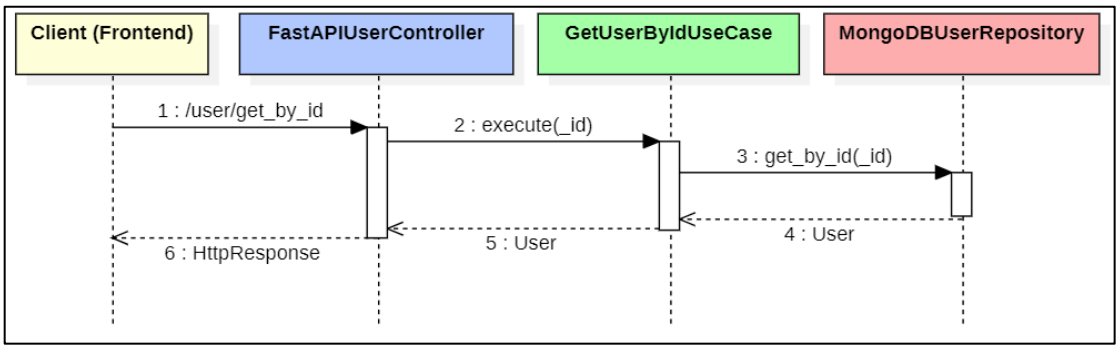


**Figura 22.** Diagrama de secuencia para "Visualizar información de un usuario"

Para el segundo endpoint, se reutilizaron las mismas clases de la entidad “User”, como se muestra en la **Figura 23**. El endpoint “/user/list_my_publications” (GET) interactúa con el caso de uso “ListMyPublicationsUseCase” y el repositorio “MongoDBUserRepository” para obtener información de paginación y las publicaciones de adopción realizadas por el usuario actual.

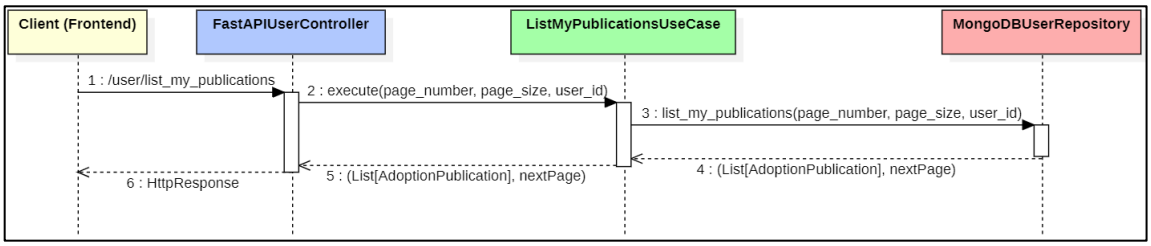


**Figura 23.** Diagrama de secuencia para "Visualizar mis publicaciones de adopción"

Finalmente, la tarea final de esta historia de usuario abarca la modificación de las entidades “User”, “Comment” y “Like”, con el objetivo de agregar el atributo de “fecha de nacimiento” en el usuario (este atributo no se había implementado, pero en futuras funcionalidades sería necesario). En consecuencia, en las clases “Comment” y “Like” se incluyeron los campos para identificar al usuario que realizó el comentario o hizo “like”:

- En la clase “Like” se incluyó el ID del usuario con el objetivo de evitar interacciones duplicadas.
- En el comentario se incluyó el ID, nombre y apellido del usuario. Con esto se podrá acceder al perfil de un usuario desde un comentario que haya realizado.

#### *2.4.3.3 PAWQ-26: Registrar capacidad para adoptar*

Para esta historia de usuario fue necesario implementar un endpoint que permita guardar la información del usuario con respecto a su aptitud para adoptar. Al estar esta historia de usuario relacionada con la entidad “User”, se utilizaron las mismas clases para los endpoints y el repositorio de la historia de usuario PAWQ-25. Los elementos software creados involucrados en la implementación de esta historia de usuario, así como su interacción, se muestra en la **Figura 24**. El endpoint “/user/update_user” (PUT) recibe la información actualizada del usuario y utiliza el caso de uso “UpdateProfileUseCase” y el repositorio “MongoDBUserRepository” para editar los datos de un usuario en la base de datos, entre ellos, los datos de aptitud. El objetivo de la clase “UpdateProfileUseCase” es actualizar la información del perfil de usuario y dado que la información de aptitud es parte de los datos del usuario, la información de aptitud será actualizada mediante este endpoint

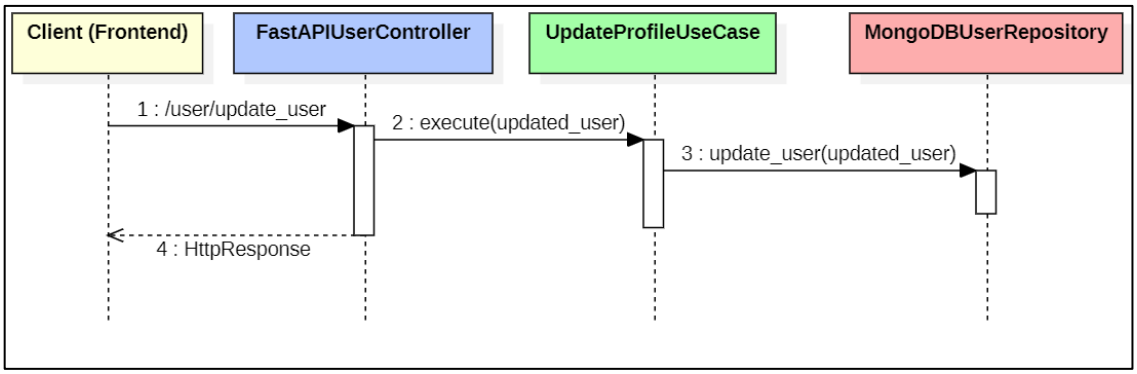


**Figura 24.** Diagrama de secuencia para "Actualizar perfil de usuario"

### 2.4.4 Revisión

Usando la aplicación, las funcionalidades definidas en la Tabla 3 fueron demostradas al Product Owner en la reunión de revisión.

Ya que se cumplieron todas las tareas y se cumplió la definición de Done, se consideró entregado el incremento y por tanto completado el Sprint. El gráfico burndown (**Figura 25**) indica que las tareas en la primera semana del sprint fueron siendo completadas según lo planificado (directriz) pero, desde la mitad del Sprint hasta el final, las tareas se mantuvieron en desarrollo y se finalizaron en el término del Sprint. Este comportamiento se considera de valor y se lo tomará en cuenta para futuros Sprints, ya que se pretende mejorar tanto la planificación de tareas como la realización de pruebas tempranas.

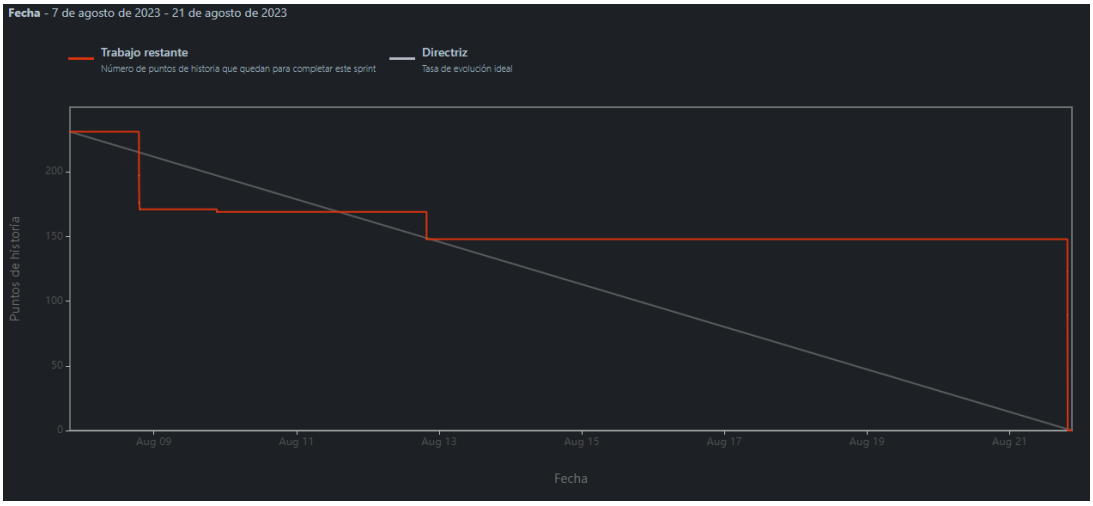


**Figura 25.** Burndown chart Sprint 3

### 2.4.5 Retrospectiva

Para la reunión de retrospectiva resaltó el compromiso en mejorar la planificación de tareas (incluyendo su estimación) y se insistió en que deben ser realizadas pruebas tempranas

para evitar problemas o errores no considerados en las reuniones de revisión (Review). A pesar de esto, el trabajo fue realizado correctamente y sin sobresaltos considerables, por lo que no surgieron nuevas conclusiones.

## 2.5 SPRINT 4

### 2.5.1 Introducción

El objetivo del Sprint 4 es “permitir al usuario realizar likes y comentarios en las publicaciones, contactarse con otros usuarios mediante aplicaciones externas y compartir publicaciones con personas externas a la aplicación”.

### 2.5.2 Planificación

Para el componente backend, en este Sprint se planificó 1 historia de usuario y 1 tarea de corrección de errores. La Tabla 4 describe los PBI mencionados.

**Tabla 4.** División de tareas del Sprint 4

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-14. Como promotor de la adopción responsable, quiero hacer likes y agregar comentarios para mostrar mi interés y expresar mi opinión personal. | 55 | Media | • Crear un endpoint para agregar “like” a una publicación (NOTA: Posteriormente se implementaría un endpoint para eliminar el “like” de una publicación, pero sería implementado en la misma tarea de agregar “like”).<br>• Crear un endpoint para agregar un comentario a una publicación.<br>• Crear un endpoint para recuperar todos los comentarios de una publicación. |
| PAWQ-134. Corregir error en endpoint /user/get_by_id (creación de objeto “User” falla, y | N/A | Media | N / A |

| | | | |
|---|---|---|---|
| cambio de nombre al caso de uso) | | | |

El PBI PAWQ-134 no tiene asignado un valor en sus puntos de esfuerzo pues se consideró una tarea muy pequeña y que sería resuelta rápidamente.

### 2.5.3 Implementación

#### *2.5.3.1 PAWQ-134: Corrección error endpoint /user/get_by_id*

Al inicio del cuarto Sprint, el equipo descubrió un error menor que ocurría al usar el endpoint /user/get_by_id, en donde el usuario obtenido al consumir el endpoint no contenía la información correcta de las publicaciones favoritas de un usuario. Para esto, se modificó el método “get_by_id()” del repositorio “MongoDBUserRepository” para que el usuario devuelto contenga la lista de sus publicaciones favoritas (sus ID). No se requirieron cambios en la estructura interna del funcionamiento del endpoint.

#### *2.5.3.2 PAWQ-14: Realizar Likes y Comentarios*

Esta historia de usuario abarca dos de las principales interacciones que se pueden realizar con las publicaciones (las otras dos son: compartir la publicación fuera de la aplicación y guardarla en favoritos). Para esto se deben implementar endpoints para las siguientes funcionalidades:

- Agregar un like a una publicación.
- Eliminar un like (previamente agregado) a una publicación.
- Agregar un comentario en una publicación.
- Visualizar todos los comentarios realizados en una publicación.

NOTA: La interacción de compartir la publicación fuera de la aplicación se la implementó directamente en el frontend, de ahí que no se han implementado endpoints relacionados.

Todas las funcionalidades tienen relación directa con las publicaciones, por lo que inicialmente se consideró trabajar con las mismas entidades de “AdoptionPublication” y “ExperiencePublication” (además de los repositorios y clases contenedoras de endpoints). Sin embargo, se decidió separar la lógica de “Like” y “Comment” como entidades separadas de las publicaciones, aunque con relación a la entidad “Publication” (“AdoptionPublication” y “ExperiencePublication”). En consecuencia, la implementación final tiene las siguientes consideraciones: 1) una nueva clase para los endpoints relacionados con los “Likes” (“FastAPILikeController”), 2) otra para los comentarios (“FastAPICommentController”) y 3) casos de uso para cada funcionalidad, 4) En cuanto a

la lógica del repositorio, los casos de uso relacionados con los “Likes” usarían métodos de los repositorios de las publicaciones (“Adoption” y “Experience”, dependiendo el caso), 5) los casos de uso relacionados con los comentarios usarán su propia clase “Repository”, esto debido a que los “Likes” se tratan como atributos de las publicaciones, 6) se implementó una nueva colección para almacenar específicamente los comentarios de cada publicación.

Para el proceso de hacer “like” a una publicación, se diseñó el diagrama de secuencia de la **Figura 26**:

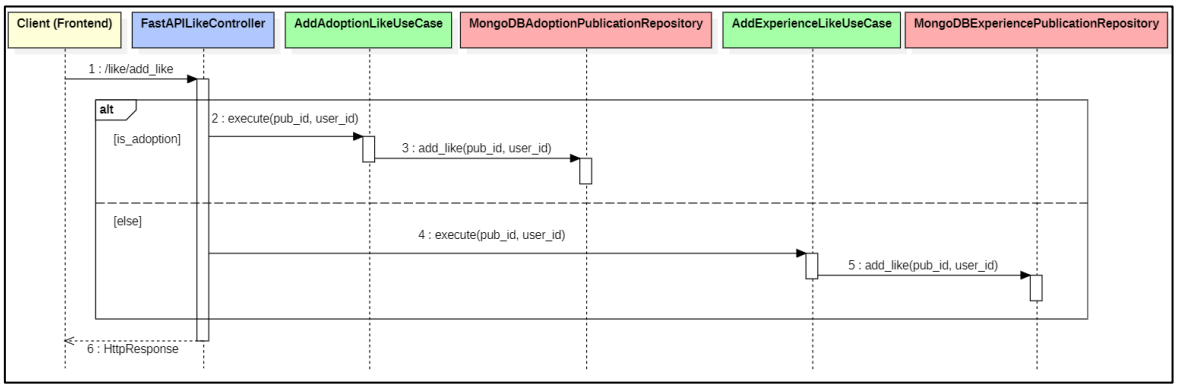


**Figura 26.** Diagrama de secuencia para "Dar Like a una publicación"

En la **Figura 26** podemos ver que el endpoint “/like/add_like” (POST) evalúa si la publicación en cuestión es de adopción o experiencia (el cuerpo de la solicitud incluye este dato, además del ID de la publicación y el ID del usuario actual) y en base a esto se ejecuta el caso de uso “AddAdoptionLikeUseCase” o “AddExperienceLikeUseCase”, los cuales utilizan sus respectivos repositorios. Este proceso permite que el “like” se registre en la base de datos (cada publicación tiene una lista de IDs de los usuarios que han dado like a la publicación).

El proceso para eliminar un like de una publicación es similar al de la **Figura 26**, con la diferencia que se hace uso de los casos de uso y métodos respectivos, como se muestra en la **Figura 27**:

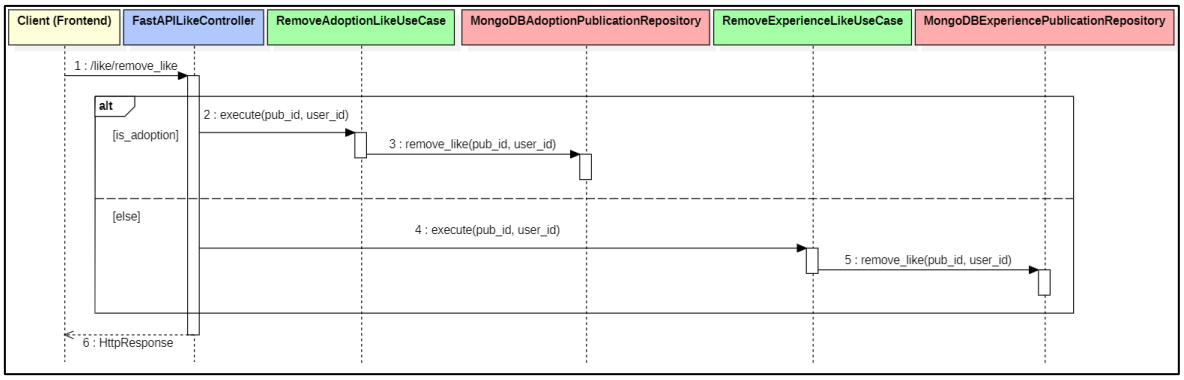


**Figura 27.** Diagrama de secuencia para "Eliminar Like de una publicación"

En este caso tenemos el endpoint “/like/remove_like” (DELETE) que recibe la misma información que el endpoint anterior y analiza si la publicación actual es de adopción o de experiencia. Si es de adopción, utiliza el caso de uso “RemoveAdoptionLikeUse” y su respectivo método en la clase “MongoDBAdoptionPublicationRepository”. Si es de experiencia, se trabaja con el caso de uso “RemoveExperienceLikeUseCase” y su método en el repositorio de las experiencias.

Considerando que los comentarios se trabajarán con una colección aislada (y por tanto un nuevo repositorio), el proceso de agregar un comentario se muestra en la **Figura 28**. El endpoint “/comments/add_comment” (POST) crea un comentario en la base de datos (una colección dedicada solo a comentarios) mediante el caso de uso “CreateCommentUseCase” y el repositorio “MongoDBCommentRepository”. Nótese que el endpoint espera el ID de la publicación, el objeto “User” correspondiente al usuario actual, el texto del comentario y la fecha de publicación del comentario.

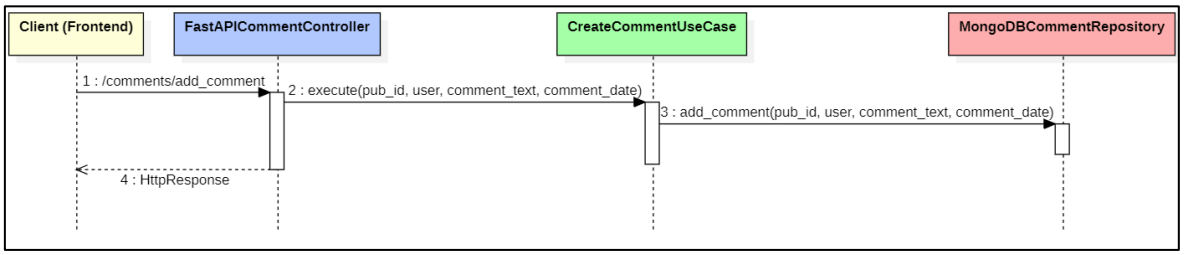


**Figura 28.** Diagrama de secuencia para "Agregar comentario en una publicación"

Finalmente, la **Figura 29** muestra la interacción entre objetos para visualizar todos los comentarios de una publicación. El endpoint “/comments/list_comments” (GET) requiere el ID de la publicación en cuestión y datos de paginación para recuperar los comentarios de una publicación desde la base de datos mediante el caso de uso “ListCommentsUseCase” y el repositorio “MongoDBCommentRepository”.

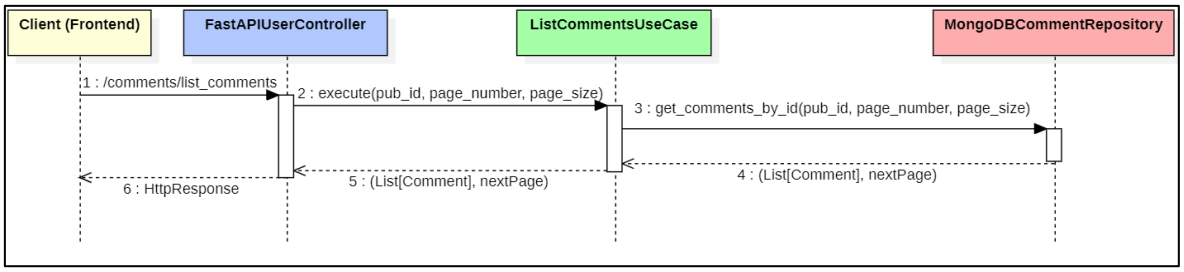


**Figura 29.** Diagrama de secuencia para "Visualizar los comentarios de una publicación"

### 2.5.4 Revisión

La demostración de las funcionalidades en esta reunión se realizó sin inconvenientes. Se cumplieron todas las tareas con sus criterios de aceptación y se cumplió la definición de Done.

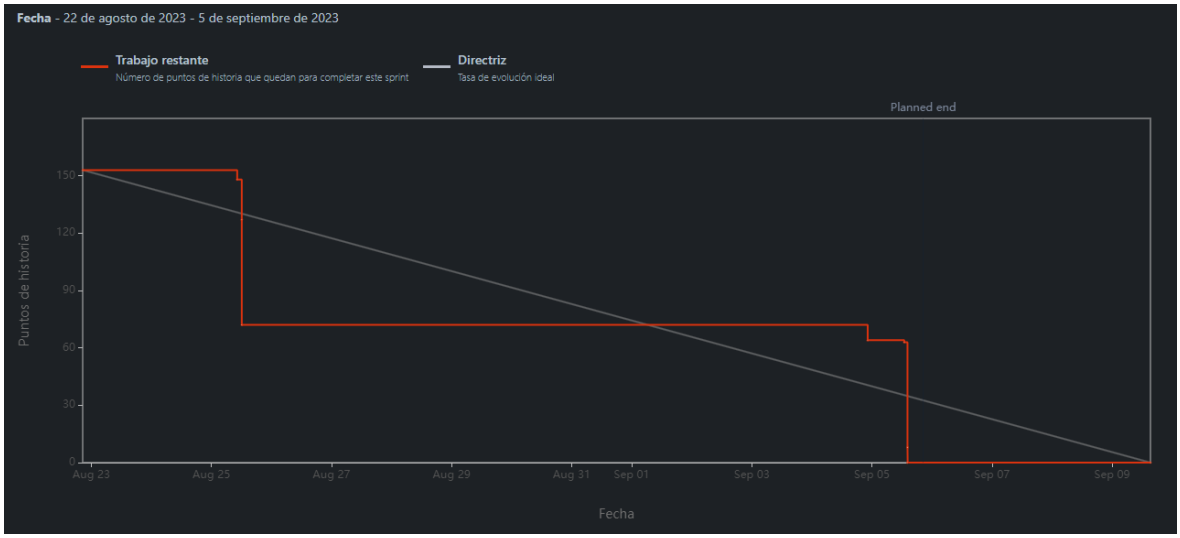


**Figura 30.** Burndown chart Sprint 4

El gráfico burndown de la **Figura 30** muestra que el avance del Sprint 4 fue el más fluido hasta este punto. Es visible que el trabajo restante fue disminuyendo a un ritmo relativamente constante (cerca de la directriz) y no se muestran tareas finalizadas muy cerca del final del Sprint, evidenciando que se mejoró la capacidad de planificación y se tuvo el tiempo necesario para realizar las pruebas respectivas.

### 2.5.5 Retrospectiva

Al igual que el Sprint anterior, no se tuvieron nuevas sugerencias o conclusiones, especialmente porque este Sprint avanzó sin inconvenientes al seguir todos los consejos surgidos de las reuniones de retrospectiva previas.

## 2.6 SPRINT 5

### 2.6.1 Introducción

Este sprint tiene por objetivo “proveer el servicio de autenticación del usuario, además de implementar dos juegos para instruirlo”. Este objetivo abarca todo el proceso de registro e inicio de sesión, así como la implementación de dos juegos dentro de la aplicación móvil.

### 2.6.2 Planificación

Para lograr el objetivo del Sprint, se seleccionaron 6 historias de usuario y 1 tarea de despliegue (ver Tabla 5).

**Tabla 5**. División de tareas del Sprint 5

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-137. Cambio de servidor de desarrollo Railway (Backend) | 30 | Media | N/A |
| PAWQ-22. Como persona interesada en la adopción responsable de mascotas, quiero aprender sobre las sanciones y leyes que garantizan el bienestar animal para estar informado de las normas que rigen el movimiento de adopción | 89 | Media | • Crear un endpoint para recuperar la información de la temática de la base de datos (Adivina la palabra).<br>• Crear endpoint para guardar la puntuación obtenida en el juego. |
| PAWQ-23. Como usuario, quiero aprender, mediante un juego, sobre la | 89 | Media | • Crear un endpoint para recuperar la información de la temática de la base de datos (Sopa de letras). |

| | | | |
|---|---|---|---|
| temática de tenencia responsable | | | |
| PAWQ-31. Como usuario nuevo, quiero poder registrarme en la aplicación para crear una cuenta de usuario que me permita usar la aplicación | 45 | Media | • Crear endpoint para registrar un usuario en la base de datos |
| PAWQ-32. Como usuario registrado, quiero ingresar a la aplicación con mis credenciales para realizar publicaciones y gestionar adopciones de mascotas | 50 | Media | • Crear un endpoint para autenticar a un usuario específico<br>• Crear un endpoint para recuperar la información del usuario que ha iniciado sesión<br>• Agregar protección de endpoints (JWT) |
| PAWQ-33. Como usuario registrado, quiero poder modificar los datos de mi perfil para mantener mi información actualizada | 35 | Media | • Crear un endpoint para actualizar los datos del usuario en la base de datos |
| PAWQ-16. Como promotor de la adopción responsable, quiero conocer sobre las | 89 | Media | • Almacenar una lista inicial de organizaciones en la base de datos<br>• Crear un endpoint para recuperar una lista de todas las |

| | | | |
|---|---|---|---|
| plataformas de fundaciones y asociaciones de protección animal para estar informado | | | organizaciones almacenadas previamente en la base de datos. |

### 2.6.3 Implementación

#### *2.6.3.1 PAWQ-137: Cambio de servidor de desarrollo (BackEnd)*

Esta tarea fue agregada a la planificación del sprint debido a que, el servidor donde se habían realizado las pruebas en sprints anteriores (Railway) ya no se encontraba disponible (el proyecto creado había agotado los créditos de su versión gratuita). Por tanto, se decidió crear un nuevo proyecto (con un nuevo servidor y una nueva instancia de base de datos) en donde alojar las futuras pruebas hasta el sprint siguiente en donde se realizará el despliegue final en la misma plataforma.

Esta tarea fue la primera en completarse en el sprint, con el objetivo de dirigir el enfoque hacia las historias de usuario y que las pruebas en el servidor de Railway no tengan inconvenientes.

#### *2.6.3.2 PAWQ-22: Aprender sanciones y leyes de bienestar animal*

Esta historia de usuario abarca la lógica del juego propuesto “Adivina la palabra” (o “Wordle”) el cual estará relacionado con la temática de leyes de bienestar animal. Aunque inicialmente el juego fue propuesto en la forma de crucigrama, se decidió cambiar esta idea por un juego moderno como “Wordle”. El juego básicamente consiste en, dada una pista, adivinar una palabra en el menor número de intentos posibles; mientras más intentos se utilicen, menos puntaje se logrará (con un máximo número de intentos de 5). Estas palabras están relacionadas con subtemas específicos, por ejemplo, sobre leyes de tenencia responsable, de registro de mascotas, de vacunación y esterilización, entre otros.

Con esta lógica definida, se creará una entidad que represente la partida de este juego (heredando de la clase “Match” creada en el Sprint 2). Esta clase se llamará “WordleMatch” y tendrá atributos adicionales que representen el subtema de la partida, su descripción y la lista de posibles palabras para el juego (recordar que la clase Match ya tenía atributos relacionados con la puntuación y el ID del usuario). Los endpoints a implementar son similares a los creados para el juego “Quiz”, es decir, un endpoint para recuperar la información del juego (subtema, palabras, puntaje) y otro para guardar la puntuación de la

partida. Estos procesos funcionarán en torno a la entidad mencionada, y por tanto se tendrá su clase factory, su propio repositorio y su clase contenedora de endpoints. También se definió una nueva colección en la base de datos, que contenga la información de los subtemas y sus respectivas palabras. Cuando se genere la partida, se escogerá aleatoriamente un subtema de esta colección.

El proceso para obtener la información del juego se describe con el diagrama de secuencia de la **Figura 31**. El endpoint "/wordle/get_wordle_match" (GET) utiliza el caso de uso "GetWordleMatchUseCase" y el repositorio "MongoDBWordleRepository" para obtener los datos de una partida para el juego "Adivina la palabra" desde la base de datos. Es importante recordar que, similar al juego "Quiz", se crea una partida por cada usuario nuevo. Si el usuario ya tiene su instancia de partida creada en la base de datos, el endpoint no crea una nueva, sino que recupera la información de esta partida existente.

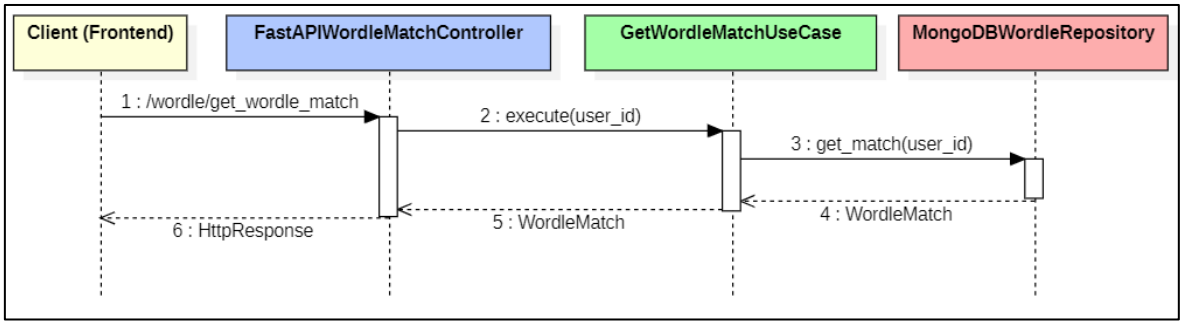


**Figura 31.** Diagrama de secuencia para "Obtener información del juego 'Adivina la palabra' "

Para el proceso de guardado de puntuación, se sigue la lógica descrita en la **Figura 32**. El endpoint "/wordle/put_wordle_match" es de tipo PUT y permite guardar la puntuación de una partida del juego "Adivina la palabra", mediante el caso de uso "SaveWordleMatchUseCase" y el repositorio de la entidad "WordleMatch". El endpoint crea una nueva partida del juego, en el caso de que no exista una en la base de datos; caso contrario, actualiza la partida con los nuevos valores indicados en la solicitud HTTP, de ahí el valor booleano "was_created" que se retorna.

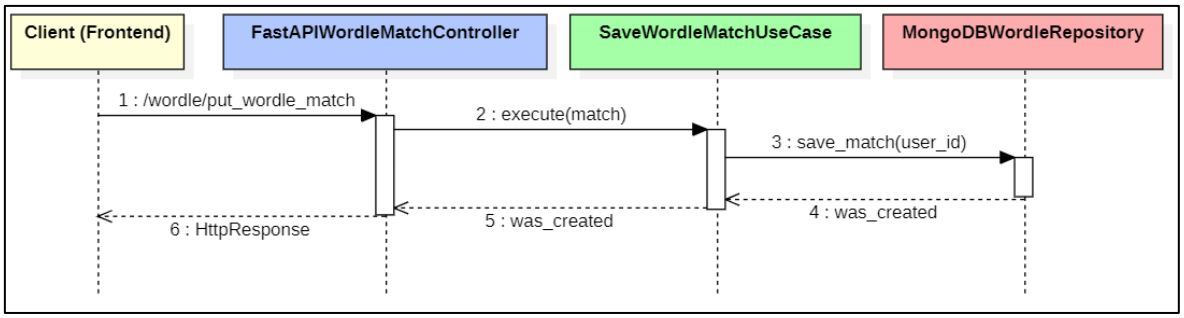


**Figura 32.** Diagrama de secuencia para "Guardar información de partida del juego 'Adivina la palabra' "

### *2.6.3.3 PAWQ-23: Aprender con juego de tenencia responsable*

Esta historia de usuario está relacionada con crear un juego para el aprendizaje del tema “tenencia responsable”. La lógica del juego se basa en una sopa de letras en donde el usuario debe encontrar un conjunto de palabras relacionados con una temática y pistas. El juego no considera un sistema de puntuación, sino que limitará el tiempo en el cual el usuario debe encontrar las palabras. Por tanto, aunque se ha creado una entidad “WordSearchGameMatch” y sus clases factory y repositorio, solo se trabajará con un endpoint para recuperar la información de una partida del juego (temática y palabras a encontrar). El proceso se define en la **Figura 33** donde, el endpoint “/word_search_game/get_match” (GET) utiliza el caso de uso “GetWordSearchGameMatchUseCase” y el repositorio de la entidad principal de la partida para recuperar la información de una partida de este juego. Igualmente, a los otros dos juegos, cada partida está asociada a un usuario y el endpoint crea la partida (si la partida no existe) o la actualiza (ya existe una partida para el usuario actual).

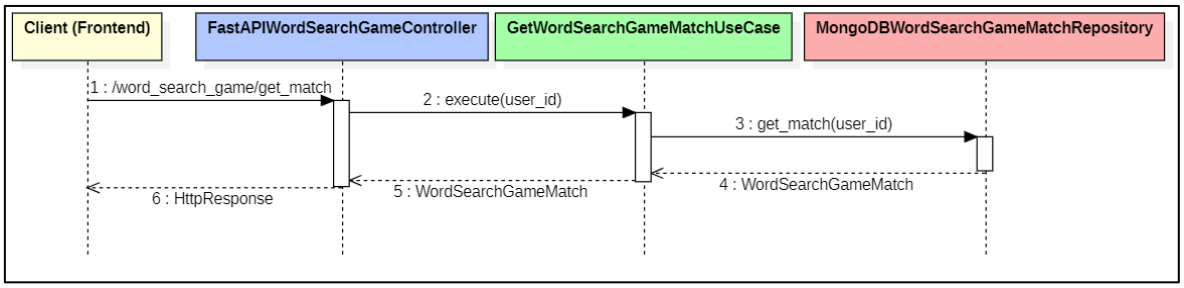


**Figura 33.** Diagrama de secuencia para "Obtener información del juego 'Sopa de letras' "

### *2.6.3.4 PAWQ-31: Registro de Usuario*

Esta historia de usuario busca registrar a nuevos usuarios en la base de datos. Por tanto, se usará la entidad “User” y sus elementos de la capa de aplicación e infraestructura para crear un endpoint que permita guardar la información de un nuevo usuario.

Cabe aclarar que tanto para el proceso de registro como para el proceso de inicio de sesión (PAWQ-32) se usa la lógica de JWT, es decir, generar un token de acceso para el usuario. La implementación siguió la arquitectura por capas, teniendo una clase en el dominio del usuario llamada “AuthService” (en donde se definen métodos abstractos definidos para la autenticación como el encriptado de credenciales o la verificación de éstas) y su implementación en la capa de infraestructura con la clase “JWTAuthService” (que utiliza la tecnología de la librería “jose”).

El funcionamiento del endpoint se muestra en la **Figura 34**. El endpoint “/user/register” (POST) utiliza el caso de uso “RegisterUserUseCase”, el cual verifica que no exista un usuario con el mismo número de teléfono o correo electrónico. Si existe, se arroja una excepción, caso contrario se procede a registrar su información en la base de datos. Nótese que el caso de uso no solo utiliza el repositorio “MongoDBUserRepository” para la verificación y registro del usuario, sino también utiliza el servicio “JWTAuthService” para la encriptación de la contraseña y la generación de un token de acceso.

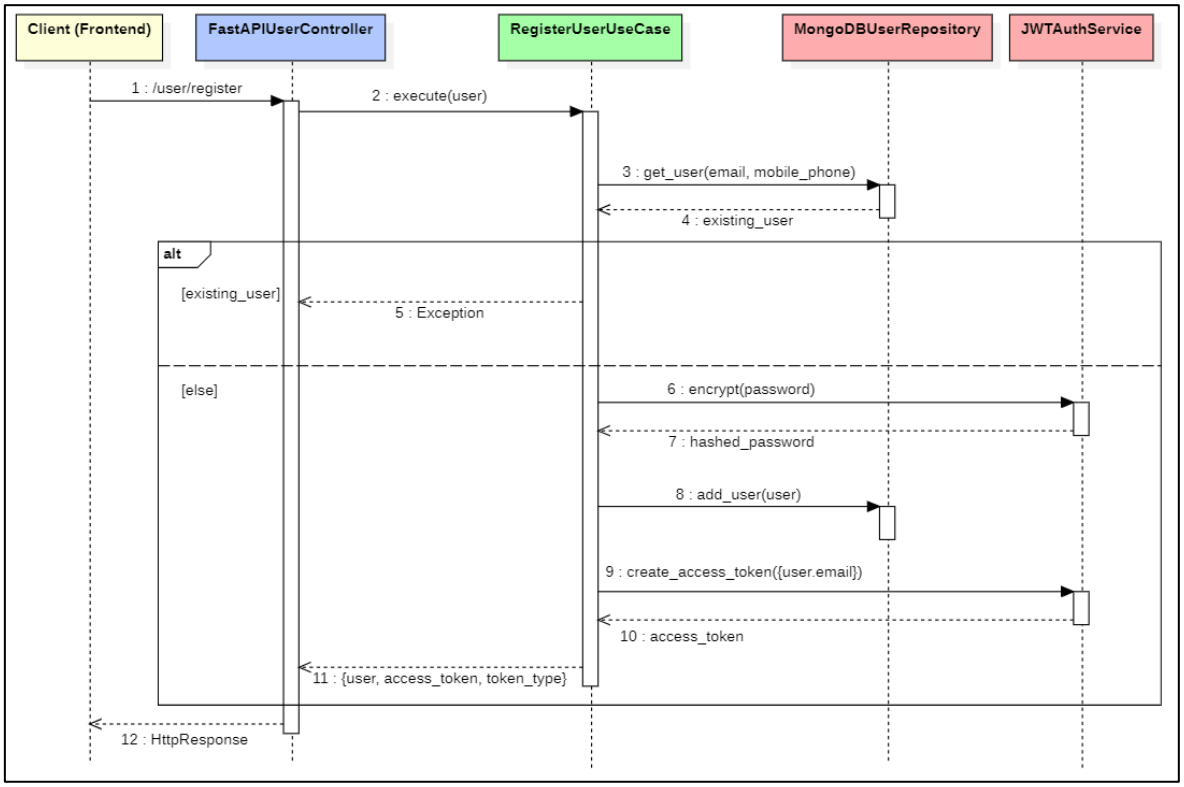


**Figura 34.** Diagrama de secuencia para "Regisrar usuario"

### *2.6.3.5 PAWQ-32: Login (Inicio de sesión)*

Para el inicio de sesión, se trabaja con el mismo servicio “JWTAuthService”. El objetivo del endpoint que permite realizar el inicio de sesión es verificar que el usuario exista y que las credenciales enviadas sean correctas. Este proceso se detalla en la **Figura 35**. El endpoint

“/user/token” es de tipo POST para que el envío de la información de inicio de sesión se realice en el cuerpo de la solicitud y no en la URL. Éste endpoint utiliza el caso de uso “LoginUseCase” para verificar las credenciales (si existe un usuario con el correo indicado y si su respectiva contraseña es correcta). La verificación de la contraseña se la realiza con el método “verify_password” implementado en “JWTAuthService” y que compara el valor de la contraseña indicada en la solicitud con el valor ingresado en la base de datos (éste último encriptado). Si las credenciales no son correctas se levanta una excepción, caso contrario, se genera un token de acceso y se devuelve esta información en la respuesta HTTP.

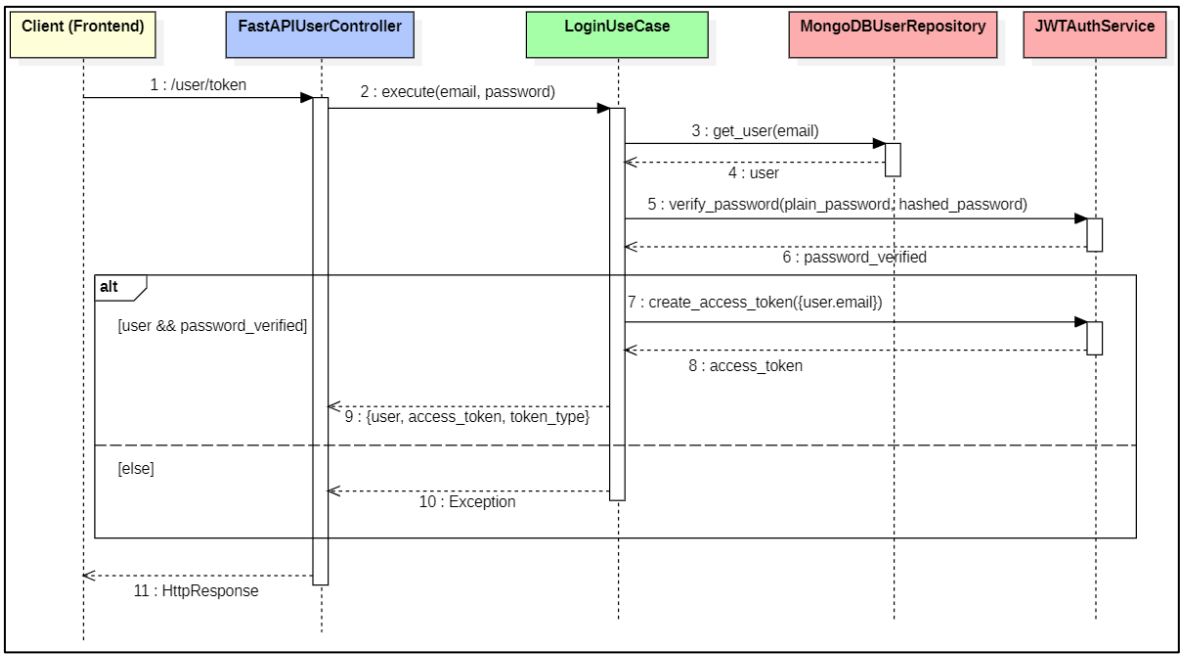


**Figura 35.** Diagrama de secuencia para "Iniciar sesión"

También se creó un endpoint que permita recuperar la información del usuario actual (que haya iniciado sesión previamente). Este proceso se detalla en el diagrama de secuencia de la **Figura 36**. El endpoint “/user/user_me” (GET) utiliza el mismo caso de uso que el inicio de sesión (“LoginUseCase”) debido a que su funcionalidad se relaciona con los datos del usuario que ha iniciado sesión, específicamente con el token generado previamente. Con este token y el método “get_current_user_email()” de la clase “JWTAuthService” se puede obtener el correo del usuario actual (esto se logra porque esta misma credencial fue usada para generar el token, por lo que a partir del token podemos obtener el correo). Con este correo, se usa el repositorio “MongoDBUserRepository” para obtener los datos del usuario con dicho correo (si existe en la base de datos).

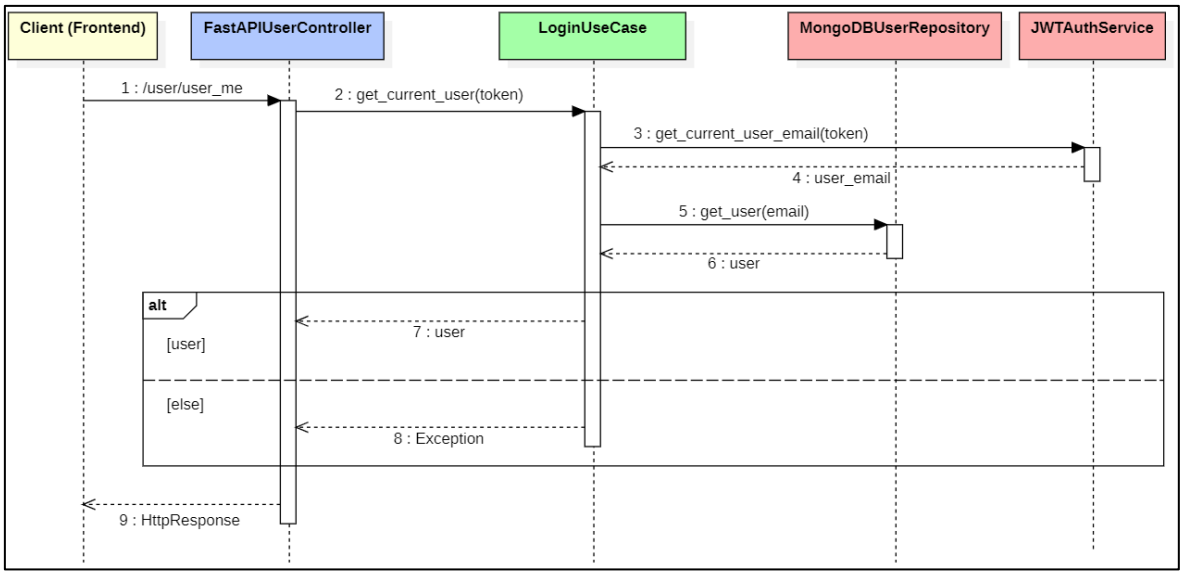


**Figura 36.** Diagrama de secuencia para "Obtener información del usuario actual"

Finalmente, se ha agregado la protección a los endpoints relacionados con la modificación de datos o aquellos relacionados con la entidad “User”, para que no puedan ser utilizados sin la autenticación previa del usuario (es decir, sin el token generado en la cabecera de la petición “Authorization”). Endpoints con propósito informativo (Ej: “/parish/get_all” – ver **Figura 10**) no tienen esta protección. La implementación de esta protección se ha realizado con las herramientas incluidas en FastAPI (Ej: “Depends”) que permiten indicar los endpoints que esperan recibir en la cabecera de la solicitud el token del usuario. En este punto es importante aclarar que la configuración de generación del token al registrar un usuario o al iniciar sesión establece un tiempo determinado para la validez del token.

#### *2.6.3.6 PAWQ-33: Modificar datos de perfil*

Para la funcionalidad de modificar los datos de perfil, se utiliza el mismo endpoint creado en el sprint 3 (PAWQ-26), debido a que su propósito es editar los atributos (o un subconjunto de estos) de un usuario (incluyendo sus datos personales o sus datos de aptitud). Por tanto, el proceso de esta funcionalidad se define por el diagrama de la **Figura 24**. Aunque este endpoint cumple con su funcionalidad, se agregó la actualización del objeto “User” en todas sus publicaciones de la base de datos; una funcionalidad que se completó y terminó en este Sprint.

#### *2.6.3.7 PAWQ-16: Conocer plataformas de fundaciones/asociaciones de protección animal*

Para esta historia de usuario se ha creado un endpoint que recupera la información de organizaciones de protección animal. Esta información se ha guardado previamente en la base de datos, con datos recolectados de las redes sociales de las principales

organizaciones en Quito. Para el manejo de esta información, se creó la entidad “Organization” con atributos que indican el nombre de la organización, su descripción, su foto y enlaces a sus redes sociales (y todos los archivos de aplicación e infraestructura necesarios, incluyendo su clase factory, su repositorio, y la clase que contenga al endpoint).

El proceso para visualizar información de organizaciones de protección animal se define por el diagrama de secuencia de la **Figura 37**. El endpoint “/organization/get_organizations” (GET) necesita datos de paginación (página actual y tamaño de página) para usar el caso de uso “ListOrganizationsUseCase” y el repositorio “MongoDBOrganizationRepository” para recuperar la lista de las organizaciones guardadas en la base de datos.

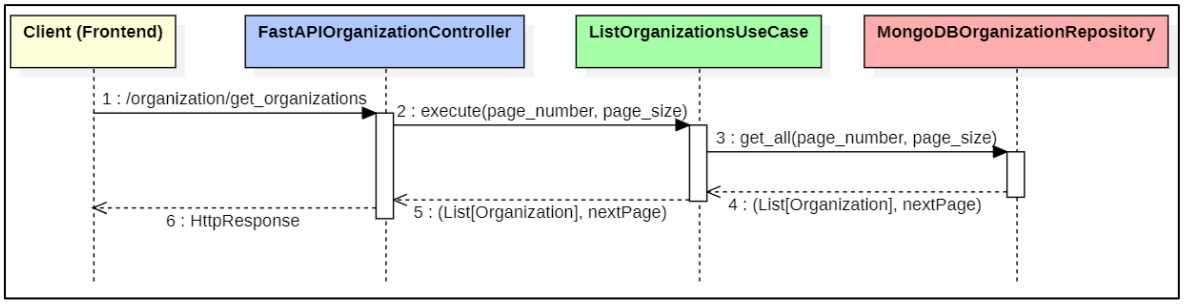


**Figura 37.** Diagrama de secuencia para "Visualizar información de organizaciones"

### 2.6.4 Revisión

Este Sprint, al tener más funcionalidades que anteriores Sprints, fue considerado para ser finalizado en 3 semanas. En la reunión de revisión, se hizo la demostración del funcionamiento de la aplicación (y, por tanto, los endpoints generados). La conclusión fue que se considera la entrega del incremento, sin embargo, se sugirieron algunos cambios y corrección de errores menores que fueron identificados al final del Sprint.

El gráfico burndown de la **Figura 38** establece que se mantuvo el ritmo del trabajo del Sprint, sin embargo, se puede notar que la primera mitad del Sprint avanzó más lento que la segunda parte. De todas formas, se puede visualizar un ritmo relativamente normal y con tiempo de gracias para pruebas finales.

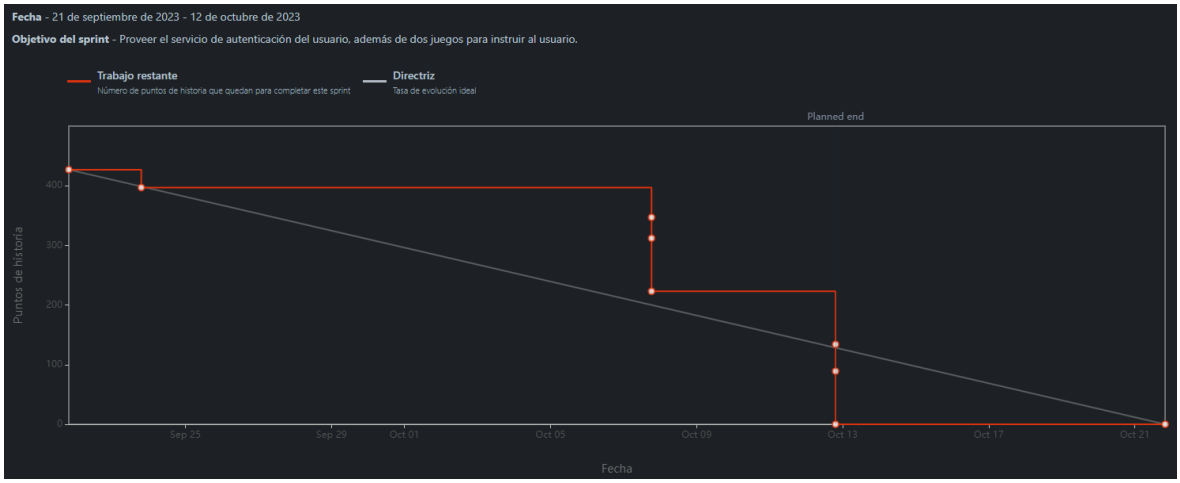


**Figura 38.** Burndown chart Sprint 5

### 2.6.5 Retrospectiva

Se identificó la falta de inclusión de algunas tareas de despliegue final, por lo que, se asignaron estas tareas para el Sprint final (6). Algunas conclusiones de esta reunión fueron:

- Mejorar la planificación general del Sprint (para evitar que no se tomen en cuenta tareas como las mencionadas del despliegue)
- Monitorear el backlog (Product Backlog y Sprint Backlog) con el objetivo de verificar que no se descarten historias de usuario, tareas u otros PBI.

## 2.7 SPRINT 6

### 2.7.1 Introducción

Este es el sprint final del componente y su objetivo es desplegar la aplicación. Para el caso del Backend, se busca desplegarlo en Railway con las configuraciones finales, además de realizar pruebas finales de integración con el Frontend.

### 2.7.2 Planificación

Para lograr el objetivo de este sprint, se planificó 1 historia de usuario para el componente del backend. Su descripción y tareas asociadas se detallan en la Tabla 6.

**Tabla 6.** División de tareas del Sprint 6

| Historia de usuario / PBI | Estimación | Prioridad | Tareas |
|---|---|---|---|
| PAWQ-142. Como desarrollador, | 55 | Alta | • Inserción de dataset inicial de juegos en la base de datos |

| | | | |
|---|---|---|---|
| quiero desplegar el Backend y la base de datos en un servidor, para que la aplicación pueda ser accedida en la web. | | | • Desplegar el backend en el servidor<br>• Pruebas de integración con Frontend |

### 2.7.3 Implementación

#### *2.7.3.1 PAWQ-142: Desplegar backend en railway*

Para completar esta historia de usuario se comenzó poblando la base de datos con información acerca de los juegos. En el proceso de desarrollo se usaron datos de ejemplo para centrarse en la funcionalidad, sin embargo, en este punto fue necesario colocar información real y más extensa. Por ejemplo, se agregaron más preguntas para el juego “Quiz” o más preguntas y pistas para los juegos “Wordle” o “Sopa de letras”. De esta manera los juegos tendrán partidas variadas.

Es importante indicar también que se agregó la configuración para implementar un contenedor para el backend. Es decir, se tendrán dos contenedores, uno para la base de datos MongoDB y otro para la lógica de la API (Backend). Cabe aclarar que esta configuración tiene propósitos de desarrollo futuro, puesto que, como se explicará más adelante en esta misma sección, el servidor de Railway crea automáticamente un contenedor para el backend.

Una vez configurado el dataset final, se creó un nuevo proyecto en Railway para el despliegue (distinto al usado en el proceso de desarrollo) y se modificaron las variables de entorno ya que las que se usaron en servidores anteriores no eran seguras. En específico, se cambiaron las credenciales para el acceso a la base de datos y el valor de tiempo de expiración del token de autenticación (10080 segundos). Se ejecutó el servidor y la API operativa puede ser accedida mediante el enlace del ANEXO VII.

Finalmente, se realizaron las pruebas de integración con el Frontend mediante pruebas de usuario en las cuales mediante la aplicación móvil se hicieron las siguientes tareas:

1. Iniciar sesión en una cuenta previamente creada.
2. Jugar el juego “Leyes y sanciones” (Wordle) y terminar una partida.

3. Realizar una publicación de adopción.
4. Visualizar publicaciones de adopción con y sin filtros de búsqueda.
5. Interactuar con las publicaciones (Like, Comentar, Compartir)

Todas estas tareas se seleccionaron para las pruebas ya que constituyen las funcionalidades principales de la aplicación. En todas las pruebas, se corroboró la comunicación exitosa entre la versión final de la aplicación móvil con el backend (API). Todas las funcionalidades se ejecutaron sin errores, por lo que el componente del backend respondió exitosamente a todas las solicitudes recibidas.

### 2.7.4 Revisión

En la reunión de revisión del Sprint 6, se mostró el funcionamiento de la aplicación usando la API desplegada en Railway. Todas las funcionalidades pueden ser accedidas mediante este servidor.

El gráfico burndown de la **Figura 39** muestra que el avance de las tareas fue anómalo y mucho más lento en comparación con la directriz. Sin embargo, es pertinente mencionar que esto se debió a la ausencia de uno de los miembros del equipo (por razones personales). A pesar de esta situación adversa, se pudieron terminar todas las tareas pendientes al final del Sprint. El aprendizaje resultante es que el equipo debe prepararse para este tipo de situaciones y que el ritmo de trabajo se vea afectado en lo mínimo posible.

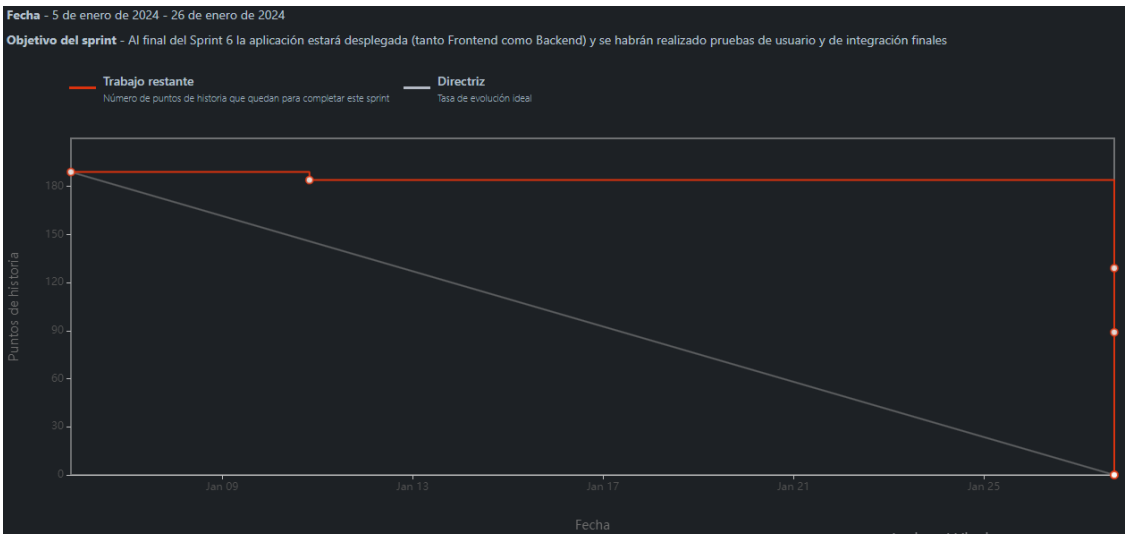


**Figura 39.** Burdnown chart Sprint 6

### 2.7.5 Retrospectiva

El trabajo desarrollado en los primeros cinco Sprints del proyecto permitieron obtener pautas para este Sprint final y para proyectos futuros. El aprendizaje de esta última reunión de retrospectiva es el hecho de planificar el Sprint y dividir responsabilidades de tal manera que el equipo pueda sobrellevar una situación compleja como la salida temporal (o permanente) de un miembro.

# 3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

## 3.1 Resultados

Después de finalizar la implementación de 20 historias de usuario en el transcurso de 6 sprints (además de tareas de despliegue y corrección de errores), el resultado de este componente es una API compuesta por 27 endpoints, los cuales se detallan en la Tabla 7 y se clasifican por su entidad asociada.

**Tabla 7.** Endpoints de la API expuesta por el Backend*

| Endpoint | Método | Descripción |
|---|---|---|
| /user/token | POST | Permite iniciar sesión, generando un JWT para la autenticación |
| /user/register | POST | Registra un nuevo usuario |
| /user/user_me | GET | Obtiene la información del usuario actual |
| /user/get_by_id | GET | Obtiene la información de un usuario específico en base a su ID |
| /user/list_favorite_adoptions | GET | Obtiene la lista de publicaciones de adopción favoritas del usuario actual |
| /user/update_user | PUT | Actualiza la información de un usuario |
| /user/add_favorite_adoption | POST | Agrega una publicación de adopción en la lista de favoritos del usuario actual |
| /user/remove_favorite_adoption | DELETE | Elimina una publicación de adopción de la lista de favoritos del usuario actual |
| /user/list_my_publications | GET | Obtiene la lista de las publicaciones de adopción realizadas por el usuario actual. |

| | | |
|---|---|---|
| /parish/get_all | GET | Obtiene una lista de todas las parroquias de Quito. |
| /photo/upload | POST | Carga una imagen al servicio de almacenamiento (Firebase Cloud Storage) |
| /adoptions/add | POST | Crea una publicación de adopción |
| /adoptions/list | GET | Obtiene la lista de todas las publicaciones de adopción |
| /experiences/add | POST | Crea una publicación de experiencia |
| /experiences/list | GET | Obtiene la lista de todas las publicaciones de experiencias |
| /comments/add_comment | POST | Agrega un comentario en una publicación |
| /comments/list_comments | GET | Obtiene una lista de todos los comentarios de una publicación |
| /game/get_games | GET | Obtiene la información de todos los juegos |
| /match/quiz_game | GET | Obtiene la información de una partida de un usuario en el juego “Quiz” |
| /match/quiz_game | PUT | Actualiza la información de una partida de un usuario en el juego “Quiz” |
| /match/leaderboard | GET | Obtiene una lista de las primeras posiciones del juego “Quiz” (Leaderboard) |
| /word_search_game/get_match | GET | Obtiene la información para una partida del juego “Sopa de letras” |
| /like/add_like | POST | Agrega un “like” en una publicación |
| /like/remove_like | DELETE | Elimina un “like” en una publicación |
| /organization/get_organizations | GET | Obtiene la información de todas las organizaciones guardadas en la base de datos |
| /wordle/get_wordle_match | GET | Obtiene la información de una partida de un usuario en el juego “Wordle” |
| /wordle/put_wordle_match | PUT | Actualiza la información de una partida de un usuario en el juego “Wordle” |

Nota: Los parámetros y la estructura del cuerpo de la solicitud de los endpoints se describen en la sección de “Implementación” de cada Sprint.

El uso de FastAPI y sus herramientas integradas (Pydantic para la validación de datos y Swagger UI para la generación automática de documentación) permiten tener una interfaz visual para el uso de la API, además de servir como documentación de cada uno de los endpoints (su nombre, su método HTTP y el tipo de datos de entrada o salida, además de estar clasificados por cada una de las entidades del dominio). El resultado de la interfaz visual se muestra en la **Figura 40**.

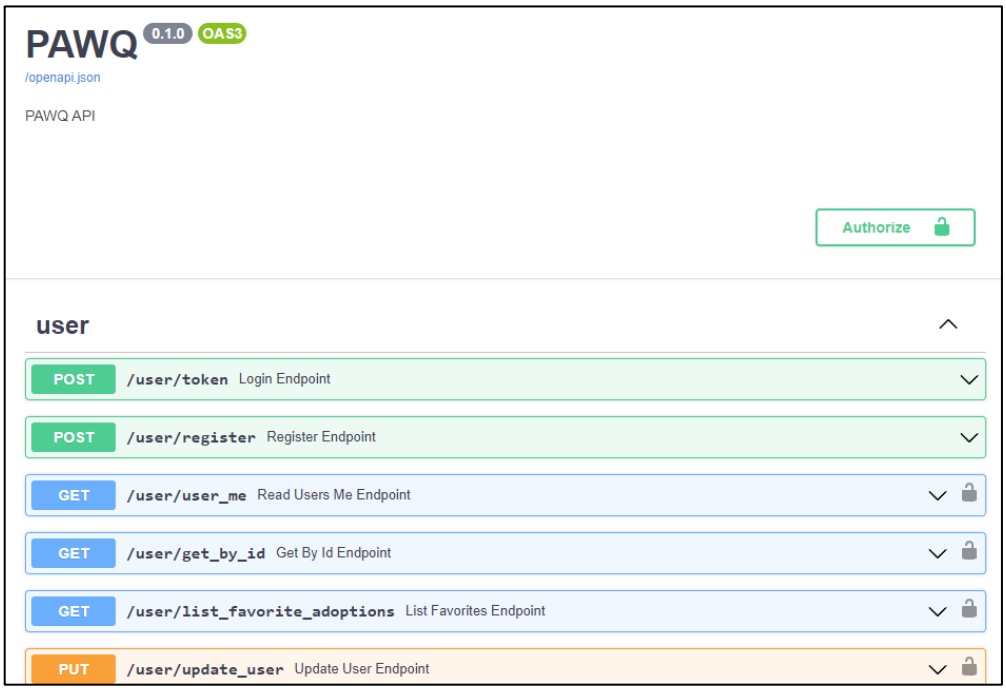


**Figura 40.** Interfaz gráfica de la API (Swagger UI)

Cada endpoint trabaja por detrás con casos de uso y repositorios. El resultado fueron 46 clases relacionadas con la capa de dominio (Entidades, Interfaces de Repositorio, Clases Factory), 32 casos de uso (capa de aplicación), y 24 clases de la capa de Infraestructura, incluidos “Controladores” (clases contenedoras de endpoints), implementaciones de Repositorios, y servicios.

También se implementó una base de datos no relacional en MongoDB con todas las colecciones de documentos relacionados con las entidades del dominio del proyecto, incluyendo publicaciones (de adopción y experiencias), datos de cada uno de los juegos, parroquias de Quito, usuarios, likes y comentarios de las publicaciones. La base de datos no puede ser accedida directamente (las credenciales fueron generadas automáticamente

por Railway y solamente son usadas por el contenedor del Backend). En la **Figura 41** se muestran las colecciones resultantes (desde MongoDB Compass).

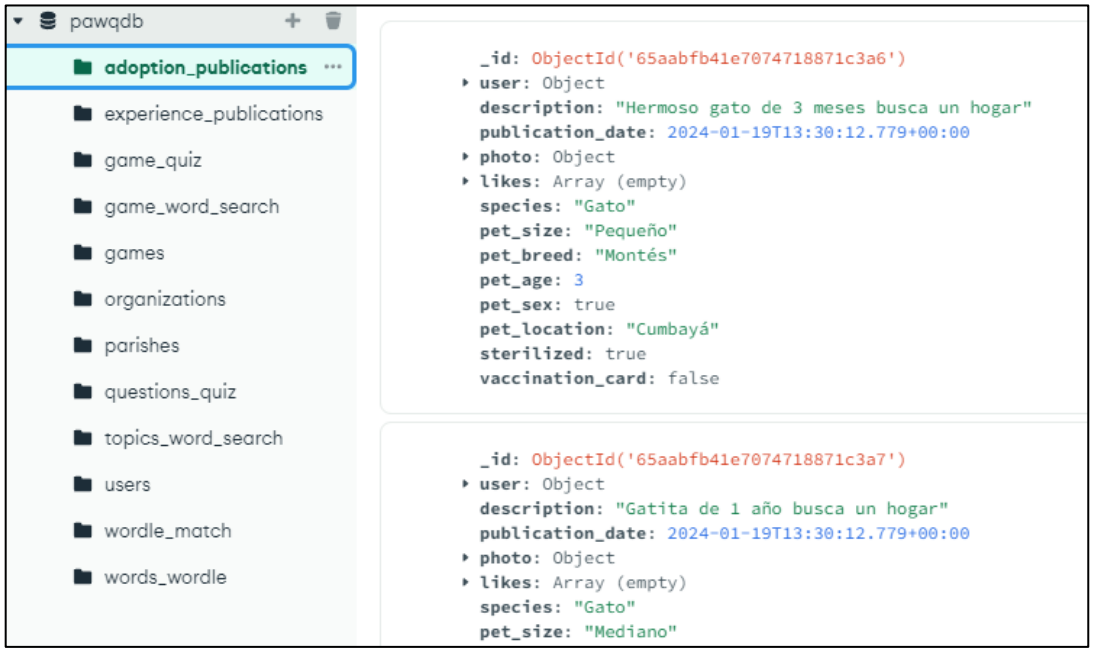


**Figura 41.** Base de datos del proyecto implementada en MongoDB (Vista desde MongoDB Compass)

Finalmente, con el objetivo de minimizar los errores y generar incrementos funcionales en cada Sprint, se realizaron pruebas unitarias con cada funcionalidad agregada al backend (pruebas no automatizadas). El proceso de estas pruebas fue:

1. Ejecutar la API de manera local.
2. Mediante la interfaz de Swagger UI realizar una petición al endpoint implementado.
3. Volver a realizar una petición al endpoint, esta vez mediante la aplicación móvil.
4. En ambos casos (pasos 2 y 3) comprobar el estado de la respuesta entregada por la API.
5. Corroborar el cumplimiento de los criterios de aceptación de la historia de usuario asociada a la funcionalidad evaluada.

Estas pruebas unitarias permitieron identificar y solucionar errores en el proceso de creación de los endpoints. Además, al evaluar continuamente los criterios de aceptación de cada historia de usuario (y por ende de sus tareas), se garantizaba el cumplimiento con la definición de Done de cada incremento.

## 3.2 Conclusiones

Las conclusiones mostradas a continuación se describen mediante los objetivos específicos del componente planteado.

**En relación con el objetivo específico "1. Analizar los requisitos funcionales de la aplicación"**, se construyó un Product Backlog (gestionado en JIRA) que recopilaba todas las historias de usuario y otros PBI relacionados con el dominio de la problemática y la solución planteada. Se analizaron todos los requerimientos funcionales y se incluyeron en cada Sprint dependiendo de su prioridad y las directrices del Product Owner. Todas las historias de usuario del Product Backlog se detallan en la sección de "Planificación" de cada Sprint.

**En relación con el objetivo específico "2. Diseñar la arquitectura de software del backend de la aplicación",** se decidió implementar una arquitectura "limpia", en específico se tomaron los principios de la arquitectura cebolla y la arquitectura hexagonal. En términos generales, se trabajó con tres capas: Dominio, para representar las entidades del negocio e interfaces; Aplicación, para los casos de uso, es decir, las funcionalidades específicas de la aplicación; e Infraestructura, para la implementación de las interfaces del dominio y cualquier utilización de tecnología específica (FastAPI, MongoDB, Firebase). En la sección "Implementación" del Sprint 1 se detalla el proceso de definición de este tipo de arquitectura y su implementación en el componente. El resultado fueron 46 clases de la capa de dominio, 32 clases de la capa de aplicación y 24 de la capa de infraestructura.

**En relación con el objetivo específico "3. Implementar la Base de Datos que almacene los datos de la aplicación"**, se levantó una base de datos no relacional en MongoDB bajo la estructura de colecciones y documentos en donde se guardan todos los datos procesados por la API. La base de datos fue contenerizada y desplegada en Railway, y su acceso directo está restringido para uso exclusivo de los endpoints de la API. Las colecciones de la base de datos se basaron en las entidades del negocio. El proceso de levantamiento de la base de datos se detalla en la sección "Implementación" del Sprint 1.

**En relación con el objetivo específico "4. Exponer la lógica de negocio y las funcionalidades a través de una API"**, se construyó una API mediante el framework de FastAPI en Python. El resultado fueron 27 endpoints que representan las funcionalidades principales para ser consumidas por el cliente (Frontend – Aplicación móvil). La API fue contenerizada y desplegada en un servidor de Railway, y se puede acceder a ella desde el enlace indicado en el ANEXO VII.

## 3.3 Recomendaciones

En esta sección se describen las recomendaciones surgidas del aprendizaje y la experiencia adquiridas en el proceso de desarrollo de este componente. Su objetivo es plantear oportunidades de mejora y temas de discusión.

- Automatizar el proceso de pruebas unitarias (y otro tipo de pruebas como de estrés o carga) con el objetivo de minimizar el tiempo dedicado a estas pruebas a medida que el proyecto crece. En el caso del presente componente, las pruebas unitarias manuales fueron ejecutadas con éxito, pero podrían realizarse de manera más eficiente si se automatizan con un framework.

- Documentar claramente los criterios de aceptación de las historias de usuario, con el propósito de que el equipo de desarrollo tenga una pauta sobre lo que el cliente desea.

- Establecer una clara Definición de Done (DoD), pues esto permite que el equipo de desarrollo identifique, sin ambigüedad, si un incremento se considera terminado y funcional para el cliente. El primer Sprint de este componente permitió deducir que, sin una Definición de Done clara, no se puede establecer con bases que el trabajo de un Sprint ha terminado satisfactoriamente.

# 4 REFERENCIAS BIBLIOGRÁFICAS


[1] P. Sandøe, J. B. Jensen, F. Jensen y S. S. Nielsen, «Shelters Reflect but Cannot Solve Underlying Problems with Relinquished and Stray Animals—A Retrospective Study of Dogs and Cats Entering and Leaving Shelters in Denmark from 2004 to 2017,» *Animals,* vol. 9, nº 10, p. 765, 2019.

[2] A. Abdulkarim, M. A. K. Goriman Khan y E. Aklilu, «Stray Animal Population Control: Methods, Public Health Concern, Ethics, and Animal Welfare Issues,» *World's Veterinary Journal,* vol. 11, nº 3, pp. 319-326, 2021.

[3] M. Downes, M. Canty y S. More, «Demography of the pet dog and cat population on the island of Ireland and human factors influencing pet ownership,» *Prev Vet med,* vol. 92, nº 1-2, pp. 140-149, 1 Noviembre 2009.

[4] I. Mantilla, «Más de 1 millón de perros y gatos deambulan por Quito,» *El Comercio,* 15 Agosto 2022.

[5] QuitoInforma, «No más animales en las calles,» 4 Abril 2021. [En línea]. Available: http://www.quitoinforma.gob.ec/2021/04/04/no-mas-animales-en-las-calles/.

[6] Redacción El Comercio, «Adoptar por redes, más que un ‘like’,» 9 Mayo 2017.

[7] F. Faiaella y M. Ricciardi, «Gamification and learning: a review of issues and research,» *Journal of e-Learning and Knowledge Society,* vol. 11, nº 3, 2015.

[8] K. Rubin, Essential Scrum. A practical guide to the most popular agile process, Addison-Wesley, 2013.

[9] Amazon Web Services, «What is Mobile Application Development?,» 28 Marzo 2019. [En línea]. Available: https://aws.amazon.com/mobile/mobile-application-development/. [Último acceso: 24 Noviembre 2023].

[10] L. Simmons, «Front-End vs. Back-End: What’s the Difference?,» 9 Noviembre 2023. [En línea]. Available: https://www.computerscience.org/bootcamps/resources/frontend-vs-backend/. [Último acceso: 19 Noviembre 2023].

[11] Amazon Web Services, «¿Cuál es la diferencia entre el front end y back end en el desarrollo de aplicaciones?,» 2 Junio 2023. [En línea]. Available: https://aws.amazon.com/es/compare/the-difference-between-frontend-and-backend/. [Último acceso: 19 Noviembre 2023].

[12] IBM, «What is an API?,» 1 Septiembre 2020. [En línea]. Available: https://www.ibm.com/topics/api. [Último acceso: 20 Noviembre 2023].

[13] Amazon Web Services, «¿Qué es una API RESTful?,» 7 Mayo 2022. [En línea]. Available: https://aws.amazon.com/es/what-is/restful-api/. [Último acceso: 24 Noviembre 2023].

[14] Red Hat, «REST API,» 31 Julio 2023. [En línea]. Available: https://www.redhat.com/es/topics/api/what-is-a-rest-api. [Último acceso: 3 Diciembre 2023].

[15] B. Cooksey, An Introduction to APIs, Zapier, Inc, 2014.

[16] Mozilla Corporation, «HTTP request methods,» 10 Abril 2023. [En línea]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods. [Último acceso: 24 Noviembre 2023].

[17] Amazon Web Services, «¿Qué es Python?,» 2023. [En línea]. Available: https://aws.amazon.com/es/what-is/python/. [Último acceso: 24 Noviembre 2023].

[18] Python Software Foundation, «General Python FAQ,» Python docs, 23 Noviembre 2023. [En línea]. Available: https://docs.python.org/3/faq/general.html#what-is-python. [Último acceso: 24 Noviembre 2023].

[19] G. Reynolds, R. Stair, J. Byrant, M. Frydenberg, H. Greenberg y G. Schell, Principles of Information Systems, Boston: Cengage, 2021.

[20] N. Thakur y N. Gupta, «Relational and Non Relational,» *Journal of University of Shanghai for Science and Technology,* vol. 23, nº 8, pp. 117-121, 2021.

[21] MongoDB, «Introduction to MongoDB,» 2023. [En línea]. Available: https://www.mongodb.com/docs/manual/introduction/. [Último acceso: 20 Noviembre 2023].

[22] Google for Developers, «Almacenamiento en la nube para Firebase,» 17 Enero 2024. [En línea]. Available: https://firebase.google.com/docs/storage?hl=es. [Último acceso: 18 Enero 2024].

[23] Auth0 by Okta, «Introducción a los tokens web JSON,» 5 Febrero 2023. [En línea]. Available: https://auth0.com/es/learn/json-web-tokens. [Último acceso: 18 Enero 2024].

[24] M. Davis, «JOSE implementation in Python (python-jose 3.3.0),» 4 Junio 2021. [En línea]. Available: https://pypi.org/project/python-jose/. [Último acceso: 18 Enero 2024].

[25] R. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Pearson, 2018.

[26] S. M. Khan, Popular Software Architecture Used In Software Development, Kindle Direct Publishing, 2023.

[27] S. Ramírez, «Clean Architectures,» 2 Julio 2023. [En línea]. Available: https://github.com/stivenramireza/clean-architectures. [Último acceso: 4 Diciembre 2023].

[28] M. Coppola, «Qué es la arquitectura hexagonal, ventajas y desventajas,» 10 Mayo 2023. [En línea]. Available: https://blog.hubspot.es/website/que-es-arquitectura-hexagonal. [Último acceso: 19 Noviembre 2023].

[29] A. Cockburn, «Hexagonal architecture,» 4 Enero 2005. [En línea]. Available: https://alistair.cockburn.us/hexagonal-architecture/. [Último acceso: 19 Noviembre 2023].

[30] S. Ramírez, «FastAPI,» 15 Diciembre 2018. [En línea]. Available: https://fastapi.tiangolo.com/. [Último acceso: 16 Noviembre 2023].

[31] S. Ramírez, «FastAPI Learn Tutorial - User Guide,» 1 Enero 2019. [En línea]. Available: https://fastapi.tiangolo.com/tutorial/first-steps/. [Último acceso: 24 Noviembre 2023].

[32] Docker Inc., «Docker overview,» 2023. [En línea]. Available: https://docs.docker.com/get-started/overview/. [Último acceso: 20 Noviembre 2023].

[33] Docker, Inc, «Docker: The compact container platform,» 2017. [En línea]. Available: https://www.aeris-consulting.com/wp-content/uploads/2022/04/Docker.pdf. [Último acceso: 24 Noviembre 2023].

[34] Railway, «Railway Docs: About Railway,» 2 Diciembre 2023. [En línea]. Available: https://docs.railway.app/overview/about-railway. [Último acceso: 6 Enero 2024].

[35] DeployPRO, «AWS vs Railway,» 16 Septiembre 2023. [En línea]. Available: https://www.docs.deploypro.dev/glossary/aws-vs-railway. [Último acceso: 6 Enero 2024].

[36] Microsoft, «Visual Studio code: Code editing. Redefined,» 2023. [En línea]. Available: https://code.visualstudio.com/. [Último acceso: 24 Noviembre 2023].

[37] Python poetry, «Introduction,» 11 Diciembre 2019. [En línea]. Available: https://python-poetry.org/docs/. [Último acceso: 29 Diciembre 2023].

[38] Python poetry, «Basic usage,» 11 Diciembre 2019. [En línea]. Available: https://python-poetry.org/docs/basic-usage/. [Último acceso: 29 Diciembre 2023].

[39] Ł. Langa, «Black documentation. The uncompromising code formatter,» 29 Abril 2018. [En línea]. Available: https://black.readthedocs.io/en/stable/index.html#. [Último acceso: 29 Diciembre 2023].

[40] K. Schwaber y J. Sutherland, «The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game,» Noviembre 2020. [En línea]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf. [Último acceso: 16 Noviembre 2023].

[41] A. Dantorp, «Epics vs. User Stories: what’s the difference?,» 8 Julio 2021. [En línea]. Available: https://www.delibr.com/post/epics-vs-user-stories-whats-the-difference. [Último acceso: 29 Diciembre 2023].

[42] M. Rehkopf, «Stories, epics, and initiatives,» 6 Marzo 2018. [En línea]. Available: https://www.atlassian.com/agile/project-management/epics-stories-themes. [Último acceso: 29 Diciembre 2023].

[43] Á. Castellanos, «SCRUM y los puntos de historia, ¿Cómo funcionan?,» 29 Enero 2020. [En línea]. Available: https://www.incentro.com/es-ES/blog/scrum-puntos-de-historia-como-funcionan. [Último acceso: 6 Enero 2024].

[44] S. Laoyan, «Planning poker: la estrategia integral para la estimación ágil,» 19 Diciembre 2022. [En línea]. Available: https://asana.com/es/resources/planning-poker. [Último acceso: 6 Enero 2024].

[45] We Agile You, «Planning poker online | Scrum poker | We Agile You,» 19 Noviembre 2019. [En línea]. Available: https://planningpokeronline.com/. [Último acceso: 6 Enero 2024].

[46] Atlassian, «Bienvenido a Jira Software,» 31 Enero 2019. [En línea]. Available: https://www.atlassian.com/es/software/jira/guides/getting-started/introduction#what-is-jira-software. [Último acceso: 25 Noviembre 2023].

[47] GitHub, Inc, «Github Docs: Hello World,» 24 Febrero 2022. [En línea]. Available: https://docs.github.com/en/get-started/quickstart/hello-world#introduction. [Último acceso: 25 Noviembre 2023].

[48] Notion Labs, Inc, «Notion docs: The next gen of notes & docs,» 1 Diciembre 2022. [En línea]. Available: https://www.notion.so/product/docs. [Último acceso: 29 Diciembre 2023].

[49] Amazon Web Services, «¿Cuál es la diferencia entre las bases de datos relacionales y las no relacionales?,» 22 Junio 2023. [En línea]. Available: https://aws.amazon.com/es/compare/the-difference-between-relational-and-non-relational-databases/. [Último acceso: 18 Enero 2024].

[50] The Sheriff Team, «The Sheriff Documentation,» 2019. [En línea]. Available: https://thesheriff.readthedocs.io/en/latest/. [Último acceso: 28 Diciembre 2023].

[51] Mozilla, «Introducción a Django,» [En línea]. Available: https://developer.mozilla.org/es/docs/Learn/Server-side/Django/Introduction.

[52] Pallets, «Flask documentation,» [En línea]. Available: https://flask.palletsprojects.com/en/2.3.x/.

[53] VMWare Tanzu, «Spring Boot,» [En línea]. Available: https://spring.io/projects/spring-boot.

[54] D. Roth, R. Anderson y S. Luttin, «Overview of ASP.NET Core,» 10 Abril 2023. [En línea]. Available: https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-7.0.

[55] StrongLoop, IBM, «Express documentation,» 2017. [En línea]. Available: https://expressjs.com/.

[56] Google for Developers, «Cloud Firestore,» 19 Enero 2024. [En línea]. Available: https://firebase.google.com/docs/firestore?hl=es-419.

[57] Oracle, «MySQL Documentation,» 2024. [En línea]. Available: https://dev.mysql.com/doc/.

[58] The PostgreSQL Global Development Group, «PostgreSQL 16.1 Documentation,» 2024. [En línea]. Available: https://www.postgresql.org/docs/current/index.html.

[59] Desarrollo Web Digital Guide IONOS, «Git vs. SVN: ¿cuál es el mejor sistema de control de versiones?,» 12 Marzo 2020. [En línea]. Available: https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/git-vs-svn-una-comparativa-del-control-de-versiones/.

[60] «Mercurial documentation,» [En línea]. Available: https://www.mercurial-scm.org/.

[61] Atlassian, «Qué es Trello,» 2023. [En línea]. Available: https://trello.com/es/tour.

[62] C. Díaz Alcolea, «Qué es Azure DevOps,» 4 Mayo 2023. [En línea]. Available: https://openwebinars.net/blog/que-es-azure-devops/.

[63] Microsoft, «Visual Studio Community Edition,» 2024. [En línea]. Available: https://visualstudio.microsoft.com/es/vs/community/.

[64] Eclipse Foundation, «Eclipse Documentation,» 2023. [En línea]. Available: https://www.eclipse.org/documentation/.

[65] The Apache Software Foundation, «Welcome to Apache Netbeans,» 2024. [En línea]. Available: https://netbeans.apache.org/front/main/.

[66] Jetbrains, «Getting started with WebStorm,» 9 Enero 2024. [En línea]. Available: https://www.jetbrains.com/help/webstorm/getting-started-with-webstorm.html.

[67] Adaptable.io, «Adaptable | Pricing,» 2024. [En línea]. Available: https://adaptable.io/pricing.

[68] Amazon Web Services, «Características de Amazon EC2,» 2023. [En línea]. Available: https://aws.amazon.com/es/ec2/features/?trk=ec2_landing.

[69] Micorosft, «Crea soluciones en la nube con una cuenta gratuita de Azure,» 2024. [En línea]. Available: https://azure.microsoft.com/es-mx/free/.

[70] Fly.io, «Fly.io developer documentation,» [En línea]. Available: https://fly.io/docs/.

[71] Google for Developers, «Programa gratuito de Google Cloud,» 3 Agosto 2022. [En línea]. Available: https://cloud.google.com/free/docs/free-cloud-features?hl=es#always-free-usage-limits.

[72] Oracle, «Cloud Documentation,» [En línea]. Available: https://docs.oracle.com/en/cloud/get-started/index.html.

# 5 ANEXOS

ANEXO I. Tabla comparativa de herramientas para el desarrollo del Backend [30] [51] [52] [53] [54] [55].

| Características | Django | Flask | FastAPI | Spring Boot | ASP.NET CORE | Express |
|---|---|---|---|---|---|---|
| **Lenguajes de programación** | Framework web de alto nivel escrito en Python. | Framework web de alto nivel escrito en Python. | Framework para el desarrollo web basado en Python. | Framework de desarrollo de aplicaciones basado en Java. | Framework de desarrollo multiplataformas con alta compatibilidad con lenguajes de programación como C#. | Framework diseñado para Node.js. Es compatible con lenguajes de programación como JavaScript. |
| **Framework y librerías** | Existe una gran diversidad de frameworks que permiten agilizar y complementar el desarrollo web como integraciones a motores de búsqueda, funciones y tareas síncronas y asíncronas entre otras. | Altamente flexible lo que permite integrarse con otros frameworks de Python. | Es ligero y fácil de integrarse con otros frameworks de python que facilitan el desarrollo web. | Limitada a frameworks de Java y basado en Spring. | Cuenta con amplia gama de frameworks para integración con base de datos, seguridad y autenticación entre otras. | Puede integrar con frameworks de Node.js |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Rendimiento y escalabilidad** | Es altamente escalable para aumentar el tráfico al agregar hardware en cualquier nivel: servidores de cache, servidores de bases de datos o servidores de aplicación | Según el proyecto, este es altamente escalable, sobre todo para aplicaciones web medianas. | Es altamente escalable y capaz de manejar trabajos exigentes como gestión de BDD | Enfocado en el rendimiento y escalabilidad de aplicaciones. | ASP.NET Core es diseñado para ofrecer un alto rendimiento y escalabilidad. | Diseñado y conocido por su alto rendimiento y escalabilidad. |
| **Integración con bases de datos** | Integración con base de datos bastante sencilla y cuenta con frameworks que hacen que esta sea segura. Además, cuenta con ORM que facilita la gestión y consultas a la BDD | No tiene integración ORM, pero es compatible con una gran variedad de base de datos ya que posee SQLAIChemy. | Compatible con una gran variedad de base de datos además que dé puede integrarse con frameworks que permite que este sea sencillo como SQLAIchemy o ORM. | Cuenta con Spring Data que facilita la persistencia en bases de datos tanto SQL como no SQL | Dispone de una gran flexibilidad para la integración con diferentes bases de datos así mismo cuenta como un ORM que le permite interactuar fácilmente con base de datos relaciones. | Puede ser integrados con frameworks para la integración con BDD ya que este no cuenta con uno nativo |
| **API y servicios web** | Tiene una alta compatibilidad con | Cuenta con diversas | Este es un framework | Posee herramientas | Usado por varios desarrolladores | Framework diseñado para la |

| | | | | | | |
|---|---|---|---|---|---|---|
| | framework y creación de APIs, además, facilita de serialización y deserialización de objetos. | funciones para crear y consumir Servicios Web ya que este es muy flexible. | diseñado para el desarrollo de APIs lo cual lo hace altamente flexible y compacto para esta tarea. | para el desarrollo de APIRest y creación de ENDPOINTS. | para la creación de APIs, además que cuenta con frameworks integrados para el desarrollo de estas. | creación de APIs y servicios web |
| **Seguridad** | Cuenta con una alta seguridad y garantiza la inyección en las bases de datos, posee por defecto algoritmos de encriptación, restricciones de control de acceso. | Cuenta con funcionalidades que ayudan a mejorar la seguridad, pero este depende mucho del desarrollador y su nivel de experiencia. | Cuenta con herramientas ya integradas que mejoran y garantizan la seguridad además de que permite incorporar lógicas de autenticación. | Incluye un módulo de seguridad. | Cuenta con una sólida seguridad y diferentes modos de autenticación. | NO incluye una capa de seguridad integrada, se puede hacer por frameworks |
| **Documentación y recursos de aprendizaje** | Cuenta con documentación oficial en su sitio web. Además, existe una gran fuente de | Cuenta con documentación oficial en su sitio web. Además, existe | Existe gran cantidad de documentación lo cual lo hace | Cuenta con documentación oficial además de foros de ayuda. | Existe una gran cantidad foros y documentación oficial de Microsoft. | Cuenta con documentación actualizada. |

| | | | | | | |
|---|---|---|---|---|---|---|
| | aprendizaje de este al ser muy popular. | una gran fuente de aprendizaje de este al ser muy popular. | fácil de aprender. | | | |
| **Comunidad y soporte** | Cuenta con una comunidad muy activa con foros de consultas, actualizaciones entre otras. | Cuenta con una amplia comunidad de desarrolladores que se mantienen activos. | Cuenta con una amplia comunidad de desarrolladores que se mantienen activos. | Cuenta con un repositorio en git que mantiene las últimas actualizaciones del framework. | Cuenta con una comunidad activa | Cuenta con foros en línea que mantiene a su comunidad activa |
| **Experiencia previa y preferencias** | Nulo | Nulo | Medio | Medio Bajo | Medio | Medio |

ANEXO II. Tabla comparativa de Bases de Datos [21] [56] [57] [58].

| Características | MongoDB | Firestore | MySQL | PostgreSQL |
|---|---|---|---|---|
| **Modelo de datos** | Orientado a documentos y colecciones, archivos JSON | Orientado a documentos y colecciones, archivos JSON | Modelo relacional con Tablas | Modelo relacional con Tablas |
| **Escalabilidad** | Escalabilidad Horizontal | Escalabilidad Horizontal | Escalabilidad Vertical | Escalabilidad Vertical |
| **Rendimiento** | Manejo de índices, mayor velocidad de lectura y escritura | Manejo de índices, mayor velocidad de lectura y escritura | Bueno, especialmente en transacciones | Bueno, especialmente en transacciones y consultas complejas |
| **Flexibilidad del esquema** | Alta, sin esquemas predefinidos | Alta, sin esquemas predefinidos | Baja, esquemas predefinidos | Baja, esquemas predefinidos |
| **Comunidad y soporte** | Amplia comunidad y soporte oficial MongoDB | Amplia comunidad y soporte oficial de Google | Amplia comunidad y soporte oficial de Oracle | Amplia comunidad y soporte oficial PostgreSQL |
| **Precio / Licencia** | Open source, con opciones comerciales | Basado en consumo, con planes de pago | General Public License, Open source, con opciones comerciales | PostgreSQL License, Open source |
| **Experiencia previa y preferencias** | Medio | Medio | Medio | Alto |

ANEXO III. Tabla comparativa de herramientas de Control de Versiones [59] [60].

| Características | Git | SVN | Mercurial |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Funcionalidades básicas** | Creación y clonación de repositorios, creación y fusión de ramas, commits, etc. | Crear y clonar repositorios, crear y fusionar ramas, hacer commits, etc. | Creación y fusión de ramas, reversión de cambios |
| **Arquitectura y modelo de datos** | Arquitectura distribuida | Arquitectura centralizada | Arquitectura distribuida |
| **Rendimiento** | Rápido y eficiente para gestión de grandes repositorios (clonar, commits, fusionar, etc.) | Velocidad menor que Git en las operaciones básicas. | Alto rendimiento y escalabilidad en proyectos grandes con varios colaboradores. |
| **Soporte para ramas y fusiones** | Permite el trabajo en paralelo, ramas, fusiones automáticas, tags. etc. | Su modelo de datos lineal con gestión de ramas y fusiones compleja. | Capacidades avanzadas de ramificación y fusión. Incluye soporte para ramas nombradas, marcadores, múltiples cabezas. |
| **Integración y compatibilidad** | Compatible con GitHub, GitLab, BitBucket. | Compatible con GitHub, GitLab, BitBucket | Compatible con GitHub y BitBucket. (usando extensiones). |
| **Facilidad de uso y curva de aprendizaje** | Curva de aprendizaje empinada, aunque cuentan con varios herramientas y recursos. | Curva de aprendizaje menos empinada que Git | Curva de aprendizaje no muy alta en comparación con otras alternativas de versionamiento. |
| **Seguridad y control de acceso** | Autenticación y autorización para el control de acceso a repositorios, encriptación de datos y firma digital de commits. | Autenticación y autorización para el control de acceso a repositorios. Permite la encriptación de datos y firma digital de commits. | Autenticación para el acceso a repositorios y encriptación de datos. |

| | | | |
|---|---|---|---|
| **Comunidad y soporte** | Gran comunidad de usuarios y desarrolladores. | Comunidad activa, aunque su nivel de popularidad ha disminuido en favor de Git. | Comunidad activa que proporciona documentación y recursos de apoyo. |

ANEXO IV. Tabla comparativa de herramientas de Gestión de Proyectos [46] [48] [61] [62].

| Características | Jira | Trello | Azure DevOps | Notion |
|---|---|---|---|---|
| **Seguimiento de tareas y gestión de proyectos** | Amplias capacidades de seguimiento de tareas y gestión de proyectos. | Funciones básicas de seguimiento de tareas y gestión de proyectos. | Completo conjunto de herramientas para la gestión de proyectos. | Alta personalización y plantillas para la gestión de proyectos y tareas. |
| **Tableros y vistas visuales** | Tableros personalizables y vistas Kanban y Scrum. | Tableros Kanban | Tableros Kanban, y vistas personalizables | Plantillas de Scrum, sprint y vistas visuales personalizables. |
| **Colaboración y comunicación** | Funcionalidades de colaboración y comunicación sólidas, como comentarios y notificaciones. | Comentarios y notificaciones básicas. | Funciones de colaboración y comunicación completas, incluyendo chat integrado. | Funciones de colaboración y comunicación nativas limitadas. |
| **Integración con herramientas de versionamiento** | Integración nativa con sistemas de control de versiones, como Git, bibucket, etc. | No tiene integración con herramientas de versionamiento nativa, es posible por medio de power-ups. | Integración profunda con repositorios Git y otras herramientas de DevOps. | No tiene integración con herramientas de versionamiento nativa. |

| | | | | |
|---|---|---|---|---|
| **Gestión de dependencias** | Posibilidad de gestionar dependencias y enlaces entre tareas. | No tiene una funcionalidad nativa para gestionar dependencias, es posible mediante power-ups. | Soporte para gestionar dependencias y relaciones entre elementos de trabajo. | Soporte configurable y personalizable para gestionar dependencias y relaciones entre elementos. |
| **Informes y métricas** | Amplias opciones de generación de informes y seguimiento de métricas. | Funcionalidades de informes y métricas limitadas. | Generación de informes y paneles de control personalizables. | No tiene control y generación de informes nativas. |
| **Seguridad y control de acceso** | Opciones de configuración de seguridad y control de acceso granulares. | Funcionalidades básicas de seguridad y control de acceso. | Control de acceso y políticas de seguridad personalizables. | Control de acceso. |
| **Comunidad y soporte** | Gran comunidad y amplio soporte de Atlassian, con foros y recursos en línea. | Comunidad y soporte más limitados. | Amplia comunidad y soporte de Microsoft, con documentación y recursos en línea. | Amplia comunidad, Plantillas y recursos en línea. |
| **Enfoque de Proyectos** | Muy adecuado para equipos de desarrollo y gestión de proyectos complejos. | Ideal para equipos más pequeños y proyectos sencillos. | Adecuado para equipos que utilizan otras herramientas de Microsoft y Azure. | Ideal para la centralización de documentos. |

ANEXO V. Tabla comparativa de IDE's o Editores de código [36] [63] [64] [65] [66].

| Característica | VS Code | VS Community | Eclipse | WebStorm | NetBeans |
|---|---|---|---|---|---|
| **Compatibilidad con varios lenguajes de programación.** | Alta compatibilidad | Amplia compatibilidad con varios lenguajes | Variada compatibilidad con plugins | Enfocado en desarrollo web. | Alta compatibilidad con plugins y librerías |
| **Facilidad de uso y experiencia de usuario** | Interfaz limpia, intuitiva, actualizaciones constantes. | Interfaz intuitiva con herramientas para desarrollo y depuración. | Interfaz personalizable, experiencia depende del lenguaje. | Interfaz intuitiva, familiar para usuarios JetBrains. | Interfaces sencillas e intuitivas |
| **Autocompletado y sugerencias de código** | Autocompletado con IntelliSense. | Dispone de IntelliSense. | Predicción y sugerencias inteligentes. | Detecta contexto, predicción ágil. | Predicción y sugerencia según contexto |
| **Gestión de proyectos y archivos** | Interfaz sencilla. | Interfaz sencilla. | Interfaz sencilla. | Interfaz sencilla. | Interfaz sencilla. |
| **Integración con herramientas externas/Soporte de complementos y extensiones** | Mercado de extensiones para integración fácil. | Requiere experiencia para integración. | Diversas herramientas con cierto nivel de complejidad. | Fácil integración y SPACE Marketplace. | Diversas herramientas con cierto nivel de complejidad |
| **Rendimiento y consumo de recursos** | Ligero consumo de recursos. | Alto consumo de recursos. | Medio consumo de recursos. | Mínimo consumo de recursos. | Mínimo consumo de recursos |

ANEXO VI. Tabla comparativa de servicios de Hosting [62] [67] [68] [69] [70] [71] [72].

| Características | Adaptable | AWS EC2 | Azure app service | Azure VM | Fly.io | Google App Engine | Oracle Cloud |
|---|---|---|---|---|---|---|---|

| **Tipo** | Platform as a Service | Infrastructure as a Service | Platform as a Service | Infrastructure as a Service | Platform as a Service | Platform as a Service | Infrastructure as a Service |
|---|---|---|---|---|---|---|---|
| **RAM disponible** | 256 MB | 1 GB | 1 GB | 1 GB | 256 MB (2 instancias) | 128 MB | 200 GB |
| **Almacenamiento** | 1 GB sin persistencia | Integrado y aplicable a otros servicios de Amazon | 1 GB | 2 GB sin persistencia | 1 GB persistente | 1 GB | 1 Gb para cada 2 VMs (AMD CPU) 24 GB para 4 VMs (ARM CPU) |
| **Limitaciones** | 1 contenedor por app y 1 vCPU | Gratis solo por un año, 750 horas por mes | 60 minutos de uso de CPU por dia, compartido | Gratis solo por un año, 750 horas de VM | Todos los panes requieren método de pago | Requiere método de pago | AMD CPUs limitado a 1/8 de 2 CPUs por VM, Requiere método de pago |

## ANEXO VII. Enlace API

Enlace para acceder a la API y a la documentación de cada uno de sus endpoints: https://pawsitive-backend-gmc-production.up.railway.app/

## ANEXO VIII. Enlace Proyecto Notion

Enlace a la documentación del proyecto en Notion: https://forested-rhubarb-1ab.notion.site/Documentaci-n-TIC-2023-GMC-bb5af20f6b2d499e9c19ae16fa791f48